Notes on Basefold

Collected research notes from sec-bit/mle-pcs

July 2026

Contents

Chapter 1

Basefold

1.1 Contents

1.2 Overview

1.2.1 What is Basefold?

Basefold is a new *field-agnostic* Polynomial Commitment Scheme (PCS) for multilinear polynomials that has $O(\log^2(n))$ verifier costs and $O(n \log n)$prover time.

Basefold PCS mainly combines two types of technique, one is FRI IOPP (Fast Reed-Solomon Interactive-Oracle Proof of Proximity), and the other is sumcheck. It shows that FRI and sumcheck have similar interactive structures and can run in parallel and share the verifier randomness.

1.2.2 Why Basefold?

The following are several advantages of Basefold compared with other SNARKs.

1. **Random Foldable Code(RFCs).** RFCs are a special type of Reed-Muller code. It introduces recursion and foldable into the encoding phase, allowing for the flexible handling of messages of varying sizes. Unlike the RS codes, RFCs are foldable and have $O(n \log n)$ encoding time over any sufficiently large field. The foldable attribute is the key to achieving the field-agnosticism.
2. **Field-Agnosticism.** Basefold PCS could work over any sufficiently large finite field. ****It is a major boon to SNARK. For example, suppose an application is confined to the finite field $\mathbb{F}_p$ but the SNARK is defined over a finite field $\mathbb{F}_q$. In this case, the "modulo p" operation is explicitly encoded into the circuit and needs to be invoked for each multiplication gate. A field-agnostic PCS can deal with any large finite field, making it ideal as it enables the SNARK to avoid some large overhead associated with compatibility.
3. **Efficiency.** Basefold IOPP can be used for any linear code and Basefold PCS is very efficient. It is 2-3 times faster than prior multilinear PCS constructions from FRI (like ZeromorphFri.) when defined over the same finite field.

1.2.3 How Basefold?

There is a brief introduction of the overall Basefold that contains four phases:

- Encoding phase. The prover encodes its message $\mathbf{f}$ using *random foldable code* and gets the codeword $\mathbf{c}$.
- Commit phase. The prover commits the codeword $\mathbf{c}$ by Merkle commitment and generates the Merkle root π_d as the commitment of $\mathbf{c}$. The further process is similar to FRI which has many rounds, each round generates a new vector $\mathbf{c}_i$ and a commitment π_i of $\mathbf{c}_i$. Finally, The prover sends all the commitments $(\pi_d, ..., \pi_0)$ as oracles to the verifier.
- Query phase. The verifier queries a few points to check the consistency of these commitments $(\pi_d, ..., \pi_0)$.
- Evaluate phase. The verifier runs Basefold IOPP and sumcheck in parallel to check the commitment $\pi_d = commit(Enc(\mathbf{f}))$ and the evaluation $f(\mathbf{z}) = y$.

1.3 Background

1.3.1 Interactive Oracle Proof (IOP)

IOP is a cryptographic protocol that allows a prover to convince a verifier about the correctness of a statement without revealing the underlying data.

A k-round public coin IOP runs as follows:

Initially, the prover **sends an oracle string π_0 to the verifier. Then, in each round $i \in [1, k]$, the verifier sends a random challenge α_i to the prover, and the prover replies with a new oracle string π_i. After** k rounds of communications, the verifier **queries some entries of those oracle strings** $\pi_0, \pi_1, ..., \pi_k$ and outputs a bit b that indicates if the verifier has been convincing.

1.3.2 Interactive Oracle Proof of Proximity (IOPP)

An IOPP is a special proof system for proofing that a ***committed vector*** over a field $\mathbb{F}$ is close to a ***codeword*** in some linear error-correcting code.

1.3.3 FRI

FRI stands for Fast Reed-Solomon Interactive oracle proof of proximity. It allows us to prove that the evaluations of a given polynomial p, over a domain D_0, correspond to the evaluations of a low-degree polynomial.

The main idea of FRI is a low-degree test and it executes by "split and fold". It first splits the original polynomial p into even and odd parts, and using a random β folds the two parts. Then the folded polynomial is the half degree of the original polynomial.

Let us see an example of "split and fold".

let $p(x) = 1 + 2x + 3x^2 + 4x^3$

then split $p(x)$ into even part $p_e(x^2)$ and odd part $p_o(x^2)$.

$p_e(x^2) = 1 + 3x^2$

$p_o(x^2) = 2 + 4x^2$

We can see that: $p(x) = p_e(x^2) + xp_o(x^2)$

Then, we using a random β fold $p(x)$ and get a new polynomial $p^{(1)}(x) = p_e(x) + \beta p_o(x) = 1 + 2\beta + (3 + 4\beta)x$.

We can see that the degree of the new polynomial $p^{(1)}(x)$ is half the degree of $p(x)$.

The split and fold process can be recursively executed until the final folded polynomial which is a constant polynomial.

1.3.4 Polynomial Commitment Scheme (PCS)

A polynomial commitment scheme (PCS) is a cryptographic primitive that allows a prover to commit to a polynomial $f \in \mathbb{F}[x]$ of degree d using a short commitment and later, the prover proves the committed polynomial satisfying $f(\alpha) = \beta$. It is a building block for many cryptographic applications.

A PCS consists of a tuple of algorithms (Setup, Commit, Open, Eval).

- *Setup* takes security parameters and other information of witness as input and outputs public parameters.
- *Commit* takes a polynomial f as input and outputs a commitment C.
- *Open* takes a commitment C and a polynomial as input and outputs a bit b. (b indicates whether $C = commit(f)$).
- *Eval* takes a commitment C, an evaluation point $\mathbf{z}$, a value y and a polynomial f (only the prover knows) as input. Then, the prover wants to convince the verifier that f is an opening to C and $f(\mathbf{z}) = y$. The verifier outputs a bit b indicating whether it has been convinced.

1.4 Linear Code

In coding theory, a linear code is an error-correcting code for which any linear combination of codewords is also a codeword.

1.4.1 Reed-Solomon Code (RS code)

Reed-Solomon is a type of error-correcting code that transforms a message of size k into a codeword of size n. The **code rate** ρ is defined as $\rho = \frac{k}{n}$.

The encoding process is associated with a generator matrix, such that encoding of a vector $\mathbf{v} \in \mathbb{F}^k$ is $\mathbf{v} \cdot G$.

The encoding process uses the following equation:

$\mathbf{c} = \mathbf{v} \cdot G$

Let's see an example:

Suppose we want to encode a message of size 4: $\mathbf{m} = (m_0, m_1, m_2, m_3)$, and the generator matrix G of size 4×8:

$$G = \begin{bmatrix} g_{0,0} & g_{0,1} & g_{0,2} & g_{0,3} & g_{0,4} & g_{0,5} & g_{0,6} & g_{0,7} \\ g_{1,0} & g_{1,1} & g_{1,2} & g_{1,3} & g_{1,4} & g_{1,5} & g_{1,6} & g_{1,7} \\ g_{2,0} & g_{2,1} & g_{2,2} & g_{2,3} & g_{2,4} & g_{2,5} & g_{2,6} & g_{2,7} \\ g_{3,0} & g_{3,1} & g_{3,2} & g_{3,3} & g_{3,4} & g_{3,5} & g_{3,6} & g_{3,7} \end{bmatrix}$$

Then, we compute $\mathbf{c} = \mathbf{m} \cdot G$ to get the codeword: $\mathbf{c} = (c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7)$.

1.4.2 Reed-Muller Code (RM code)

Reed-Muller code also is a type of error-correcting code used in coding theory. Here we introduce a special type of RM code named foldable RM code whose generator matrices have a foldable structure. A foldable RM code includes generator matrices($G_0, ..., G_{d-1}$) and diagonal matrices($T_0, ..., T_{d-1}$), ($T_0^{'}, ..., T_{d-1}^{'}$). The foldable structure means that the generator matrix G_{i+1} can be generated from the previous generator matrix G_i and two diagonal matrices T_i and $T_i^{'}$, satisfying the following formula:

$$G_{i+1} = \begin{bmatrix} G_i & G_i \\ G_i \cdot T_i & G_i \cdot T_i^{'} \end{bmatrix}$$

Note that the elements on those diagonal matrices are distinct from each other.

1.4.3 Random Foldable code (RFC)

The RFC, a variant of the foldable RM code, follows a recursive approach to generate its generator matrix. RFC is foldable and has $O(n \log n)$ encoding time over any sufficiently large field, better than the traditional encoding method with $O(n^2)$ encoding time.

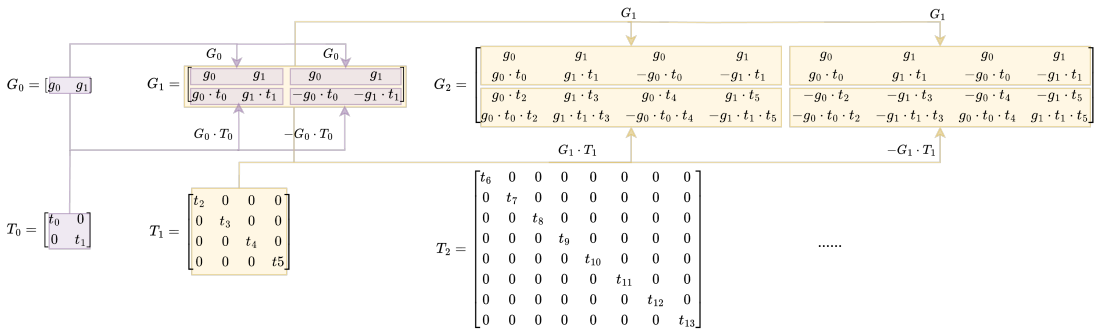
RFC introduces two modifications on diagonal matrices:

1. It assigns T_i as a diagonal matrix with entries drawn uniformly at random from $\mathbb{F}^{\times}$($\mathbb{F}^{\times}$ denotes $\mathbb{F} \setminus \{0\}$).
2. It sets $-T_i$ as T_i'.

The generator matrices are created as follows:

$$G_{i+1} = \begin{bmatrix} G_i & G_i \\ G_i \cdot T_i & G_i \cdot -T_i \end{bmatrix}$$

The following graph shows an example of the process of creating generator matrices recursively.



1.5 Basefold

1.5.1 Basefold Encoding

- **Encoding phase**

Basefold encoding algorithm takes the original message $\mathbf{m} \in \mathbb{F}^{k_d}$ as input and outputs codeword $\mathbf{c} \in \mathbb{F}^{n_d}$. d denotes the logarithm of $\mathbf{m}$'s length. If $length(\mathbf{m}) = 4$, then $d = \log 4 = 2$. There are initial generator matrix $\mathbf{G}_0$ and diagonal matrices $(T_0, T_1, ..., T_{d-1})$ as public parameters.

Basefold uses a recursive way to encode messages rather than directly compute $\mathbf{c} = \mathbf{m} \cdot G$. It split the message $\mathbf{m} = (\mathbf{m}_l, \mathbf{m}_r)$, $\mathbf{m}_l$ represents the first half of $\mathbf{m}$, and $\mathbf{m}_r$ represents the second half of $\mathbf{m}$. Then, it encodes $\mathbf{m}_l$ and $\mathbf{m}_r$, respectively.

In the encoding phase, there are two message statuses in the algorithm:

1. The message $\mathbf{m}$ is the smallest unit of encoding, the algorithm will return the encoding result $Enc_0(\mathbf{m}) = \mathbf{m} \cdot G_0$.
2. The message $\mathbf{m}$ is not the smallest unit, then further split $\mathbf{m}$ into $(\mathbf{m}_l, \mathbf{m}_r)$. Then the algorithm can reduce the $Enc_d(\mathbf{m})$ into $Enc_{d-1}(\mathbf{m}_l)$ and $Enc_{d-1}(\mathbf{m}_r)$ such that:

$Enc_d(\mathbf{m}) = (Enc_{d-1}(\mathbf{m}_l) + \mathbf{t} \circ Enc_{d-1}(\mathbf{m}_r), Enc_{d-1}(\mathbf{m}_l) - \mathbf{t} \circ Enc_{d-1}(\mathbf{m}_r))$

, where $\mathbf{t} = \text{diag}(T_{d-1})$ represents a vector whose elements are diagonal elements of T_{d-1}. For example, let $T = \begin{bmatrix} t_0 & 0 \\ 0 & t_1 \end{bmatrix}$, then $\text{diag}(T) = [t_0, t_1]$.

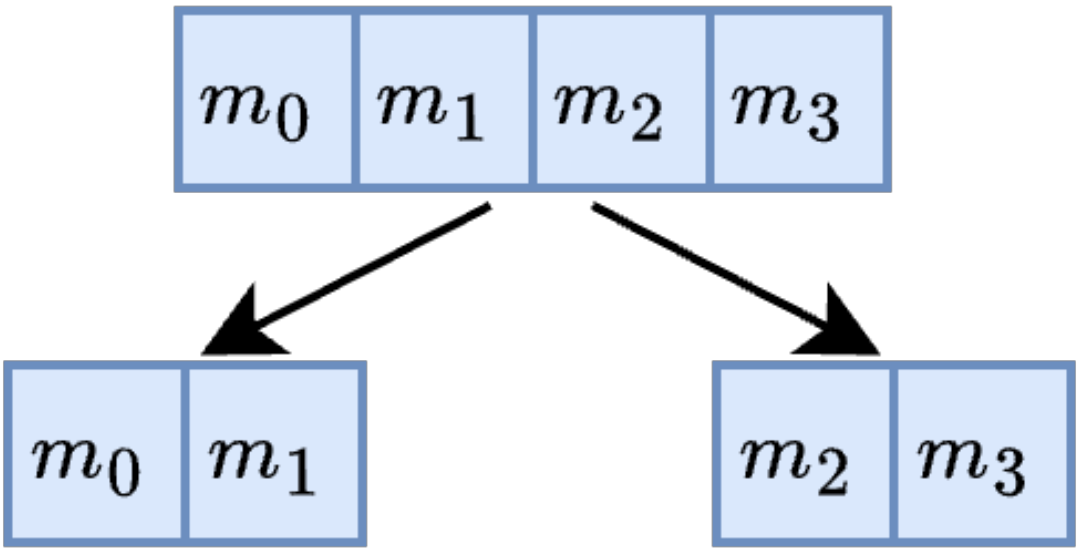
Let's see an example of encode a message $\mathbf{m} = (m_0, m_1, m_2, m_3) \in \mathbb{F}^{k_2}$.

$$\mathbf{m} = (m_0, m_1, m_2, m_3) \xrightarrow{\text{encoding}} \mathbf{c} = (c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7)$$

Given the public parameters $G_0 = (g_0, g_1)$, $T_0 = \begin{bmatrix} t_0 & 0 \\ 0 & t_1 \end{bmatrix}$, $T_1 = \begin{bmatrix} t_2 & 0 & 0 & 0 \\ 0 & t_3 & 0 & 0 \\ 0 & 0 & t_4 & 0 \\ 0 & 0 & 0 & t_5 \end{bmatrix}$. Therefore, $\mathbf{t}_0 = [t_0, t_1]$, $\mathbf{t}_1 = [t_2, t_3, t_4, t_5]$.

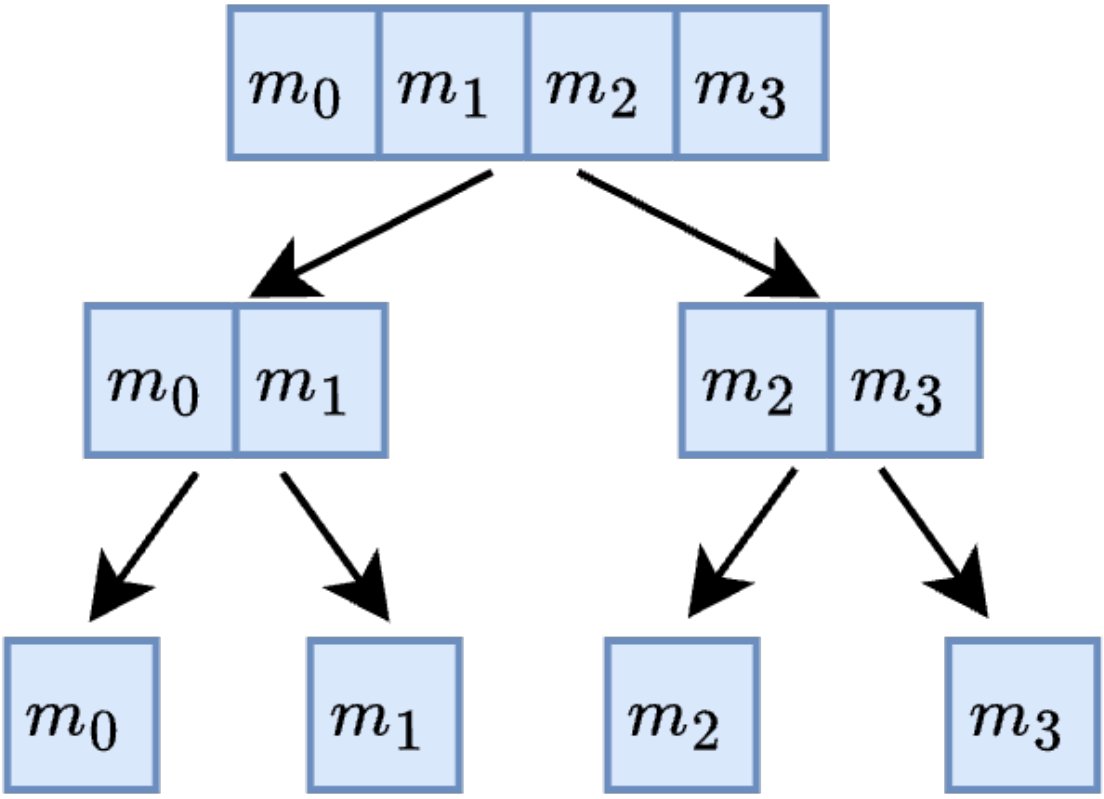
Our goal is encode $\mathbf{m}$ and get $Enc_2(\mathbf{m})$.

At first, $\mathbf{m} = (m_0, m_1, m_2, m_3)$ is not the smallest encoding unit. Therefore, we split (m_0, m_1, m_2, m_3) into $\mathbf{m}_l = (m_0, m_1)$ and $\mathbf{m}_r = (m_2, m_3)$.



Let $\mathbf{l} := Enc_1(\mathbf{m}_l)$, $\mathbf{r} := Enc_1(\mathbf{m}_r)$. We get : $Enc_2(\mathbf{m}) = (\mathbf{l} + \mathbf{t}_1 \circ \mathbf{r}, \mathbf{l} - \mathbf{t}_1 \circ \mathbf{r})$

At this point, we can recursively compute $Enc_1(\mathbf{m}_l)$ and $Enc_1(\mathbf{m}_r)$. We further split (m_0, m_1) into (m_0), (m_1) and split (m_2, m_3) into (m_2), (m_3).



Due to all (m_0), (m_1), (m_2), (m_3) are the smallest units. We then encode the them and get $Enc_0(m_0), Enc_0(m_1), Enc_0(m_2), Enc_0(m_3)$.

$Enc_0(m_0) = m_0 \cdot G_0 = [m_0 g_0 \quad m_0 g_1]$

$Enc_0(m_1) = m_1 \cdot G_0 = [m_1 g_0 \quad m_1 g_1]$

$Enc_0(m_2) = m_2 \cdot G_0 = [m_2 g_0 \quad m_2 g_1]$

$Enc_0(m_3) = m_3 \cdot G_0 = [m_3 g_0 \quad m_3 g_1]$

And then we can compute $\mathbf{l} = Enc_1(m_0, m_1), \mathbf{r} = Enc_1(m_2, m_3)$ using $Enc_0(m_0), Enc_0(m_1), Enc_0(m_2), Enc_0(m_3)$.

$\mathbf{l} = (Enc_0(m_0) + \mathbf{t}_0 \circ Enc_0(m_1), Enc_0(m_0) - \mathbf{t}_0 \circ Enc_0(m_1)) = [m_0 g_0 + m_1 g_0 t_0 \quad m_0 g_1 + m_1 g_1 t_1 \quad m_0 g_0 - m_1 g_0 t_0 \quad m_0 g_1 - m_1 g$

$\mathbf{r} = (Enc_0(m_2) + \mathbf{t}_0 \circ Enc_0(m_3), Enc_0(m_2) - \mathbf{t}_0 \circ Enc_0(m_3)) = [m_2 g_0 + m_3 g_0 t_0 \quad m_2 g_1 + m_3 g_1 t_1 \quad m_2 g_0 - m_3 g_0 t_0 \quad m_2 g_1 - m_3 g$

Finally, we compute $Enc_2(\mathbf{m}) = Enc_2(m_0, m_1, m_2, m_3)$.

$Enc_2(m_0, m_1, m_2, m_3)$
$= [\mathbf{l} + \mathbf{t}_1 \circ \mathbf{r}, \mathbf{l} - \mathbf{t}_1 \circ \mathbf{r}]$

$$= \begin{bmatrix} m_0g_0 + m_1g_0t_0 + m_2g_0t_2 + m_3g_0t_0t_2 \\ m_0g_1 + m_1g_1t_1 + m_2g_1t_3 + m_3g_1t_1t_3 \\ m_0g_0 - m_1g_0t_0 + m_2g_0t_4 - m_3g_0t_0t_4 \\ m_0g_1 - m_1g_1t_1 + m_2g_1t_5 - m_3g_1t_1t_5 \\ m_0g_0 + m_1g_0t_0 - m_2g_0t_2 - m_3g_0t_0t_2 \\ m_0g_1 + m_1g_1t_1 - m_2g_1t_3 - m_3g_1t_1t_3 \\ m_0g_0 - m_1g_0t_0 - m_2g_0t_4 + m_3g_0t_0t_4 \\ m_0g_1 - m_1g_1t_1 - m_2g_1t_5 + m_3g_1t_1t_5 \end{bmatrix} = \begin{bmatrix} c_0 & c_1 & c_2 & c_3 & c_4 & c_5 & c_6 & c_7 \end{bmatrix}$$

At this point, we have obtained the codeword $\mathbf{c} = (c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7)$. The result of using the recursive encoding method is the same as the result of directly calculating $\mathbf{c} = \mathbf{m} \cdot \mathbf{G}_2$.

In this case, the message of size 4 and the codeword of size 8, thus the code rate equal to $\frac{message\ size}{codeword\ size} = \frac{1}{2}$.

1.5.2 Basefold IOPP

Basefold IOPP generalized the FRI IOPP to any foldable linear code. The Goal of Basefold IOPP is to ensure the verifier checks an oracle π_d sent by the prover is close to a codeword C_d.

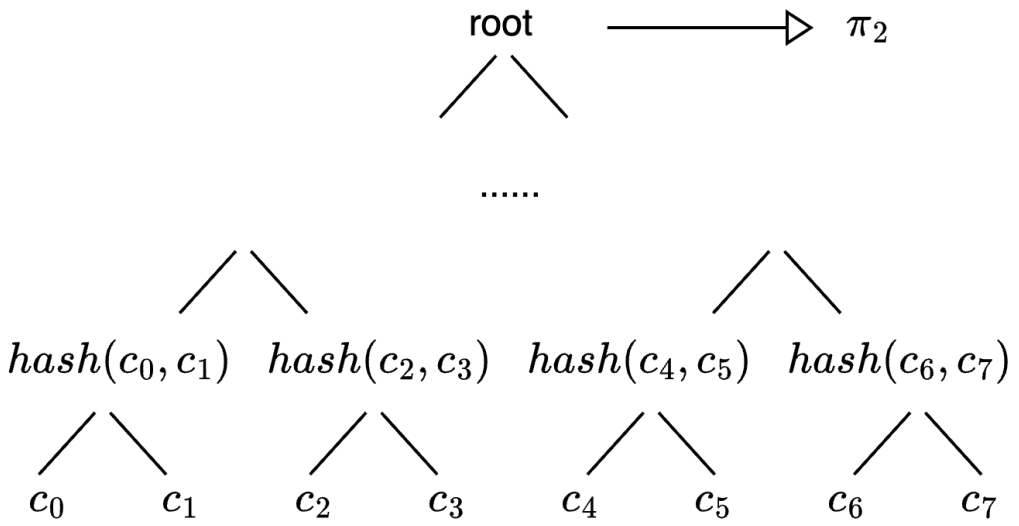
Basefold IOPP consists of two phases: Commit phase and Query phase.

Commit phase: **the prover commits a polynomial f and generates a list of oracles $(\pi_d, ..., \pi_0)$ given the verifier's folding challenge $r_i (0 \leq i < d)$.

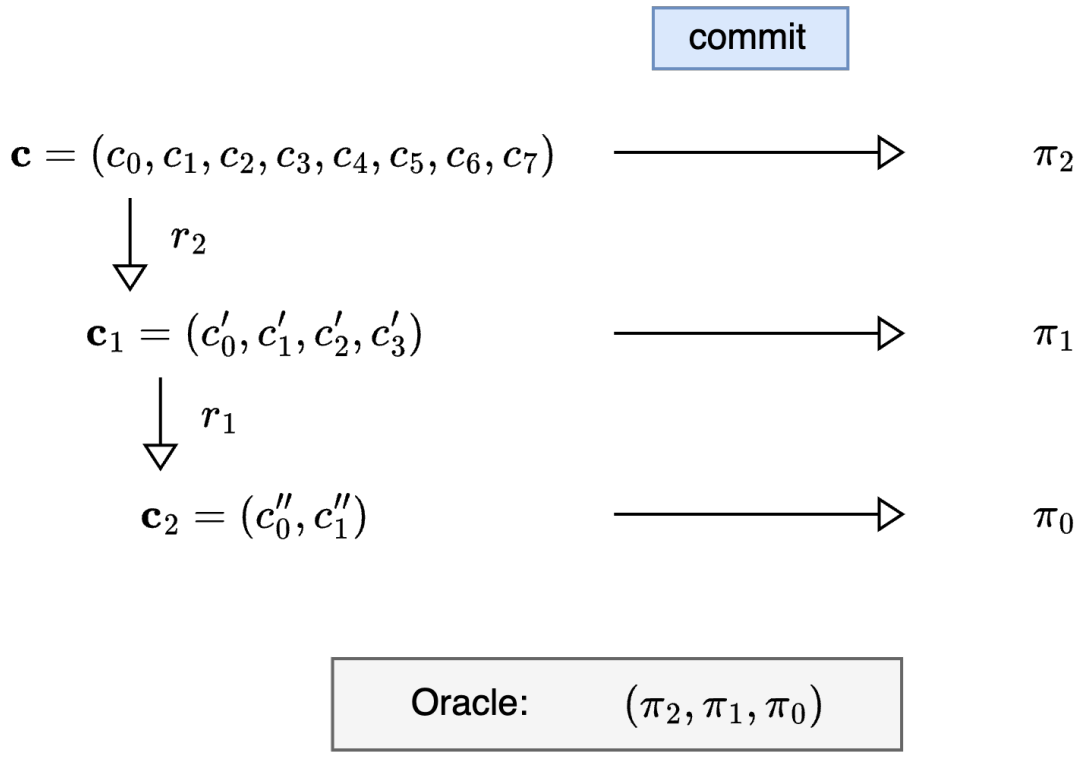
Query phase: the verifier samples a query index to check the consistency between oracles.

- Commit phase

Basefold IOPP uses Merkle commitment to generate commitments. The graph below illustrates the process of deriving oracle π_2 from $\mathbf{c} = (c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7)$. The Merkle root is exactly the commitment of $\mathbf{c}$.



Continuing from the previous example, let's demonstrate how the prover generates a list of oracles (π_2, π_1, π_0) given the verifier's folding challenges $r_i (0 \leq i < 2)$. The prover first computes the Merkle root of $\mathbf{c}$ and gets the commitment π_2. Next, upon receiving the folding challenge r_2, the prover can fold the vector $\mathbf{c}$ into $\mathbf{c}_1$, which is half the size of $\mathbf{c}$. Then commit $\mathbf{c}_1$ to get π_1. And using another folding challenge r_1 fold $\mathbf{c}_1$ to get $\mathbf{c}_2$ which is half the size of $\mathbf{c}_1$. Then commit $\mathbf{c}_2$ to get π_0.

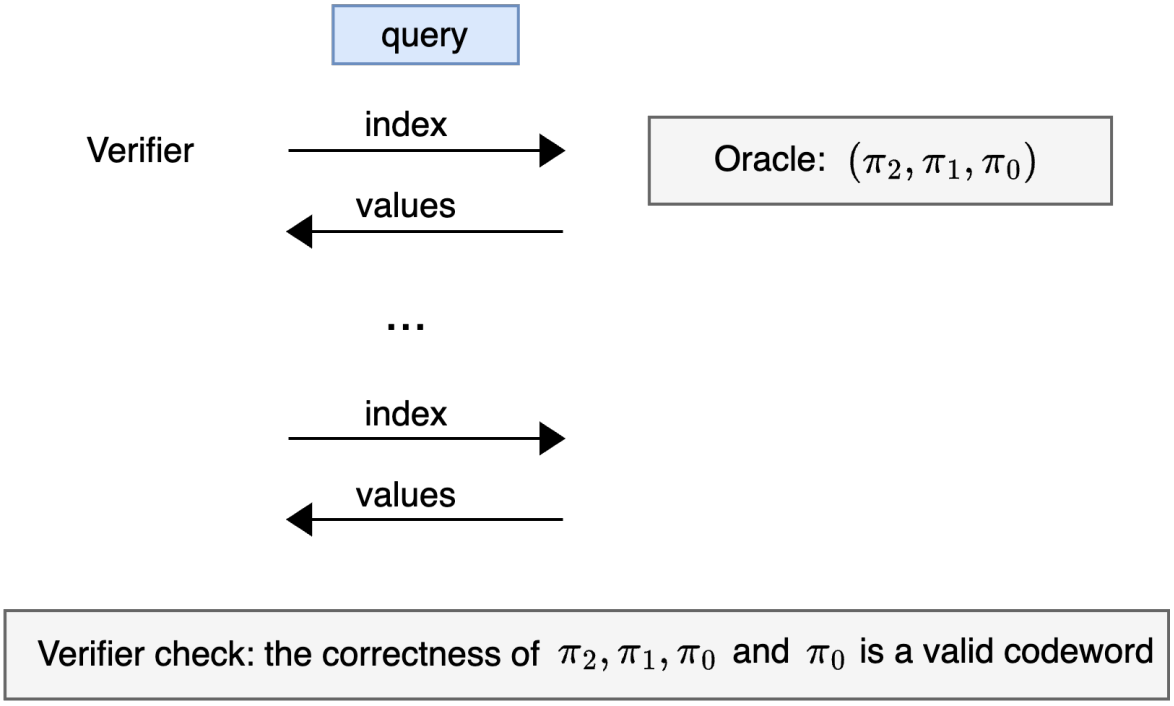


The prover sends all the oracles (π_2, π_1, π_0) to the verifier.

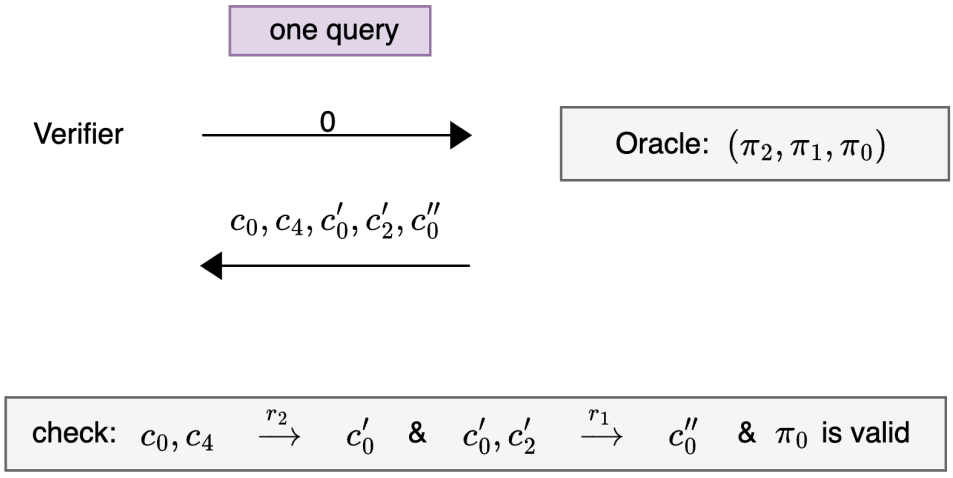
Note that given an input oracle that is a commitment of an encoding of a polynomial f, the last oracle π_0 sent by the honest prover in the IOPP protocol is exactly an encoding of a random evaluation $f(\mathbf{r})$. Therefore, this ultimate oracle π_0 must be a valid codeword.

- Query phase

In this phase, the verifier samples a query index $\mu \in [1, n_{d-1}]$ to check the consistency between oracles. The figure below shows the query phase. The verifier can query oracles several times. The larger the number of queries made, the higher security level of this model.



Let's focus on a single query to see more details. The verifier selects an index 0, and each oracle returns the value at that position and the value at the corresponding position after halving the vector. In this case, oracles return $(c_0, c_4, c_0', c_2', c_0'')$. Then, the verifier checks if c_0' is accurately computed from c_0, c_4, and the folding challenge r_2, and checks if c_0'' is correctly computed from c_0', c_2', and the folding challenge r_1. The verifier also checks π_0 is a valid codeword.



After several rounds, the prover passed all queries from the verifier. Finally, the verifier will believe the prover π_2 is truly commitment of the polynomial f.

1.5.3 Basefold PCS

Basefold PCS interleaves Basefold IOPP and sumcheck. The Goal of Basefold PCS is the prover wants to convince the verifier that it knows an opening of a private polynomial that $f(\mathbf{z}) = y$, with the polynomial's

commitment being π_d.

Therefore, Basefold PCS consists of two parts to prove:

1. $Commit(Enc(f)) = \pi_d$.
2. $f(\mathbf{z}) = y$.

The first statement can be proved by Basefold IOPP, while the second statement can be proved by using sumcheck. Let's see the further details:

It is known that by using the classic sum-check protocol with a multilinear extension, we can transform the evaluation check of a multilinear polynomial $f \in \mathbb{F}[X_1, ..., X_d]$ at a point $\mathbf{z} \in \mathbb{F}^d$into an evaluation check of f **at a random point $\mathbf{r} \in \mathbb{F}^d$. In the sumcheck protocol, the prover and the verifier interactive d rounds. In each round, a variate X_i is fixed using the verifier's challenge, consequently reducing the dimensionality by 1 of f.

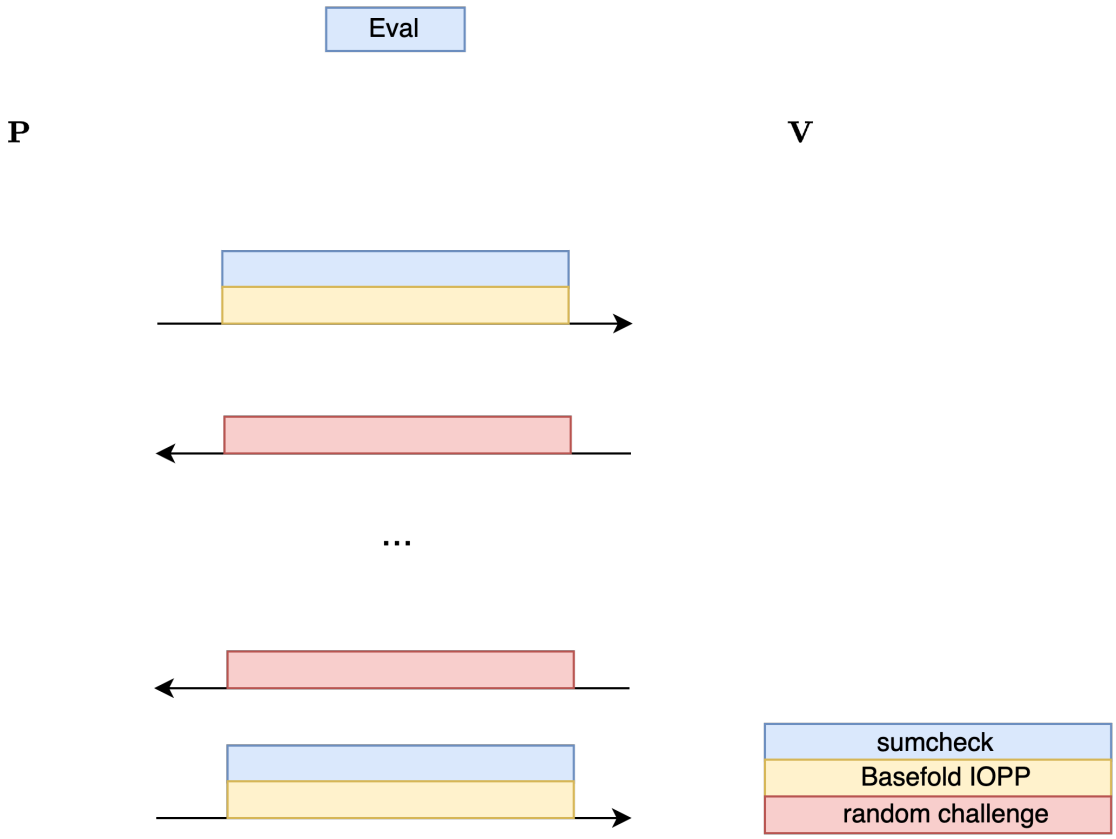
As the earlier discussion on Basefold IOPP, shows that Basefold IOPP also involves d interactive rounds between the prover and the verifier. In each round, the codeword is halved in size.

It can be seen that Basefold IOPP and sumcheck have a similar structure. Therefore, in the evaluation phase, the prover and verifier can simultaneously execute a Basefold IOPP protocol and a sumcheck protocol, using the identical set of random challenges $\mathbf{r} = (r_1, ..., r_d)$. At the end of the Basefold PCS, the verifier checks that the claimed evaluation in the last round of the sumcheck protocol is consistent with the last prover message of the IOPP protocol.

- **Basefold Evaluation phase**

  Given a commitment C, a point $\mathbf{z}$ and value y , the prover wants to convince the verifier that it knows an opening of $f \in \mathbb{F}[X_1, ..., X_d]$ such that $f(\mathbf{z}) = y$. Basefold PCS interleaves sumcheck with Basefold IOPP, operating as depicted in the the following figure, which illustrates the interactive process between the prover and the verifier.

  As from the figure, Basefold PCS executes Basefold IOPP and sumcheck in parallel. The blue part represents the sumcheck interactive process while the yellow part indicates the Basefold IOPP interactive process. The red part denotes the random challenge from the verifier.



Let's see an example, suppose the prover holds a polynomial $f(\vec{x})$ whose coefficients are $\mathbf{f} = (f_0, f_1, f_2, f_3)$. First, the prover using Basefold encoding algorithm to encode $\mathbf{f}$ into a codeword $\mathbf{c} = (c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7)$, and commit $\mathbf{c}$ to produce the commitment π_2.

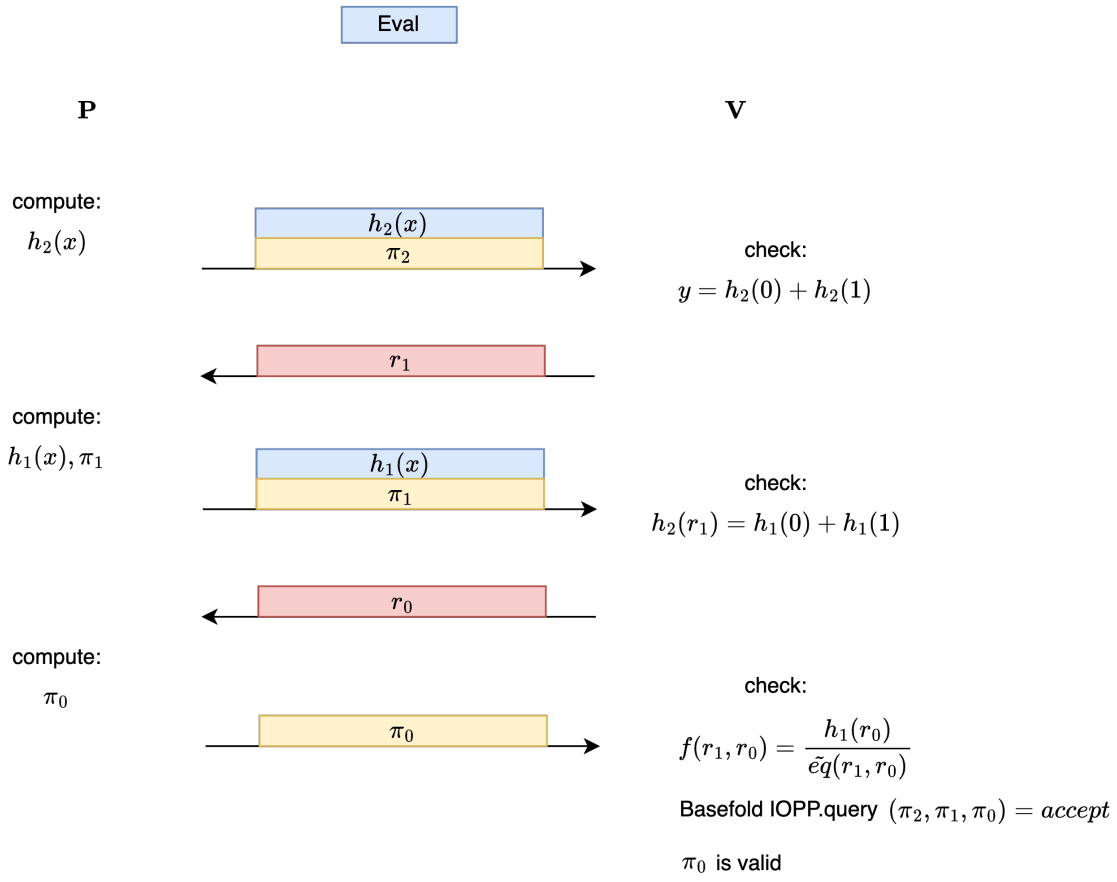
The prover wants to prove :

1. $Commit(Enc_2(\mathbf{f})) = \pi_2$
2. $f(\mathbf{z}) = y$

For prove 1, we utilize the Basefold IOPP. In the commit phase, the prover generates oracles (π_2, π_1, π_0) given the verifier's challenge (r_1, r_0) and sents oracles to the verifier. Then, in the query phase, the verifier queries some points, if the prover can pass all the verifier's checks and π_0 is a valid codeword, the verifier will accept it.

Regard to prove 2, we re-express the polynomial by the multilinear extension, $f(\vec{x})$ can be expressed as the sum: $f(x_1, x_2) = \sum_{\mathbf{b}\in\{0,1\}^2} f(\mathbf{b}) \cdot \tilde{eq}_{\mathbf{b}}(x_1, x_2)$. Thus, checking $f(\mathbf{z}) = y$ is equivalent to checking the sumcheck claim $y = \sum_{\mathbf{b}\in\{0,1\}^2} f(\mathbf{b}) \cdot \tilde{eq}_{\mathbf{b}}(\mathbf{z})$. Then, we can utilize the sumcheck protocol to prove it.

The Basefold PCS runs as the following graph:



We can see that Basefold IOPP and sumcheck share all the verifier's challenges (r_1, r_0). If the prover passes all the verifier's checks, then it convinces the verifier.

Chapter 2

Notes on Basefold (Part I): Foldable Linear Codes

- Yu Guo yu.guo@secbit.io
- Jade Xie jade@secbit.io

Basefold can be regarded as an extension of FRI, thereby supporting Proximity Proofs and Evaluation Arguments for Multi-linear Polynomials. Compared to Libra-PCS, Hyrax-PCS, and Virgo-PCS, Basefold does not rely on the MLE Quotients Equation to prove the value of an MLE at the point $\mathbf{u}$:

$$\tilde{f}(X_0, X_1, X_2, \dots, X_{n-1}) = \sum_{i_0=0}^{n-1} (X_i - u_i) \cdot q_i(X_0, X_1, X_2, \dots, X_{i-1})$$

Instead, Basefold leverages the Sumcheck protocol to reduce the value of $\tilde{f}(\mathbf{X})$ at a pre-specified point to its value at a random point $\vec{\alpha}$. The latter can then utilize an FRI-like approach, where the verifier provides a challenge vector $\vec{\alpha}$ to recursively fold $\tilde{f}(\mathbf{X})$. This approach allows the prover to simultaneously demonstrate an upper bound on the degree of $\tilde{f}(\mathbf{X})$ (Proof of Proximity) and, based on the characteristics of MLE, prove the value of $\tilde{f}(\vec{\alpha})$. By combining the Sumcheck protocol with an FRI-style folding protocol, we elegantly obtain an Evaluation Argument for an MLE.

Another significant insight of Basefold is how to apply an FRI-style Proof of Proximity over arbitrary finite fields. It is known that the FRI protocol fully utilizes the structure of Algebraic FFTs over finite fields and performs interactive folding, which allows the codeword length to be exponentially reduced while maintaining the minimum relative Hamming distance bound, thereby enabling the verifier to easily verify a folded short codeword. However, for general finite fields where constructing FFTs is challenging, the FRI protocol cannot be directly applied.

Basefold introduces the concept of Random Foldable Codes. This is a framework for encoding using a recursive approach, where recursive encoding can be seen as the inverse process of recursive folding. In the Commit phase of the Basefold protocol, the prover first encodes each segment of the message using a base encoding scheme G_0, then encodes these codewords pairwise (a process similar to the butterfly operation in FFT computations), and finally obtains a single codeword. In the Commit-phase stage, the prover and verifier interactively perform half-folding on the committed single codeword and then commit to the half-folded codeword. It can be proven that the Relative Hamming Distance of this half-folded codeword remains above a clear lower bound. The parties then continue folding until the length is reduced to that of G_0-encoded length. In this way, both parties obtain a series of committed codewords. Note that since the encoding process is performed recursively, the codeword possesses folding capabilities similar to those of RS Code encoding. The advantage of this approach is evident: using Recursive Folding FRI protocols results in significantly smaller proof sizes compared to protocols like Tensor Codes. Additionally, Basefold's technique can be applied to any

finite field ($|\mathbb{F}| > 2^{10}$), including extension fields $\mathbb{F}_{p^k}$. Naturally, similar to the FRI protocol, the commitment computation in the Basefold protocol requires $O(N\log(N))$ time complexity, while the prover's computational effort in the subsequent proof process remains at $O(N)$. If a Merkle Tree is used as the commitment scheme for the codeword, the proof size complexity is $O(\log^2(N))$.

In this article, we first explore the concept of Foldable Linear Codes.

2.1 What are Foldable Linear Codes

Suppose there is a linear code $C_0 : \mathbb{F}_p^{k_0} \to \mathbb{F}_p^{n_0}$ based on the finite field $\mathbb{F}_p$, where the message length is k_0, and the codeword length is n_0. Assume that the encoded length is amplified by a factor of R compared to the message length, i.e., $1/R$ is the traditional code rate. According to the definition of linear codes, there must exist an encoding matrix $G_0 : \mathbb{F}_p^{k_0 \times n_0}$ such that

$$\mathbf{m}G_0 = \mathbf{c}$$

Then, we can construct a new encoding matrix G_1 based on the encoding matrix G_0:

$$G_1 = \begin{bmatrix} G_0 & G_0 \\ G_0 \cdot T_0 & G_0 \cdot T_0' \end{bmatrix}$$

where T_0 and T_0' are two diagonal matrices with diagonal elements set to the parameters of the encoding scheme, which we will explain later.

If we consider G_1 as the encoding matrix of another linear code C_1, then the parameters of C_1 are $[R, 2k_0, 2n_0]$. That is, compared to C_0, the code rate of C_1 remains unchanged, but the message and codeword lengths are both doubled.

For example, suppose $\mathbf{m} = (m_0, m_1, m_2, m_3)$, and the base encoding scheme C_0 is a simple Repetition Code with $k_0 = 2$ and $n_0 = 4$. Thus, the message length of $\mathbf{m}$ exactly meets the requirements of G_1, i.e., $k_1 = 2 \cdot k_0 = 4$ and the encoded length is $n_1 = 2 \cdot n_0 = 8$.

The encoding matrix G_0 of the base code C_0 is defined as:

$$G_0 = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

Assume the diagonal elements of the system parameter T_0 are (t_0, t_1, t_2, t_3), and those of T_0' are (t_0', t_1', t_2', t_3'). We construct G_1 using the formula above:

$$G_1 = \left[\begin{array}{cccc|cccc} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ \hline t_0 & 0 & t_2 & 0 & t_0' & 0 & t_2' & 0 \\ 0 & t_1 & 0 & t_3 & 0 & t_1' & 0 & t_3' \end{array}\right]$$

Substituting $\mathbf{m}$ directly, we obtain:

$$\mathbf{m}G_1 = \begin{bmatrix} m_0 & m_1 & m_2 & m_3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ t_0 & 0 & t_2 & 0 & t_0' & 0 & t_2' & 0 \\ 0 & t_1 & 0 & t_3 & 0 & t_1' & 0 & t_3' \end{bmatrix}$$

We list the encoded codeword vector separately:

$$\begin{matrix} m_0 + t_0 m_2 & m_1 + t_1 m_3 & m_0 + t_2 m_2 & m_1 + t_3 m_3 \\ m_0 + t_0' m_2 & m_1 + t_1' m_3 & m_0 + t_2' m_2 & m_1 + t_3' m_3 \end{matrix}$$

Directly performing matrix operations, the structure of this encoding is not very clear. Let us approach it differently: the message $\mathbf{m}$ can be split into two equal-length parts, the left $\mathbf{m}_l = (m_0, m_1)$ and the right $\mathbf{m}_r = (m_2, m_3)$,

$$\mathbf{m}G_1 = (\mathbf{m}_l \parallel \mathbf{m}_r)G_1 = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{m}_l & 0 \\ 0 & \mathbf{m}_r \end{bmatrix} \begin{bmatrix} G_0 & G_0 \\ G_0 \cdot T_0 & G_0 \cdot T_0' \end{bmatrix} = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{m}_l G_0 & \mathbf{m}_l G_0 \\ \mathbf{m}_r G_0 T_0 & \mathbf{m}_r G_0 T_0' \end{bmatrix}$$

For the 2×2 matrix on the right side of the equation, we first compute the top-left and top-right submatrices, which are identical submatrices obtained by encoding $\mathbf{m}_l$ with G_0:

$$\mathbf{m}_l G_0 = \begin{bmatrix} m_0 & m_1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} m_0 & m_1 & m_0 & m_1 \end{bmatrix}$$

And the bottom-left submatrix is:

$$\begin{aligned} \mathbf{m}_r(G_0 T_0) = (\mathbf{m}_r G_0) T_0 &= \begin{bmatrix} m_2 & m_3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} T_0 \\ &= \begin{bmatrix} m_2 & m_3 & m_2 & m_3 \end{bmatrix} \begin{bmatrix} t_0 & 0 & 0 & 0 \\ 0 & t_1 & 0 & 0 \\ 0 & 0 & t_2 & 0 \\ 0 & 0 & 0 & t_3 \end{bmatrix} \\ &= (t_0 m_2, t_1 m_3, t_2 m_2, t_3 m_3) \end{aligned}$$

The result for $\mathbf{m}_r(G_0 T_0')$ is almost the same, with t_i replaced by t_i':

$$\mathbf{m}_r(G_0 T_0') = (t_0' m_2, t_1' m_3, t_2' m_2, t_3' m_3)$$

It is easy to verify that we obtain the same result for $\mathbf{m}$ encoded with G_1.

We can simplify this computation process with the following equation:

$$\mathbf{m}G_1 = \mathbf{m}_l G_0 + (t_0, t_1, t_2, t_3) \circ \mathbf{m}_r G_0 \parallel \mathbf{m}_l G_0 + (t_0', t_1', t_2', t_3') \circ \mathbf{m}_r G_0$$

If we consider k_0 as a system parameter (constant), the encoding computation in the above equation has a complexity of $O(n_0)$, including two G_0 encodings and two component-wise additions of length n_0. Additionally, this equation resembles the butterfly operation in (Multiplicative) FFT algorithms:

$$a' = a + t \cdot b, \quad b' = a - t \cdot b$$

In fact, as illustrated later, the encoding process of Foldable Codes is a generalized extension of RS-Code.

We can further recursively construct $G_2, G_3, \ldots, G_d$, eventually obtaining a linear code $C_d : \mathbb{F}_p^k \to \mathbb{F}_p^n$, where the message length is $k = k_0 \cdot 2^d$, and the encoding length is $n = c \cdot k_0 \cdot 2^d$. The code rate is $\rho = \frac{1}{R}$, where k_0 is the message length of the base encoding, n_0 is the base encoding length, and the choice of the base encoding is highly flexible.

We use the symbol G_d to denote the encoding matrix (or generator matrix) of C_d, $G_d \in \mathbb{F}_p^{k \times n}$. For $i \in \{1, 2, \ldots, d\}$, we have the following recursive relation:

$$G_i = \begin{bmatrix} G_{i-1} & G_{i-1} \\ G_{i-1} \cdot T_{i-1} & G_{i-1} \cdot T'_{i-1} \end{bmatrix}$$

Here, T_{i-1} and T'_{i-1} are two diagonal matrices,

$$T_{i-1} = \begin{bmatrix} t_{i-1,0} & 0 & \cdots & 0 \\ 0 & t_{i-1,1} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & t_{i-1,n_i-1} \end{bmatrix}, \quad T'_{i-1} = \begin{bmatrix} t'_{i-1,0} & 0 & \cdots & 0 \\ 0 & t'_{i-1,1} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & t'_{i-1,n_i-1} \end{bmatrix},$$

and their diagonal elements at the same positions are distinct:

$$\forall j \in [0, n_i - 1], t_{i,j} \neq t'_{i,j}$$

Assuming we can compute G_d using the above recursive equation, then for a message $\mathbf{m}_d$ of length k, the encoding process can be directly calculated as follows:

$$\mathbf{w}_d = \mathbf{m}_d G_d$$

However, encoding directly using the generator matrix G_d is relatively inefficient, with a computational complexity of $O(k \cdot n)$. Instead, based on the above derivation, if we do not directly use the G_d matrix but instead employ a recursive approach to encode $\mathbf{m}_d$, the basic idea is to split m into two parts m_l and m_r, then encode m_l and m_r separately using G_{d-1}, and finally concatenate the encoded C_l and C_r.

$$\mathbf{w}_d = \left(m_l G_{d-1} + \mathsf{diag}(T_d) \circ m_r G_{d-1}\right) \parallel \left(m_l G_{d-1} + \mathsf{diag}(T'_d) \circ m_r G_{d-1}\right)$$

In this way, the encoding process using G_d is transformed into two encoding processes using G_{d-1}. We can continue to recursively compute $m_l G_{d-1}$ and $m_r G_{d-1}$ until the split message length meets k_0, after which we simply use the generator matrix G_0 for the base encoding. The encoding time complexity at this stage is $O(k_0 \cdot n_0)$. With a total of d recursive rounds, the overall computational effort is reduced to $O(n \log(n))$.

2.2 RS code (foldable)

The RS code used in the FRI protocol satisfies the aforementioned conditions, making RS code foldable.

First, let us examine the generator matrix in the RS code:

$$\begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{k-1} & \omega^{2(k-1)} & \cdots & \omega^{(n-1)(k-1)} \end{bmatrix}$$

where ω is an n-th root of unity, satisfying $\omega^n = 1$, with n being the codeword length and k the message length. We can observe that the above matrix is a Vandermonde matrix. We will now explain how the generator matrix of the RS code satisfies the definition of Foldable Codes. For demonstration purposes, assume $k_0 = 1$, $n_0 = 1$, $d = 3$, $k_3 = 8$, $n_3 = 8$, and $\omega^8 = 1$,

$$G^{(RS)} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \omega^4 & \omega^5 & \omega^6 & \omega^7 \\ 1 & \omega^2 & \omega^4 & \omega^6 & 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^3 & \omega^6 & \omega^1 & \omega^4 & \omega^7 & \omega^2 & \omega^5 \\ 1 & \omega^4 & 1 & \omega^4 & 1 & \omega^4 & 1 & \omega^4 \\ 1 & \omega^5 & \omega^2 & \omega^7 & \omega^4 & \omega & \omega^6 & \omega^3 \\ 1 & \omega^6 & \omega^4 & \omega^2 & 1 & \omega^6 & \omega^4 & \omega^2 \\ 1 & \omega^7 & \omega^6 & \omega^5 & \omega^4 & \omega^3 & \omega^2 & \omega \end{bmatrix}$$

First, we need to reorder the rows of G^{RS} according to the Reversed Bit Order (RBO) sequence. The reason for performing RBO reordering is related to the structure of Multiplicative FFTs, as explained in another article. The RBO sequence refers to reversing the binary representation of the indices and using these reversed binary numbers as the new indices. The RBO sequence is $(0, 4, 2, 6, 1, 5, 3, 7)$. By reordering the rows of the above Vandermonde matrix according to the RBO sequence, we obtain the following matrix, denoted as G_2:

$$G_2 = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \omega^4 & 1 & \omega^4 & 1 & \omega^4 & 1 & \omega^4 \\ 1 & \omega^2 & \omega^4 & \omega^6 & 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^6 & \omega^4 & \omega^2 & 1 & \omega^6 & \omega^4 & \omega^2 \\ 1 & \omega & \omega^2 & \omega^3 & \omega^4 & \omega^5 & \omega^6 & \omega^7 \\ 1 & \omega^5 & \omega^2 & \omega^7 & \omega^4 & \omega & \omega^6 & \omega^3 \\ 1 & \omega^3 & \omega^6 & \omega^1 & \omega^4 & \omega^7 & \omega^2 & \omega^5 \\ 1 & \omega^7 & \omega^6 & \omega^5 & \omega^4 & \omega^3 & \omega^2 & \omega \end{bmatrix}$$

Observing the reordered matrix, we can see that this matrix can be decomposed into submatrix operations involving G_1 and T_1, T_1':

$$G_2 = \begin{bmatrix} G_1 & G_1 \\ G_1T_1 & G_1T_1' \end{bmatrix} = \left[\begin{array}{cccc|cccc} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \omega^4 & 1 & \omega^4 & 1 & \omega^4 & 1 & \omega^4 \\ 1 & \omega^2 & \omega^4 & \omega^6 & 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^6 & \omega^4 & \omega^2 & 1 & \omega^6 & \omega^4 & \omega^2 \\ \hline 1 & \omega & \omega^2 & \omega^3 & \omega^4 & \omega^5 & \omega^6 & \omega^7 \\ 1 & \omega^5 & \omega^2 & \omega^7 & \omega^4 & \omega & \omega^6 & \omega^3 \\ 1 & \omega^3 & \omega^6 & \omega^1 & \omega^4 & \omega^7 & \omega^2 & \omega^5 \\ 1 & \omega^7 & \omega^6 & \omega^5 & \omega^4 & \omega^3 & \omega^2 & \omega \end{array}\right]$$

where G_1 is:

$$G_1 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega^4 & 1 & \omega^4 \\ 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^6 & \omega^4 & \omega^2 \end{bmatrix}$$

We can verify that the bottom-left submatrix can be expressed as G_1T_1:

$$G_1T_1 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega^4 & 1 & \omega^4 \\ 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^6 & \omega^4 & \omega^2 \end{bmatrix} \begin{bmatrix} 1 & & & \\ & \omega & & \\ & & \omega^2 & \\ & & & \omega^3 \end{bmatrix} = \begin{bmatrix} 1 & \omega & \omega^2 & \omega^3 \\ 1 & \omega^5 & \omega^2 & \omega^7 \\ 1 & \omega^3 & \omega^6 & \omega^1 \\ 1 & \omega^7 & \omega^6 & \omega^5 \end{bmatrix}$$

Here, the matrix T_1 is indeed a diagonal matrix satisfying the requirement that its diagonal elements are distinct. The bottom-right submatrix can be decomposed as follows:

$$\begin{bmatrix} \omega^4 & \omega^5 & \omega^6 & \omega^7 \\ \omega^4 & \omega & \omega^6 & \omega^3 \\ \omega^4 & \omega^7 & \omega^2 & \omega^5 \\ \omega^4 & \omega^3 & \omega^2 & \omega^1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega^4 & 1 & \omega^4 \\ 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^6 & \omega^4 & \omega^2 \end{bmatrix} \begin{bmatrix} \omega^4 & & & \\ & \omega^5 & & \\ & & \omega^6 & \\ & & & \omega^7 \end{bmatrix} = G_1 T_1'$$

where T_1' is exactly $(-1)T_1$, ensuring that the elements of T_1' are distinct from those of T_1:

$$T_2' = \begin{bmatrix} \omega^4 & & & \\ & \omega^5 & & \\ & & \omega^6 & \\ & & & \omega^7 \end{bmatrix} = \begin{bmatrix} -1 & & & \\ & -\omega & & \\ & & -\omega^2 & \\ & & & -\omega^3 \end{bmatrix} = -T_2$$

Next, we can continue to recursively decompose G_1:

$$G_1 = \left[\begin{array}{cc|cc} 1 & 1 & 1 & 1 \\ 1 & \omega^4 & 1 & \omega^4 \\ \hline 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^6 & \omega^4 & \omega^2 \end{array}\right] = \begin{bmatrix} G_0 & G_0 \\ G_0 \cdot T_0 & G_1 \cdot T_0' \end{bmatrix}$$

Here, G_0, T_0, and T_0' are:

$$G_0 = \begin{bmatrix} 1 & 1 \\ 1 & \omega^4 \end{bmatrix}, \quad T_0 = \begin{bmatrix} 1 & \\ & \omega^2 \end{bmatrix}, \quad T_0' = \begin{bmatrix} \omega^4 & \\ & \omega^6 \end{bmatrix} = -T_0$$

By recursively decomposing, we reach the base encoding G_0, which remains an RS-Code, denoted as $RS[k_0 = 1, n_0 = 1]$. Thus, we have proven that RS-Code is a foldable code. However, it is evident that RS-Code imposes requirements on $\mathbb{F}_p$, needing it to contain a multiplicative subgroup of order 2^s, where s is a positive integer, and ensuring that s is large enough to accommodate a codeword length of $k_0 \cdot 2^{n_d}$.

2.3 References

- [ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. "BaseFold: efficient field-agnostic polynomial commitment schemes from foldable codes." In *Annual International Cryptology Conference*, pp. 138-169. Cham: Springer Nature Switzerland, 2024.
- Riad S. Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. "Doubly-efficient zkSNARKs without trusted setup." In *2018 IEEE Symposium on Security and Privacy (SP)*, pp. 926-943. IEEE, 2018.
- Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. "Libra: Succinct zero-knowledge proofs with optimal prover computation." In *Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39*, pp. 733-764. Springer International Publishing, 2019.
- Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. "Transparent polynomial delegation and its applications to zero knowledge proof." In *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 859-876. IEEE, 2020.
- Charalampos Papamanthou, Elaine Shi, and Roberto Tamassia. "Signatures of correct computation." In *Theory of Cryptography Conference*, pp. 222-242. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013.

Chapter 3

Notes on Basefold (Part II): IOPP

- Yu Guo yu.guo@secbit.io
- Jade Xie jade@secbit.io

3.1 Proof of Proximity

Below, we present a proof of implementing IOPP using Foldable codes.

Suppose there is an MLE polynomial $\tilde{f}(\mathbf{X})$ represented as follows:

$$\tilde{f}(X_0, X_1, \dots, X_{d-1}) = f_0 + f_1 X_0 + f_2 X_1 + f_3 X_0 X_1 + \cdots + f_{2^d-1} X_{d-1}$$

Since $\tilde{f}(X)$ is a multivariate polynomial, there are d unknowns, making the length of its coefficient vector 2^d. Note that we choose the Lexicographic Order as the sorting method for the polynomial.

We encode the coefficient vector $\mathbf{f}$ of $\tilde{f}(\mathbf{X})$ to obtain the codeword $c_{\mathbf{f}} = \mathsf{Enc}(\mathbf{f})$, which has a length of n_d. Then, we use a Hash-based Merkle Tree to generate the commitment:

$$\mathbf{cm}(\mathbf{f}) = \mathsf{Merklize}(\mathsf{Enc}(f_0, f_1, f_2, \dots, f_{2^d-1}))$$

Similar to the FRI protocol, the Basefold-IOPP protocol is used to prove that a commitment $\pi_d = \mathbf{cm}(\mathbf{f})$ is with high probability "close" to a vector encoded by C_d. Therefore, this protocol is called a Proof of Proximity. This protocol is one of the core protocols for constructing the Evaluation Argument.

Proof of Proximity leverages a remarkable property of linear codes: the "Proximity Gap." Specifically, if two vectors π, π' are far from the legitimate codeword space, then their random linear combination $\pi + \alpha \cdot \pi'$ will either have a very low probability of becoming legitimate or will remain far from the legitimate codeword space:

$$\begin{cases} \Delta(\pi_i, C_i) = 0 & \text{(with negligible probability)} \\ \Delta(\pi_i, C_i) \leq \Delta(\pi_{i+1}, C_{i+1}) & \text{(with non-negligible probability)} \end{cases}$$

This indicates that the folding process of the codeword does not disrupt the distance between the vector and the legitimate codeword space. By folding the vector sufficiently, the Verifier can use a very short code to verify whether the final folded vector is a legitimate codeword, thereby determining whether the original vector is a legitimate codeword.

Notes on Proximity Gap Proof of Proximity utilizes a remarkable property of linear codes: the "Proximity Gap." Specifically, for two vectors π, π', folding them with a random scalar $\alpha \in \mathbb{F}$ yields a set $A = \{\pi + \alpha \cdot \pi' : \alpha \in \mathbb{F}\}$. Different α correspond to different elements in set A. The "Proximity Gap" theorem states that the elements in this set are either all close to the legitimate codeword space C_i or only a negligible fraction of the elements are close to C_i, while the majority are at a distance of δ from C_i. In probabilistic terms:

$$\Pr_{a \in A}[\Delta(a, C_i) \leq \delta] = \begin{cases} \epsilon & \text{(small enough)} \\ 1 \end{cases}$$

Thus, the Verifier can confidently use a random scalar α for folding, because even if only one of the two vectors π, π' provided by a cheating Prover is at a distance δ from C_i, the probability that the folded result is close to C_i is only ϵ, which is very small. In other words, a cheating Prover would need to be as lucky as winning the lottery to evade detection by the Verifier's scrutiny. Therefore, if the Prover initially selects a π_d that is far from the legitimate codeword space, the Verifier selects a series of random scalars to iteratively fold it until obtaining π_0. During this process, there is a high probability that π_0 does not become close to the legitimate codeword space, allowing the Verifier to detect cheating.

The "Proximity Gap" theorem provides a significant advantage to the Verifier: instead of verifying all elements in the set $A = \{\pi + \alpha \cdot \pi' : \alpha \in \mathbb{F}\}$ to check their proximity to the legitimate codeword space, the Verifier only needs to randomly select one point for verification. This greatly reduces the Verifier's computational load.

The Proof of Proximity protocol consists of two phases: the Commit-phase and the Query-phase. The former involves the subprotocol that performs multiple folding processes of the codeword and generates commitments (or oracles) for each folded codeword. The latter, the Query-phase, involves the Verifier performing random sampling to verify the legitimacy of each folding step.

3.2 Commit-phase

First, we explain the Commit-phase. The Prover performs multiple foldings of the encoded π_d (with length n_d), obtaining codewords of lengths $n_{d-1}, \ldots, n_0$, denoted as $(\pi_{d-1}, \pi_{d-2}, \ldots, \pi_0)$, and then sends them to the Verifier. We should note that length of the all the codewords $n_{d-1}, \ldots, n_0$ has to be a power of 2.

Remember that this is an interactive protocol with a total of d rounds of interaction. In each round (assume the i-th round, $0 \leq i < d$), the Prover folds π_{i+1} based on the random scalar α_i sent by the Verifier to obtain a new codeword, denoted as π_i. After d rounds, the Prover obtains a codeword of length n_0, denoted as π_0. The Prover then commits to each $(\pi_d, \ldots, \pi_0)$ and sends $\mathsf{cm}(\pi_d), \ldots, \mathsf{cm}(\pi_0)$ as the output of $\mathsf{IOPP.Commit}$.

Next, we analyze the technical details of a single folding π_i. Suppose $\pi_i \in C_i$ is a legitimate codeword (i.e., satisfying $\pi_i = \mathbf{m}G_i$), with length n_i:

$$\pi_i = (c_0, c_1, c_2, \ldots, c_{n_i - 1}) \qquad 0 \leq i \leq d$$

We split this vector into two parts and stack them:

$$\begin{pmatrix} c_0, & c_1, & \ldots, & c_{n_{i-1}-1} \\ c_{n_{i-1}}, & c_{n_{i-1}+1}, & \ldots, & c_{n_i - 1} \end{pmatrix}$$

At this point, the Verifier needs to provide a random scalar $\alpha^{(i)}$. We perform a random linear combination of the two rows, or in other words, fold them:

$$\pi_{i-1} = (\mathsf{fold}_{\alpha_i}(c_0, c_{n_{i-1}}), \mathsf{fold}_{\alpha_i}(c_1, c_{n_{i-1}+1}), \ldots, \mathsf{fold}_{\alpha_i}(c_{n_{i-1}-1}, c_{n_i - 1}))$$

The above is the folded vector π_{i-1}. Assuming the Prover is honest, the folded vector should be a legitimate C_{i-1} codeword. The function fold_{α_i} in the above equation is defined as follows:

$$\mathsf{fold}_{\alpha}(c_j, c_{n_{i-1}+j}) = \frac{t_j \cdot c_{n_{i-1}+j} - t'_j \cdot c_j}{t_j - t'_j} + \alpha \cdot \frac{(c_j - c_{n_{i-1}+j})}{t_j - t'_j}$$

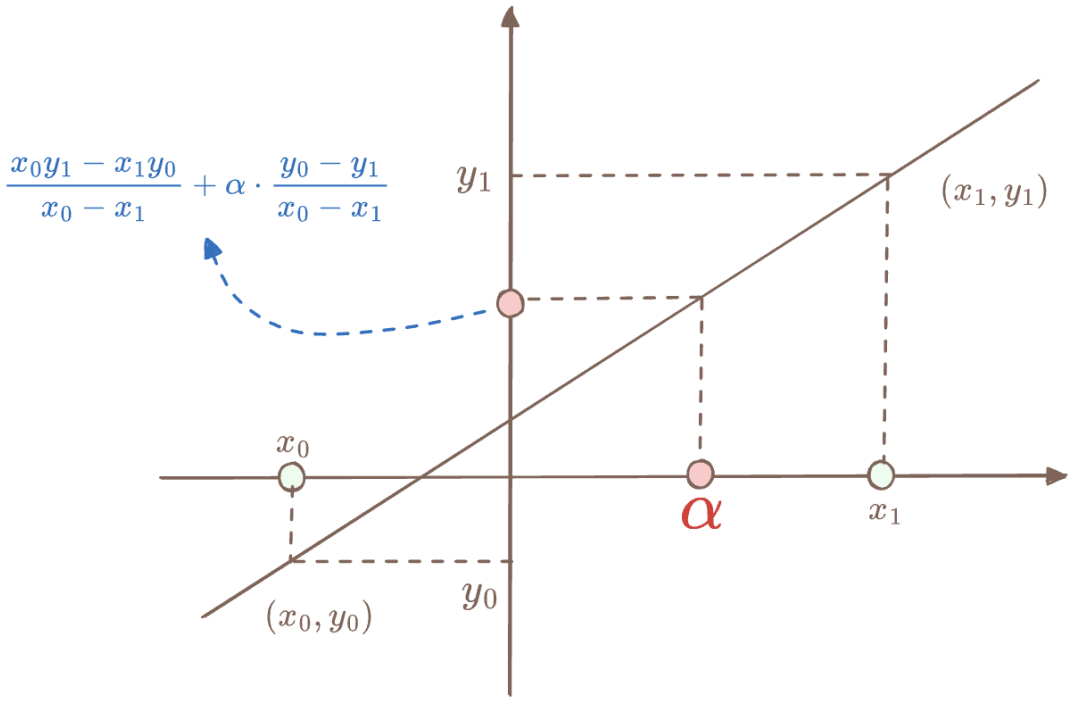
How should we understand the $\mathsf{fold}_{\alpha}(\cdot, \cdot)$ function? It is essentially a polynomial interpolation process. We treat the two rows to be folded as sets of points on two separate domains, specifically the $\mathsf{diag}(T_i) = (t_0, t_1, \ldots, t_{n_{i-1}-1})$ and $\mathsf{diag}(T'_i) = (t'_0, t'_1, \ldots, t'_{n_{i-1}-1})$ used in the recursive encoding process:

$$\begin{pmatrix} (t_0, c_0), & (t_1, c_1), & \ldots, & (t_{i-1}, c_{n_{i-1}-1}) \\ (t'_0, c_{n_{i-1}}), & (t'_1, c_{n_{i-1}+1}), & \ldots, & (t'_{i-1}, c_{n_i-1}) \end{pmatrix}$$

We then interpolate each column of the above matrix over the domain (t_j, t'_j) to produce a set of $n_{i-1} = n_i/2$ polynomials, denoted as $p_j^{(i-1)}(X)$, where $0 \leq j < n_{i-1}$. The Prover then evaluates each $p_j^{(i-1)}(X)$ at $X = \alpha_i$, resulting in n_{i-1} values at $X = \alpha_i$. These values constitute the new codeword π_{i-1}.

The definition of the folding function aligns with the linear polynomial interpolation process. We can manually derive the origin of the folding function definition. Since we are performing a half-folding of π_i, the folded codeword will have n_{i-1} values corresponding to "linear polynomials." Suppose the j-th polynomial describes a line passing through two points (x_0, y_0) and (x_1, y_1). The interpolating polynomial $p(X)$ for these two points can be defined as:

$$\begin{aligned} p(X) &= \frac{y_0}{x_0 - x_1}(X - x_1) + \frac{y_1}{x_1 - x_0}(X - x_0) \\ &= \frac{x_0 y_1 - x_1 y_0}{x_0 - x_1} + \frac{y_0 - y_1}{x_0 - x_1} \cdot X \end{aligned}$$



Substituting $x_0 = t_j, x_1 = t'_j$, and $X = \alpha$ yields the definition of the folding function $\mathsf{fold}_\alpha(y_0, y_1)$ as above.

$$\mathsf{fold}_\alpha(y_0, y_1) = p(\alpha) = \frac{t_j \cdot y_1 - t'_j \cdot y_0}{t_j - t'_j} + \frac{y_0 - y_1}{t_j - t'_j} \cdot \alpha$$

If x_0 and x_1 are negatives of each other, i.e., $t_j = -t'_j$, then $\mathsf{fold}_\alpha(y_0, y_1)$ becomes the familiar definition from the FRI protocol:

$$\mathsf{fold}_\alpha(y_0, y_1) = \frac{1}{2}(y_0 + y_1) + \alpha \cdot \frac{y_0 - y_1}{2 \cdot t_j}$$

Since we have defined the folded codeword $\mathbf{c}^{(i-1)}$, the definition of the folding function needs to be consistent with the codeword space generated by the generator matrix G_{i-1}. Continuing with this intuition, assume that in the i-th round, if $\mathbf{c}^{(i)}$ is indeed the encoding of $\mathbf{m}$, then by definition, it satisfies the properties of Foldable Codes:

$$\begin{aligned}
\pi_i &= \mathbf{m}G_i \\
&= (\mathbf{m}_l \parallel \mathbf{m}_r) \begin{bmatrix} G_{i-1} & G_{i-1} \\ G_{i-1} \cdot T_{i-1} & G_{i-1} \cdot T'_{i-1} \end{bmatrix} \\
&= \left(\mathbf{m}_l G_{i-1} + \mathbf{m}_r G_{i-1} \circ \mathbf{diag}(T_{i-1})\right) \parallel \left(\mathbf{m}_l G_{i-1} + \mathbf{m}_r G_{i-1} \circ \mathbf{diag}(T'_{i-1})\right)
\end{aligned}$$

The Prover folds it in half to obtain the new codeword:

$$\mathsf{fold}_\alpha(\pi_i) = \Big(\mathsf{fold}_\alpha(\pi_i[0], \pi_i[n_{i-1}]), \mathsf{fold}_\alpha(\pi_i[1], \pi_i[n_{i-1}+1]), \dots, \mathsf{fold}_\alpha(\pi_i[n_{i-1}-1], \pi_i[n_i-1])\Big)$$

We now verify that each $\mathsf{fold}_\alpha(\pi_i[j], \pi_i[n_{i-1}+j])$ is a linear combination of $\mathbf{m}_l G_{i-1}[j]$ and $\mathbf{m}_r G_{i-1}[j]$ with respect to α:

$$\begin{aligned}
\mathsf{fold}_\alpha(\pi_i[j], \pi_i[n_{i-1}+j]) &= \frac{1}{t_j - t'_j} \cdot \Big(t_j \cdot (\mathbf{m}_l G_{i-1}[j] + t'_j \cdot \mathbf{m}_r G_{i-1}[j]) - t'_j \cdot (\mathbf{m}_l G_{i-1}[j] + t_j \cdot \mathbf{m}_r G_{i-1}[j])\Big) \\
&+ \frac{\alpha}{t_j - t'_j} \cdot \Big(\mathbf{m}_l G_{i-1}[j] + t_j \cdot \mathbf{m}_r G_{i-1}[j] - \mathbf{m}_l G_{i-1}[j] - t'_j \cdot \mathbf{m}_r G_{i-1}[j]\Big) \\
&= \mathbf{m}_l G_{i-1}[j] + \alpha \cdot \mathbf{m}_r G_{i-1}[j]
\end{aligned}$$

Thus, the entire folding of π_i is equivalent to a linear combination of $\mathbf{m}_l G_{i-1}$ and $\mathbf{m}_r G_{i-1}$ with respect to α:

$$\mathsf{fold}_\alpha(\pi_i) = \mathbf{m}_l G_{i-1} + \alpha \cdot \mathbf{m}_r G_{i-1} = (\mathbf{m}_l + \alpha \cdot \mathbf{m}_r) G_{i-1}$$

The folded codeword is exactly the half-folded version of $\mathbf{m}$ with respect to α, denoted as $\mathbf{m}^{(i-1)}$, and then encoded with G_{i-1} to obtain π_{i-1}. This is not surprising because Foldable Codes and the recursive folding of codewords are inverse processes, so the parameters T_i and T'_i introduced by the encoding are eliminated after folding.

Below, we walk through a simple example to illustrate how the Commit-phase of the Basefold-IOPP protocol operates.

Public Input

- The codeword of the MLE polynomial $\tilde{f}$, $\pi_3 = \mathsf{Enc}_3(\mathbf{f}) = \mathbf{f}G_3$

Witness

- The coefficient vector of the MLE polynomial $\tilde{f}$, $\mathbf{f} = (f_0, f_1, f_2, f_3, f_4, f_5, f_6, f_7)$

First Round: Verifier sends a random scalar α_2

Second Round: Prover computes $\pi_2 = \mathsf{fold}_\alpha(\pi_3)$ and sends it to the Verifier

The process of computing π_2 is as follows (we have assumed the length of coefficient vector and encoded vector to be equal for simplifying the explanation):

$$\pi_2[j] = \mathsf{fold}_\alpha(\pi_3[j], \pi_3[j+4]), \quad j \in \{0, 1, 2, 3\}$$

The computed $\pi_2 = \mathsf{Enc}_2(f^{(2)})$, meaning π_2 is the codeword of $f^{(2)}$, where $f^{(2)}$ is the folding of f with respect to α_2:

$$f^{(2)}(X_0, X_1) = f(X_0, X_1, \alpha_2) = (f_0 + f_4\alpha_2) + (f_1 + f_5\alpha_2)X_0 + (f_2 + f_6\alpha_2)X_1 + (f_3 + f_7\alpha_2)X_0X_1$$

Third Round: Verifier sends a random scalar α_1

Fourth Round: Prover computes $\pi_1 = \mathsf{fold}_\alpha(\pi_2)$ and sends it to the Verifier

The process of computing π_1 is as follows:

$$\pi_1[j] = \mathsf{fold}_\alpha(\pi_2[j], \pi_2[j+2]), \quad j \in \{0, 1\}$$

The computed $\pi_1 = \mathsf{Enc}_1(f^{(1)})$, meaning π_1 is the codeword of $f^{(1)}$, where $f^{(1)}$ is the folding of $f^{(2)}$ with respect to α_1:

$$f^{(1)}(X_0) = f(X_0, \alpha_1, \alpha_2) = (f_0 + f_4\alpha_2 + \alpha_1(f_2 + +f_6\alpha_2)) + (f_1 + f_5\alpha_2 + \alpha_1(f_3 + +f_7\alpha_2))X_0$$

Fifth Round: Verifier sends a random scalar α_0

Sixth Round: Prover computes $\pi_0 = \mathsf{fold}_\alpha(\pi_1)$ and sends it to the Verifier

The process of computing π_0 is as follows:

$$\pi_0[j] = \mathsf{fold}_\alpha(\pi_1[j], \pi_1[j+1]), \quad j = 0$$

Similarly, $\pi_0 = \mathsf{Enc}_0(f^{(0)})$, meaning π_0 is the codeword of $f^{(0)}$, where $f^{(0)}$ is the folding of f with respect to $(\alpha_0, \alpha_1, \alpha_2)$:

$$f^{(0)} = f(\alpha_0, \alpha_1, \alpha_2) = f_0 + f_1\alpha_0 + f_2\alpha_1 + f_3\alpha_0\alpha_1 + f_4\alpha_2 + f_5\alpha_0\alpha_2 + f_6\alpha_1\alpha_2 + f_7\alpha_0\alpha_1\alpha_2$$

At this point, the Commit-phase ends, and the Prover has sent (π_2, π_1, π_0) to the Verifier. Upon receiving them, the Verifier first checks whether π_0 is a constant polynomial. However, this alone is insufficient; the Verifier also needs to validate that the Prover's folding operations were honest. If all foldings π_i were to be verified, the Verifier would lose succinctness and, consequently, verification efficiency. Due to the Proximity Gap property, the Verifier only needs to perform a limited number of validations to ensure that π_i is a legitimate codeword.

3.3 Query-phase

Similar to the FRI protocol, in the Query-phase, the Verifier conducts multiple rounds of random sampling on the $(\pi_d, \pi_{d-1}, \ldots, \pi_0)$ sent by the Prover to verify the honesty of the folding process. We now discuss each round of the sampling process.

The Verifier will randomly select a position μ within π_d and send it to the Prover, noting that $0 \leq \mu < n_{d-1}$, where n_{d-1} pertains only to π_d. The Prover opens the points $\pi_d[\mu]$ and $\pi_d[\mu + n_{d-1}]$ and also sends the value at position μ in the folded codeword π_{d-1}, i.e., $\pi_{d-1}[\mu]$, along with the Merkle Path for these three points.

Upon receiving these, the Verifier first verifies that these three points correspond correctly to the codewords π_d and π_{d-1}. Then, the Verifier checks whether they satisfy the folding relationship:

$$\pi_{d-1}[\mu] \stackrel{?}{=} \mathsf{fold}_{\alpha_{d-1}}(\pi_d[\mu], \pi_d[\mu + n_{d-1}])$$

Merely verifying the folding relationship from π_d to π_{d-1} is insufficient. The Verifier must also validate the folding relationships from π_{d-1} to π_0. The Prover must additionally provide the points from π_{d-1} to π_{d-2}. Here, the Verifier does not need to select new random scalars but continues to use μ, because in the next round of folding, the position $\pi_{d-1}[\mu]$ will be folded with another symmetrical point regarding α_{d-2}. The specific symmetrical position depends on the situation: if $\mu < n_{d-2}$, then $\pi_{d-1}[\mu + n_{d-2}]$ is the symmetrical point; if $\mu \geq n_{d-2}$, then $\pi_{d-1}[\mu - n_{d-2}]$ is the symmetrical point. Assume $\mu \geq n_{d-2}$; then the Prover sends $\pi_{d-1}[\mu - n_{d-2}]$ and its Merkle Path to the Verifier to allow the Verifier to check the folding relationship from π_{d-1} to π_{d-2}.

In this way, by providing a single random scalar μ, the Verifier can verify all the folding relationships from π_d to π_0. This verification process constitutes one round.

To elevate reliability to a sufficient level, the Verifier must perform multiple rounds to ensure that the Prover has no room to cheat. The Query-phase leverages the Proximity Gap property. A cheating Prover who alters

the codeword is likely to be far from the legitimate encoding space, enabling the Verifier to detect cheating with only a small number of sampling attempts.

3.4 Summary

This article described the framework of the Commit-phase and Query-phase of the Basefold-IOPP protocol. This framework generalizes and extends the FRI protocol, expanding from RS-Codes to any Foldable Linear Codes. However, it is important to note that Basefold does not support codeword folding of degree greater than 2. This is because the Basefold-IOPP protocol must not only perform Proximity Testing but also provide an operational result of an MLE polynomial. This will be the topic of the next article in this series.

3.5 References

- [ZCF23] Zeilberger, H., Chen, B., Fisch, B. (2024). BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes. In: Reyzin, L., Stebila, D. (eds) Advances in Cryptology – CRYPTO 2024. CRYPTO 2024. Lecture Notes in Computer Science, vol 14929. Springer, Cham.
- [BCIKS20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity Gaps for Reed–Solomon Codes. In *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science*, pages 900–909, 2020.

Chapter 4

Notes on Basefold (Part III): MLE Evaluation Argument

- Yu Guo yu.guo@secbit.io
- Jade Xie jade@secbit.io

Assume we have an MLE polynomial $\tilde{f}(\vec{X}) \in \mathbb{F}[\vec{X}]^{\leq 1}$, an evaluation point $\mathbf{u} \in \mathbb{F}^d$, and the result of the polynomial's operation at the evaluation point $v = \tilde{f}(\mathbf{u})$. We aim to construct a Polynomial Evaluation Argument based on the Basefold-IOPP protocol.

Based on FRI, we can utilize the DEEP Method to construct a PCS protocol, which verifies the existence of a Low Degree Polynomial $q(X) = (f(X) - f(u))/(X - u)$. However, the FRI protocol can only handle the case of Univariate Polynomials. For MLE Polynomials, we generally need to use the following extended polynomial remainder theorem to reduce it to a quotient polynomial existence problem:

$$\tilde{f}(X_0, X_1, \dots, X_{n-1}) - v = \sum_{i=0}^{n-1} (X_k - u_k) \cdot q_i(X_0, X_1, \dots, X_{k-1})$$

For example, Virgo adopts this scheme for the MLE PCS protocol. However, Basefold [ZCF23] presents a refreshing approach to implementing the MLE Evaluation Argument by combining the Basefold-IOPP protocol and the Sumcheck protocol. We can use the Sumcheck protocol's summation and proof as the main framework of the protocol, then use Basefold-IOPP to ensure the Low degree of the MLE polynomial. Additionally, the evaluation of the MLE polynomial at random points generated after multiple rounds of folding in the Basefold-IOPP protocol compensates for the correctness proof of the evaluation required in the last round of the Sumcheck protocol.

4.1 Final Output of the Basefold-IOPP Protocol

For demonstration purposes, we use an MLE polynomial $\tilde{f}(X_0, X_1, X_2)$ with $d = 3$, defined as follows:

$$\tilde{f}(\vec{X}) = f_0 + f_1 \cdot X_0 + f_2 \cdot X_1 + f_3 \cdot X_0X_1 + f_4 \cdot X_2 + f_5 \cdot X_0X_2 + f_6 \cdot X_1X_2 + f_7 \cdot X_0X_1X_2$$

Here, $\mathbf{f}$ is the coefficient vector of $\tilde{f}$, defined above as the Coefficients Form of the MLE polynomial.

Let's revisit the folding process of the Basefold-IOPP protocol. First, let $f^{(3)}(X_0, X_1, X_2)$ correspond to the codeword after encoding $\tilde{f}$ with C_3. In each folding round, the Verifier sends a random challenge α_i to the Prover, who then generates $f^{(i)}$ by folding $f^{(i+1)}$, and sends the encoded codeword (as an Oracle) to the Verifier.

After one folding round of the Basefold-IOPP protocol, the polynomial corresponding to the codeword obtained by both parties, $f^{(2)}(X_0, X_1)$, is:

$$\begin{aligned} f^{(2)}(X_0, X_1) &= f^{(3)}(X_0, X_1, \alpha_2) \\ &= (f_0 + \alpha_2 \cdot f_4) + (f_1 + \alpha_2 \cdot f_5) \cdot X_0 + (f_2 + \alpha_2 \cdot f_6) \cdot X_1 + (f_3 + \alpha_2 \cdot f_7) \cdot X_0 X_1 \end{aligned}$$

After another folding round, the polynomial corresponding to the codeword, $f^{(1)}(X_0)$, is:

$$\begin{aligned} f^{(1)}(X_0) &= f^{(2)}(X_0, \alpha_1) \\ &= (f_0 + \alpha_2 \cdot f_4 + \alpha_1 \cdot f_2 + \alpha_2\alpha_1 \cdot f_6) + (f_1 + \alpha_2 \cdot f_5 + \alpha_1 \cdot f_3 + \alpha_2\alpha_1 \cdot f_7) \cdot X_0 \end{aligned}$$

Finally, the polynomial corresponding to the codeword, the **constant polynomial** $f^{(0)}$, is as follows:

$$\begin{aligned} f^{(0)} &= f^{(1)}(\alpha_0) \\ &= (f_0 + \alpha_2 \cdot f_4 + \alpha_1 \cdot f_2 + \alpha_2\alpha_1 \cdot f_6) + (f_1 + \alpha_2 \cdot f_5 + \alpha_1 \cdot f_3 + \alpha_2\alpha_1 \cdot f_7) \cdot \alpha_0 \\ &= f_0 + f_1\alpha_0 + f_2\alpha_1 + f_3\alpha_0\alpha_1 + f_4\alpha_2 + f_5\alpha_0\alpha_2 + f_6\alpha_1\alpha_2 + f_7\alpha_0\alpha_1\alpha_2 \end{aligned}$$

Upon closer examination, $f^{(0)}$ is exactly the evaluation of $\tilde{f}(X_0, X_1, X_2)$ at $(\alpha_0, \alpha_1, \alpha_2)$.

Thus, in the final round of the Basefold-IOPP protocol, the Prover sends the folded constant polynomial to the Verifier. The Verifier then uses the Query-phase to verify the correctness of each folding step. Meanwhile, the Verifier also obtains the evaluation result $v = \tilde{f}(\vec{X})$ at the random point $\vec{X} = (\alpha_0, \alpha_1, \alpha_2)$. From another perspective, the Basefold-IOPP protocol not only completes the Proximity proof but also provides an additional evaluation of $\tilde{f}(X_0, X_1, X_2)$ at a random point. More precisely, this is a proof of a vector inner product:

$$\langle \mathbf{f}, (1, \alpha_0) \otimes (1, \alpha_1) \otimes (1, \alpha_2) \rangle = v$$

However, this MLE evaluation point is not a pre-negotiated public input but is instead composed of random challenges generated during the execution of the Basefold-IOPP protocol.

In summary, in the final round of the IOPP protocol, the Prover folds to produce a constant polynomial, denoted as $f^{(0)}$, which is precisely $\tilde{f}$ evaluated at the random point $(\alpha_0, \alpha_1, \alpha_2)$. What we need is to prove the correctness of $\tilde{f}$ at a **public point**, such as (u_0, u_1, u_2).

$$\tilde{f}(u_0, u_1, u_2) = f_0 + f_1 \cdot u_0 + f_2 \cdot u_1 + f_3 \cdot u_0u_1 + f_4 \cdot u_2 + f_5 \cdot u_0u_2 + f_6 \cdot u_1u_2 + f_7 \cdot u_0u_1u_2$$

The subsequent question becomes: How can we use the evaluation of $\tilde{f}$ at a **random point** to prove the correctness of its evaluation at another **public point**?

4.2 Leveraging Sumcheck

We can revisit $\tilde{f}(u_0, u_1, u_2)$ by utilizing the properties of MLE. According to the definition of MLE, we have the following equation:

$$\mathsf{EQ}_1: \qquad \tilde{f}(X_0, X_1, X_2) = \sum_{\mathbf{b} \in \{0,1\}^3} \tilde{f}(b_0, b_1, b_2) \cdot \tilde{eq}_{(b_0, b_1, b_2)}(X_0, X_1, X_2)$$

Here, $\tilde{eq}$ is the Lagrange Polynomial of MLE, defined as:

$$\tilde{eq}_{(b_0,b_1,\ldots,b_{n-1})}(X_0, X_1, \ldots, X_{n-1})) = \prod_{i=0}^{n-1} ((1-b_i)(1-X_i) + b_i \cdot X_i)$$

It is easily verified that when $(X_0, X_1, X_2) = \mathbf{b}'$, $\mathbf{b}' \in \{0,1\}^3$, the left side of equation EQ_1 equals $\tilde{f}(\mathbf{b}')$. Observing each summand $\tilde{eq}_{\mathbf{b}}(\mathbf{b}')$, only when the vectors $\mathbf{b} = \mathbf{b}'$ are completely identical does $\tilde{eq}_{\mathbf{b}}(\mathbf{b}') = 1$; otherwise, they all equal 0. Therefore, the right side of the equation reduces to $\tilde{f}(\mathbf{b}') \cdot \tilde{eq}_{\mathbf{b}}(\mathbf{b}') = \tilde{f}(\mathbf{b}')$, ensuring the equality holds.

This means that both sides of equation EQ_1 agree on their values over the 3-dimensional Boolean HyperCube. Based on the uniqueness property of MLE, we can conclude that EQ_1 holds for any $\mathbf{X} \in F^3$.

Although the above is fundamental knowledge of MLE, note that equation EQ_1 introduces a new perspective: we can transform the evaluation problem of $\tilde{f}(u_0, u_1, u_2)$ into a Sumcheck problem:

$$\tilde{f}(u_0, u_1, u_2) = v = \sum_{\mathbf{b} \in \{0,1\}^3} \tilde{f}(\mathbf{b}) \cdot \tilde{eq}_{\mathbf{b}}(u_0, u_1, u_2)$$

It is immediately apparent that we can use the Sumcheck protocol to perform a "sum proof" on the above equation. An important function of the Sumcheck protocol is to transform a "summation problem" into two MLE polynomial evaluation problems (specifically for $\tilde{f}$ and $\tilde{eq}$) at a random point. However, in general, this approach is meaningless because, in the last round of Sumcheck, we need to prove the evaluation of $\tilde{f}$ at a new point, leading to a circular proof situation: we originally prove the evaluation of a polynomial at one point, then use the Sumcheck protocol to transform this proof into an evaluation at another point. This makes the Sumcheck protocol process redundant and useless. Typically, the protocol would rely on another MLE Evaluation Argument protocol, where the Prover sends the final evaluation and its correctness proof, thereby concluding the Sumcheck protocol. But here, we cannot rely on another MLE PCS protocol because our goal is to construct an MLE Evaluation Argument protocol, not to depend on an existing MLE PCS.

However, this problem is not difficult to solve because the previously introduced Basefold-IOPP protocol provides a byproduct: a proof of $\tilde{f}$'s evaluation at a **random point**. If the Sumcheck protocol and the Basefold-IOPP protocol share random challenges, the value provided by Basefold-IOPP in the final step can compensate for the correctness proof of the evaluation required in the last round of Sumcheck. Another remaining issue is how to handle the evaluation proof of $\tilde{eq}$. This is simpler because $\tilde{eq}$ is a public polynomial, allowing the Verifier to compute it independently without requiring the Prover to send an evaluation proof. Moreover, computing the evaluation of $\tilde{eq}$ only has a complexity of $O(\log N)$, which does not increase the Verifier's burden or affect the Verifier's succinctness. At this point, the Sumcheck protocol's role is no longer redundant but becomes crucial: **it transforms the problem of proving a polynomial's evaluation at a public point into a proof of its evaluation at a random point**.

Next, we need to synchronize the execution of the Basefold-IOPP protocol and the Sumcheck protocol, allowing both protocols to share a set of random challenges. This way, in the final step of both protocols, we can complete the collaboration, thereby ultimately proving the correctness of $\tilde{f}(u_0, u_1, u_2) = v$.

Following this approach, we attempt to outline the protocol flow when $s = 3$.

Public Inputs

1. Commitment of $\tilde{f}$, $\pi_3 = [\tilde{f}] = \mathsf{Enc}_3(\mathbf{f})$
2. Evaluation point $\mathbf{u} = (u_0, u_1, u_2)$
3. Operation value v

Witness

- Coefficient vector of MLE $\tilde{f}$, $\mathbf{f} = (f_0, f_1, f_2, f_3, f_4, f_5, f_6, f_7)$
- Evaluation vector of $\tilde{f}$, $\mathbf{a} = (a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7)$

First Round: Prover sends the univariate polynomial $h^{(2)}(X)$

$$h^{(2)}(X) = \sum_{b_0,b_1\in\{0,1\}^2} f(b_0, b_1, X) \cdot \tilde{eq}((b_0, b_1, X), (u_0, u_1, u_2))$$

Since the right side is a degree 2 Univariate Polynomial, the Prover can compute the coefficients of $h^{(2)}(X)$:

$$\begin{aligned} h_0^{(2)} &= \sum_{i=0}^{3} a_i \cdot e_i \\ h_1^{(2)} &= \sum_{i=0}^{3} a_{i+4} \cdot e_i + a_i \cdot e_{i+4} \\ h_2^{(2)} &= \sum_{i=0}^{3} a_{i+4} \cdot e_{i+4} \end{aligned}$$

Here, $\mathbf{e}$ is the evaluation vector of $\tilde{eq}(\mathbf{X}, \mathbf{u})$. It is easy to verify that $h^{(2)}(X) = h_0^{(2)} + h_1^{(2)} \cdot X + h_2^{(2)} \cdot X^2$.

Second Round: Verifier sends challenge $\alpha_2 \leftarrow \mathbb{F}_p$

Third Round: Prover and Verifier simultaneously execute the Basefold-IOPP protocol and the Sumcheck protocol:

- Prover computes $\tilde{f}^{(2)}(X_1, X_2) = f(\alpha_0, X_1, X_2)$

$$\tilde{f}^{(2)}(X_0, X_1) = (f_0 + f_4\alpha_2) + (f_1 + f_5\alpha_2) \cdot X_0 + (f_2 + f_6\alpha_2) \cdot X_1 + (f_3 + f_7\alpha_2) \cdot X_0X_1$$

- Prover sends the folded vector encoding: $\pi_2 = \mathrm{Enc}_2[\tilde{f}^{(2)}]$
- Prover computes $h^{(2)}(\alpha_2)$ as the summation value for the next round of the Sumcheck protocol
- Prover computes and sends $h^{(1)}(X)$

$$h^{(1)}(X) = \sum_{b_0\in\{0,1\}} f(b_0, X, \alpha_2) \cdot \tilde{eq}((b_0, X, \alpha_2), (u_0, u_1, u_2))$$

Since the right side is also a degree 2 Univariate Polynomial in X, the Prover can calculate the coefficients of $h^{(1)}(X)$: $(h_0^{(1)}, h_1^{(1)}, h_2^{(1)})$.

Fourth Round: Verifier sends challenge $\alpha_1 \leftarrow \mathbb{F}_p$

Fifth Round: Prover continues executing the Basefold-IOPP protocol and the Sumcheck protocol:

- Prover computes $\tilde{f}^{(1)}(X_2)$

$$\tilde{f}^{(1)}(X_2) = f(X_0, \alpha_1, \alpha_2) = (f_0 + f_4\alpha_2 + f_2\alpha_1 + f_6\alpha_1\alpha_2) + (f_1 + f_5\alpha_2 + f_3\alpha_1 + f_7\alpha_1\alpha_2) \cdot X_0$$

- Prover sends the folded vector encoding $\pi_1 = \mathrm{Enc}_1[\tilde{f}^{(1)}]$
- Prover computes the summation value for the next round of the Sumcheck protocol: $h^{(1)}(\alpha_1)$
- Prover computes and sends $h^{(0)}(X)$

$$h^{(0)}(X) = \sum_{b_0\in\{0,1\}} f(X, \alpha_1, \alpha_2) \cdot \tilde{eq}((b_0, \alpha_1, \alpha_2), (u_0, u_1, u_2))$$

Assume the coefficients of $h^{(0)}(X)$ are represented as $(h_0^{(0)}, h_1^{(0)}, h_2^{(0)})$.

Sixth Round: Verifier sends challenge $\alpha_0 \leftarrow \mathbb{F}_p$

Seventh Round: Prover continues executing the Basefold-IOPP protocol:

- Prover computes $\tilde{f}^{(0)}$, a **constant polynomial**

$$\tilde{f}^{(0)} = f(\alpha_0, \alpha_1, \alpha_2) = f_0 + f_1\alpha_0 + f_2\alpha_1 + f_3\alpha_0\alpha_1 + f_4\alpha_2 + f_5\alpha_0\alpha_2 + f_6\alpha_1\alpha_2 + f_7\alpha_0\alpha_1\alpha_2$$

- Prover sends the folded vector encoding $\pi_0 = \mathsf{Enc}_0[\tilde{f}^{(0)}]$

Eighth Round: Verifier validates the following equations:

1. Verifier verifies several rounds of Basefold Queries, $Q = \{q_i\}$
2. Verifier checks the correctness of each folding step in Sumcheck:

$$h^{(2)}(0) + h^{(2)}(1) \stackrel{?}{=} v$$
$$h^{(1)}(0) + h^{(1)}(1) \stackrel{?}{=} h^{(2)}(\alpha_2)$$
$$h^{(0)}(0) + h^{(0)}(1) \stackrel{?}{=} h^{(1)}(\alpha_1)$$

3. Verifier checks whether the final encoding π_0 is correct

$$\pi_0 \stackrel{?}{=} \mathsf{Enc}_0\left(\frac{h^{(0)}(\alpha_0)}{\tilde{eq}((\alpha_0, \alpha_1, \alpha_2), (u_0, u_1, u_2))}\right)$$

4.3 An Alternative Folding Method

In the Basefold paper, the PCS protocol requires that the MLE polynomial $\tilde{f}(X_0, X_1, \ldots, X_{d-1})$ be converted to the Coefficients Form beforehand, and then follow the protocol described in the previous section for the proof. However, in many Sumcheck-based SNARK systems, the Sumcheck protocol finally produces the Evaluation Form of the MLE polynomial, that is, the evaluation of the MLE polynomial on a Boolean HyperCube. Therefore, if we directly connect the SNARK protocol to Basefold-PCS, we would still need to convert the MLE polynomial from the Evaluation Form to the Coefficients Form. This conversion algorithm has a complexity of $O(N\log(N))$, where $N = 2^d$, and d is the number of variables in the MLE polynomial. Below is the definition of the Evaluation Form of $\tilde{f}(X_0, X_1, \ldots, X_{d-1})$:

$$\tilde{f}(X_0, X_1, \ldots, X_{d-1}) = \sum_{\mathbf{b} \in \{0,1\}^d} a_{\mathbf{b}} \cdot eq_{\mathbf{b}}(X_0, X_1, \ldots, X_{d-1})$$

As noted in the FRI-Binius paper, we do not need to convert the MLE polynomial from the Evaluation Form to the Coefficients Form. Instead, we can directly construct the PCS protocol in the Evaluation Form. In the previous section, the reason we needed the Coefficients Form of $\tilde{f}(X_0, X_1, \ldots, X_{d-1})$ was that the Basefold-IOPP protocol's Commit method required the coefficients of $\tilde{f}(X_0, X_1, \ldots, X_{d-1})$. If we follow the folding method of the Basefold-IOPP protocol described below, we can directly construct the PCS protocol in the Evaluation Form.

Now, let's consider the folding method of the Evaluation Form of $\tilde{f}(X_0, X_1, X_2)$. First, expand the definition of the Evaluation Form of $\tilde{f}(X_0, X_1, X_2)$:

$$\begin{aligned}\tilde{f}^{(3)}(X_0, X_1, X_2) &= \tilde{f}(X_0, X_1, X_2) \\ &= a_0 \cdot eq_{(0,0,0)}(X_0, X_1, X_2) + a_1 \cdot eq_{(1,0,0)}(X_0, X_1, X_2) + a_2 \cdot eq_{(0,1,0)}(X_0, X_1, X_2) + a_3 \cdot eq_{(1,1,0)}(X_0, X_1, X_2) \\ &+ a_4 \cdot eq_{(0,0,1)}(X_0, X_1, X_2) + a_5 \cdot eq_{(1,0,1)}(X_0, X_1, X_2) + a_6 \cdot eq_{(0,1,1)}(X_0, X_1, X_2) + a_7 \cdot eq_{(1,1,1)}(X_0, X_1, X_2)\end{aligned}$$

We can fold the above Evaluation Form, ultimately obtaining the evaluation $\tilde{f}^{(3)}(\alpha_0, \alpha_1, \alpha_2)$. For example, we can fold the Evaluation Form of $\tilde{f}(X_0, X_1, X_2)$ with respect to α_2 to obtain:

$$
\begin{aligned}
\tilde{f}^{(2)}(X_0, X_1) &= \tilde{f}^{(3)}(X_0, X_1, \alpha_2) \\
&= a_0 \cdot eq_{(0,0,0)}(X_0, X_1, \alpha_2) + a_1 \cdot eq_{(1,0,0)}(X_0, X_1, \alpha_2) + a_2 \cdot eq_{(0,1,0)}(X_0, X_1, \alpha_2) + a_3 \cdot eq_{(1,1,0)}(X_0, X_1, \alpha_2) \\
&+ a_4 \cdot eq_{(0,0,1)}(X_0, X_1, \alpha_2) + a_5 \cdot eq_{(1,0,1)}(X_0, X_1, \alpha_2) + a_6 \cdot eq_{(0,1,1)}(X_0, X_1, \alpha_2) + a_7 \cdot eq_{(1,1,1)}(X_0, X_1, \alpha_2)
\end{aligned}
$$

Next, we need to decompose $eq_{\mathbf{b}}(X_0, X_1\alpha_2)$:

$$
eq_{(b_0,b_1,b_2)}(X_0, X_1, \alpha_2) = eq_{(b_0,b_1)}(X_0, X_1) \cdot \Big((1 - b_2) \cdot (1 - \alpha_2) + b_2 \cdot \alpha_2\Big)
$$

When $b_2 = 0$, the right side simplifies to $(1 - \alpha_2) \cdot eq_{(b_0,b_1)}(X_0, X_1)$; when $b_2 = 1$, it simplifies to $\alpha_2 \cdot eq_{(b_0,b_1)}(X_0, X_1)$. Then, continue simplifying $\tilde{f}^{(2)}(X_0, X_1)$:

$$
\begin{aligned}
\tilde{f}^{(2)}(X_0, X_1) &= ((1 - \alpha_2) \cdot a_0 + \alpha_2 \cdot a_4) \cdot eq_{(0,0)}(X_0, X_1) + ((1 - \alpha_2) \cdot a_1 + \alpha_2 \cdot a_5) \cdot eq_{(0,1)}(X_0, X_1) \\
&+ ((1 - \alpha_2) \cdot a_2 + \alpha_2 \cdot a_6) \cdot eq_{(1,0)}(X_0, X_1) + ((1 - \alpha_2) \cdot a_3 + \alpha_2 \cdot a_7) \cdot eq_{(1,1)}(X_0, X_1)
\end{aligned}
$$

This is equivalent to folding (linear combination) the vectors (a_0, a_1, a_2, a_3) and (a_4, a_5, a_6, a_7) with $(1-\alpha_2, \alpha_2)$. Compared with the folding method in the Basefold-IOPP protocol, it uses $(1, \alpha_2)$ to fold (linear combination) the vectors (f_0, f_1, f_2, f_3) and (f_4, f_5, f_6, f_7). If we continue to fold in the new way, we can eventually get the same folding result as the Basefold-IOPP protocol.

$$
\begin{aligned}
\tilde{f}^{(1)}(X_0) &= \tilde{f}^{(2)}(X_0, \alpha_1) \\
&= ((1 - \alpha_2) \cdot a_0 + \alpha_2 \cdot a_4) \cdot eq_{(0,0)}(X_0, \alpha_1) + ((1 - \alpha_2) \cdot a_1 + \alpha_2 \cdot a_5) \cdot eq_{(1,0)}(X_0, \alpha_1) \\
&+ ((1 - \alpha_2) \cdot a_2 + \alpha_2 \cdot a_6) \cdot eq_{(0,1)}(X_0, \alpha_1) + ((1 - \alpha_2) \cdot a_3 + \alpha_2 \cdot a_7) \cdot eq_{(1,1)}(X_0, \alpha_1) \\
&= ((1 - \alpha_2) \cdot a_0 + \alpha_2 \cdot a_4) \cdot (1 - \alpha_1) \cdot eq_{(0)}(X_0) + ((1 - \alpha_2) \cdot a_1 + \alpha_2 \cdot a_5) \cdot (1 - \alpha_1) \cdot eq_{(1)}(X_0) \\
&+ ((1 - \alpha_2) \cdot a_2 + \alpha_2 \cdot a_6) \cdot \alpha_1 \cdot eq_{(0)}(X_0) + ((1 - \alpha_2) \cdot a_3 + \alpha_2 \cdot a_7) \cdot \alpha_1 \cdot eq_{(1)}(X_0) \\
&= \Big((1 - \alpha_1) \cdot ((1 - \alpha_2) \cdot a_0 + \alpha_2 \cdot a_4) + \alpha_1 \cdot ((1 - \alpha_2) \cdot a_2 + \alpha_2 \cdot a_6)\Big) \cdot eq_{(0)}(X_0) \\
&+ \Big((1 - \alpha_1) \cdot ((1 - \alpha_2) \cdot a_1 + \alpha_2 \cdot a_5) + \alpha_1 \cdot ((1 - \alpha_2) \cdot a_3 + \alpha_2 \cdot a_7)\Big) \cdot eq_{(1)}(X_0)
\end{aligned}
$$

Continuing the folding process, we eventually obtain:

$$
\begin{aligned}
\tilde{f}^{(0)} &= \tilde{f}^{(1)}(\alpha_0) \\
&= \Big((1 - \alpha_0) \cdot ((1 - \alpha_1) \cdot ((1 - \alpha_2) \cdot a_0 + \alpha_2 \cdot a_4) + \alpha_1 \cdot ((1 - \alpha_2) \cdot a_2 + \alpha_2 \cdot a_6)) \\
&\quad + \alpha_0 \cdot ((1 - \alpha_1) \cdot ((1 - \alpha_2) \cdot a_1 + \alpha_2 \cdot a_5) + \alpha_1 \cdot ((1 - \alpha_2) \cdot a_3 + \alpha_2 \cdot a_7))\Big) \\
&= \tilde{f}^{(3)}(\alpha_0, \alpha_1, \alpha_2)
\end{aligned}
$$

Can we improve the folding method of the Basefold-IOPP protocol so that it can directly fold the Evaluation Form of the MLE polynomial?

4.4 Basefold-IOPP Protocol Based on the Evaluation Form

Next, we attempt to outline the Basefold-IOPP protocol for the Evaluation Form of $\tilde{f}(X_0, X_1, X_2)$ when $d = 3$. First, we rewrite the folding function from the previous article:

$$\mathsf{fold}^*_\alpha(y_0, y_1) = (1-\alpha) \cdot \frac{t_j \cdot y_1 - t'_j \cdot y_0}{t_j - t'_j} + \alpha \cdot \frac{y_0 - y_1}{t_j - t'_j}$$

From earlier discussions, we know that the folding function has the following homomorphic property: after folding, the codeword is equivalent to the original message being folded and then encoded, expressed as:

$$\mathsf{fold}_\alpha(\mathsf{Enc}_i(\mathbf{m})) = \mathsf{Enc}_i(\mathsf{fold}_\alpha(\mathbf{m}))$$

Similarly, we can prove that the new folding function satisfies the above property. We can attempt to fold an element in the codeword π_i using the new folding function:

$$\begin{aligned}
\mathsf{fold}^*_\alpha(\pi_i[j], \pi_i[n_{i-1}+j]) &= \frac{1-\alpha}{t_j - t'_j} \cdot \Big(t_j \cdot (\mathbf{m}_l G_{i-1}[j] + t'_j \cdot \mathbf{m}_r G_{i-1}[j]) - t'_j \cdot (\mathbf{m}_l G_{i-1}[j] + t_j \cdot \mathbf{m}_r G_{i-1}[j])\Big) \\
&+ \frac{\alpha}{t_j - t'_j} \cdot \Big(\mathbf{m}_l G_{i-1}[j] + t_j \cdot \mathbf{m}_r G_{i-1}[j] - \mathbf{m}_l G_{i-1}[j] - t'_j \cdot \mathbf{m}_r G_{i-1}[j]\Big) \\
&= (1-\alpha) \cdot \mathbf{m}_l G_{i-1}[j] + \alpha \cdot \mathbf{m}_r G_{i-1}[j]
\end{aligned}$$

The above derivation demonstrates that the new folding function also satisfies the homomorphic property concerning the encoding function:

$$\mathsf{fold}^*_\alpha(\mathsf{Enc}_i(\mathbf{m})) = \mathsf{Enc}_i(\mathsf{fold}^*_\alpha(\mathbf{m}))$$

Additionally, we can prove that the new folding process does not affect the reliability of the Basefold-IOPP protocol, i.e., the Minimum Hamming Weight of the folded codeword remains above a lower bound.

Thus, we have an important conclusion: whether we use fold_α or fold^*_α, both can be used to construct a Proximity-Proof protocol without significant differences.

Next, we outline the Commit-phase protocol flow for better understanding:

Public Inputs

- Codeword of the MLE polynomial $\tilde{f}$, $\pi_3 = \mathsf{Enc}_3(\mathbf{a}) = \mathbf{a}G_3$

Witness

- Evaluation vector of the MLE polynomial $\tilde{f}$, $\mathbf{a} = (a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7)$

First Round: Verifier sends a random number α_2

Second Round: Prover computes $\pi_2 = \mathsf{fold}^*_\alpha(\pi_3)$ and sends it to the Verifier

The process of computing π_2 is as follows:

$$\pi_2[j] = \mathsf{fold}^*_\alpha(\pi_3[j], \pi_3[j+4]), \quad j \in \{0, 1, 2, 3\}$$

The computed $\pi_2 = \mathsf{Enc}_2(f^{(2)})$, meaning that π_2 is the codeword of $f^{(2)}$, where $f^{(2)}$ is the folded result of f with respect to α_2:

$$
\begin{aligned}
f^{(2)}(X_0, X_1) &= ((1-\alpha_2)a_0 + \alpha_2 a_4) \cdot eq_{(0,0)}(X_0, X_1) + ((1-\alpha_2)a_1 + \alpha_2 a_5) \cdot eq_{(1,0)}(X_0, X_1) \\
&+ ((1-\alpha_2)a_2 + \alpha_2 a_6) \cdot eq_{(0,1)}(X_0, X_1) + ((1-\alpha_2)a_3 + \alpha_2 a_7) \cdot eq_{(1,1)}(X_0, X_1) \\
&= f(X_0, X_1, \alpha_2)
\end{aligned}
$$

Third Round: Verifier sends a random number α_1

Fourth Round: Prover computes $\pi_1 = \mathsf{fold}^*_\alpha(\pi_2)$ and sends it to the Verifier

The process of computing π_1 is as follows:

$$
\pi_1[j] = \mathsf{fold}^*_\alpha(\pi_2[j], \pi_2[j+2]), \quad j \in \{0, 1\}
$$

The computed $\pi_1 = \mathsf{Enc}_1(f^{(1)})$, where $f^{(1)}$ is the folded result of $f^{(2)}$ with respect to α_1:

$$
\begin{aligned}
f^{(1)}(X_0) &= \Big((1-\alpha_1)((1-\alpha_2)a_0 + \alpha_2 a_4) + \alpha_1((1-\alpha_2)a_1 + \alpha_2 a_5)\Big) \cdot eq_{(0)}(X_0) \\
&+ \Big((1-\alpha_1)((1-\alpha_2)a_2 + \alpha_2 a_6) + \alpha_1((1-\alpha_2)a_3 + \alpha_2 a_7)\Big) \cdot eq_{(1)}(X_0) \\
&= f(X_0, \alpha_1, \alpha_2)
\end{aligned}
$$

Fifth Round: Verifier sends a random number α_0

Sixth Round: Prover computes $\pi_0 = \mathsf{fold}_\alpha(\pi_1)$ and sends it to the Verifier

The process of computing π_0 is as follows:

$$
\pi_0[j] = \mathsf{fold}^*_\alpha(\pi_1[j], \pi_1[j+1]), \quad j = 0
$$

Similarly, $\pi_0 = \mathsf{Enc}_0(f^{(0)})$, meaning π_0 is the codeword of $f^{(0)}$, where $f^{(0)}$ is the folded result of f with respect to $(\alpha_0, \alpha_1, \alpha_2)$:

$$
f^{(0)} = f(\alpha_0, \alpha_1, \alpha_2)
$$

Then, we can improve the Evaluation Argument protocol.

4.5 Basefold Evaluation Argument Protocol Based on the Evaluation Form

Public Inputs

1. Commitment of $\tilde{f}$, $\pi_3 = \mathsf{Enc}_3(\mathbf{a})$
2. Evaluation point $\mathbf{u}$
3. Operation value $v = \tilde{f}(\mathbf{u})$

Witness

- Evaluation vector of MLE $\tilde{f}$, $\mathbf{a} = (a_0, a_1, \ldots, a_7)$

Such that

$$
\tilde{f}(X_0, X_1, X_2) = \sum_{\mathbf{b} \in \{0,1\}^3} a_{\mathbf{b}} \cdot eq_{\mathbf{b}}(X_0, X_1, X_2)
$$

First Round: Prover sends the evaluations of $h^{(2)}(X)$ at points $(0, 1, 2)$

$$h^{(2)}(X) = \sum_{b_1, b_2 \in \{0,1\}^2} f(X, b_1, b_2) \cdot \tilde{eq}((X, b_1, b_2), \mathbf{u})$$

Since the right side is a degree 2 Univariate Polynomial, the Prover can compute the evaluations of $h^{(2)}(X)$ at $X = 0, 1, 2$:

$$\begin{aligned}
h^{(2)}(0) &= a_0 \cdot e_0 + a_1 \cdot e_1 + a_2 \cdot e_2 + a_3 \cdot e_3 \\
h^{(2)}(1) &= a_4 \cdot e_4 + a_5 \cdot e_5 + a_6 \cdot e_6 + a_7 \cdot e_7 \\
h^{(2)}(2) &= \sum_{i=0}^{3} (2 \cdot a_{i+4} - a_i) \cdot (2 \cdot e_{i+4} - e_i)
\end{aligned}$$

Here, $\mathbf{e}$ is the evaluation vector of $\tilde{eq}(\mathbf{X}, \mathbf{u})$.

Second Round: Verifier sends challenge $\alpha_2 \leftarrow \mathbb{F}_p$

Third Round: Prover simultaneously executes the Basefold-IOPP protocol and the Sumcheck protocol:

- Prover sends the folded vector encoding: $\pi_2 = \mathsf{fold}^*_{\alpha_2}(\pi_3)$
- Prover computes $h^{(2)}(\alpha_2)$ as the summation value for the next round of the Sumcheck protocol
- Prover computes the evaluations vector of $f^{(2)}(X_0, X_1)$: $\mathbf{a}^{(2)} = \mathsf{fold}^*_{\alpha_2}(\mathbf{a})$
- Prover computes and sends $h^{(1)}(X)$

$$h^{(1)}(X) = \sum_{b_0 \in \{0,1\}} f(b_0, X, \alpha_2) \cdot \tilde{eq}((b_0, X, \alpha_2), (u_2, u_1, u_0))$$

Since the right side is also a degree 2 Univariate Polynomial in X, the Prover can compute the evaluations of $h^{(1)}(X)$ at $X = 0, 1, 2$: $(h^{(1)}(0), h^{(1)}(1), h^{(1)}(2))$.

Fourth Round: Verifier sends challenge $\alpha_1 \leftarrow \mathbb{F}_p$

Fifth Round: Prover continues executing the Basefold-IOPP protocol and the Sumcheck protocol:

- Prover sends the folded vector encoding $\pi_1 = \mathsf{fold}^*_{\alpha_1}(\pi_2)$
- Prover computes the summation value for the next round of the Sumcheck protocol: $h^{(1)}(\alpha_1)$
- Prover computes the evaluations vector of $\mathbf{a}^{(1)} = \mathsf{fold}^*_{\alpha_1}(\mathbf{a}^{(2)})$
- Prover computes and sends the evaluations of $h^{(0)}(X)$ at $X = 0, 1, 2$: $(h^{(0)}(0), h^{(0)}(1), h^{(0)}(2))$

$$h^{(0)}(X) = f(X_0, \alpha_1, \alpha_2) \cdot \tilde{eq}((X_0, \alpha_1, \alpha_2), (u_0, u_1, u_2))$$

Sixth Round: Verifier sends challenge $\alpha_0 \leftarrow F$

Seventh Round: Prover continues executing the Basefold-IOPP protocol:

- Prover sends the folded vector encoding $\pi_0 = \mathsf{fold}^*_{\alpha_0}(\pi_1)$

Eighth Round: Verifier validates the following equations:

1. Verifier sends several rounds of Query, $Q = \{q_i\}$
2. Verifier checks the correctness of each folding step in Sumcheck:

$$h^{(2)}(0) + h^{(2)}(1) \stackrel{?}{=} v$$
$$h^{(1)}(0) + h^{(1)}(1) \stackrel{?}{=} h^{(2)}(\alpha_2)$$
$$h^{(0)}(0) + h^{(0)}(1) \stackrel{?}{=} h^{(1)}(\alpha_1)$$

3. Verifier checks whether the final encoding π_0 is correct

$$\pi_0 \stackrel{?}{=} \mathsf{enc}_0 \left(\frac{h^{(0)}(\alpha_0)}{\tilde{eq}((\alpha_0, \alpha_1, \alpha_2), \mathbf{u})} \right)$$

At this point, we have obtained a Basefold Evaluation Argument protocol based on the Evaluation Form. It can be directly connected to Sumcheck-based zkSNARKs or similar Jolt-style zkVMs.

4.6 References

- [ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. "BaseFold: efficient field-agnostic polynomial commitment schemes from foldable codes." Annual International Cryptology Conference. Cham: Springer Nature Switzerland, 2024.
- Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. "Transparent polynomial delegation and its applications to zero knowledge proof." In *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 859-876. IEEE, 2020.

Chapter 5

Notes on BaseFold (Part IV): Random Foldable Codes

- Jade Xie jade@secbit.io
- Yu Guo yu.guo@secbit.io

Previous articles have mentioned that BaseFold extends the FRI IOPP by introducing the concept of *foldable codes*. Additionally, by combining the Sumcheck protocol, it can support PCS for multi-linear polynomials. The next crucial question is how to explicitly construct such *foldable codes*. We aim for these foldable codes to possess the following properties:

1. **Efficient Encoding**
2. **Field Agnostic**, i.e., applicable even for small fields
3. **Compatible with PCS for Multi-linear Polynomials**

Another important aspect of encoding is the consideration of the Minimum Relative Hamming Distance. If readers are familiar with the FRI protocol, the Reed-Solomon codes used are likely not unfamiliar. They have a desirable property: their distance meets the Singleton bound, i.e., $d = n - k + 1$, and are thus known as Maximum Distance Separable (MDS) codes. These codes balance code length and error-correcting capability effectively, providing strong error detection and correction with minimal redundancy, thereby saving encoding space. In PCS protocols, this allows verifiers to perform checks more efficiently. Therefore, from a practical perspective, we also desire that such foldable codes satisfy the fourth property:

4. **Good Relative Minimum Distance**

The BaseFold paper [ZCF23] constructs a type of code called *Random Foldable Code* (RFCs), which satisfies the aforementioned properties. Next, we will explore how it achieves these points.

5.1 Efficient Encoding Algorithms

The first article in this series has already introduced the concept of *foldable linear codes* and the BaseFold encoding algorithm. Here is a brief review.

Definition 1 [ZCF23, Definition 5] ((c, k_0, d) - Foldable Linear Codes). Let $c, k_0, d \in \mathbb{N}$ and $\mathbb{F}$ denote a finite field. A linear code $C_d : \mathbb{F}^{k_0 \cdot 2^d} \to \mathbb{F}^{ck_0 \cdot 2^d}$ with generator matrix $\mathbf{G}_d$ is called ***foldable*** if there exists a sequence of generator matrices $(\mathbf{G}_0, \ldots, \mathbf{G}_{d-1})$ and diagonal matrices $(T_0, \ldots, T_{d-1})$ and $(T'_0, \ldots, T'_{d-1})$ such that for any $i \in [1, d]$, the following holds:

1. The diagonal matrices $T_{i-1}, T'_{i-1} \in F^{ck_0 \cdot 2^{i-1} \times ck_0 \cdot 2^{i-1}}$ satisfy $\mathrm{diag}(T_{i-1})[j] \neq \mathrm{diag}(T'_{i-1})[j]$ for all $j \in [ck_0 \cdot 2^{i-1}]$;
2. The matrix $\mathbf{G}_i \in F^{k_0 \cdot 2^i \times ck_0 \cdot 2^i}$ (arranged row-wise) is equal to

$$\mathbf{G}_i = \begin{bmatrix} \mathbf{G}_{i-1} & \mathbf{G}_{i-1} \\ \mathbf{G}_{i-1} \cdot T_{i-1} & \mathbf{G}_{i-1} \cdot T'_{i-1} \end{bmatrix}.$$

To efficiently construct a foldable linear code, a uniform sampling method is employed by first defining a set of random foldable distributions.

Definition 2 [ZCF23, Definition 9] ((c, k_0) - Foldable Distributions). Fix a finite field $\mathbb{F}$ and $c, k_0 \in \mathbb{N}$. Let $\mathbf{G}_0 \in \mathbb{F}^{k_0 \times ck_0}$ be the generator matrix of an $[ck_0, k_0]$ linear code that satisfies maximum distance separability, and let D_0 be the distribution that outputs $\mathbf{G}_0$ with probability 1. For each $i > 0$, we recursively define the distribution D_i, which samples the generator matrices $(\mathbf{G}_0, \mathbf{G}_1, \ldots, \mathbf{G}_i)$ where $\mathbf{G}_i \in F^{k_i \times n_i}$ with $k_i := k_0 \cdot 2^i$, $n_i := ck_i$:

1. Sample $(\mathbf{G}_0, \ldots, \mathbf{G}_{i-1}) \leftarrow D_{i-1}$;
2. Sample $\mathrm{diag}(T_{i-1}) \leftarrow \$(\mathbb{F}^\times)^{n_{i-1}}$ and define $\mathbf{G}_i$ as

$$\mathbf{G}_i = \begin{bmatrix} \mathbf{G}_{i-1} & \mathbf{G}_{i-1} \\ \mathbf{G}_{i-1} \cdot T_{i-1} & \mathbf{G}_{i-1} \cdot -T_{i-1} \end{bmatrix}.$$

Once the initial generator matrix $\mathbf{G}_0$ is determined, uniformly sample n_0 random elements from $\mathbb{F}^\times$ (i.e., excluding the zero element) to generate the diagonal elements of T_0, thereby obtaining the next generator matrix $\mathbf{G}_1$. This process is then recursively applied to generate $(\mathbf{G}_2, \ldots, \mathbf{G}_i)$. In PCS, generating foldable codes via uniform sampling aids in achieving an efficient prover.

Note that the above definition requires the initial $\mathbf{G}_0$ to be the generator matrix of a linear code satisfying the MDS property. However, as mentioned in a footnote in [ZCF23], this requirement is not strictly necessary. Including this property is merely for simplifying the analysis of the code distance later on. In fact, the distance analysis holds for any linear code.

Protocol 1 Enc_d **[ZCF23, Protocol 1]: BaseFold Encoding Algorithm**

Input: Original message $\mathbf{m} \in \mathbb{F}^{k_d}$

Output: $\mathbf{w} \in \mathbb{F}^{n_d}$ such that $\mathbf{w} = \mathbf{m} \cdot \mathbf{G}_d$

Parameters: $\mathbf{G}_0$ and diagonal matrices $(T_0, T_1, \ldots, T_{d-1})$

1. If $d = 0$ (i.e., $\mathbf{m} \in \mathbb{F}^{k_0}$):
   - (a) Return $\mathrm{Enc}_0(\mathbf{m})$
2. Else:
   - (a) Split $\mathbf{m} := (\mathbf{m}_l, \mathbf{m}_r)$
   - (b) Let $\mathbf{l} := \mathrm{Enc}_{d-1}(\mathbf{m}_l)$, $\mathbf{r} := \mathrm{Enc}_{d-1}(\mathbf{m}_r)$, and $\mathbf{t} = \mathrm{diag}(T_{d-1})$
   - (c) Return $(\mathbf{l} + \mathbf{t} \circ \mathbf{r}, \mathbf{l} - \mathbf{t} \circ \mathbf{r})$

By analyzing Protocol 1, we can see that encoding to obtain C_d requires only $\frac{dn_d}{2}$ field multiplications and dn_d field additions, i.e., $0.5n \log n$ field multiplications and $n \log n$ field additions. Overall, the encoding complexity is $O(n \log n)$. Thus, we have introduced the explicit construction of Random Linear Foldable Codes provided by BaseFold and verified that they indeed support efficient encoding.

5.2 Polynomials Save the World: Polynomial-Based Encoding

Next, we examine the second and third properties of *Random Foldable Codes*:

2. **Field Agnostic**, i.e., applicable even for small fields
3. **Compatible with PCS for Multi-linear Polynomials**

As mentioned earlier, Reed-Solomon codes can achieve the Singleton bound, but only when the alphabet size is relatively large (i.e., $q \gg n$). Fortunately, we can extend Reed-Solomon codes to Reed-Muller codes, transitioning from univariate polynomial codes to multivariate polynomial codes. This allows applicability over

small fields ($q \ll n$), although there is a slight trade-off in the balance between distance and error-correcting capability. However, this is worthwhile.

Appendix D of [ZCF23] informs us that *Random Foldable Codes* are a special case of truncated Reed-Muller codes (Punctured Reed-Muller Codes). Thus, *Random Foldable Codes* are field agnostic and, by venturing into the realm of multivariate polynomials, are suitable for PCS of multi-linear polynomials.

We know that the encoding space of Reed-Solomon codes consists of univariate polynomials of degree at most d. Reed-Muller codes extend this to multivariate polynomials, with the encoding space comprising multivariate polynomials of total degree at most d.

For n, d, q with $d < q$, define Reed-Muller encoding as ([GJX15])

$$\mathrm{RM}_q(n,d) := \{(F(\mathbf{u}))_{\mathbf{u}\in\mathbb{F}_q^n} : F \in \mathbb{F}_q[X_1, \cdots, X_n], \deg(F) \leq d\}.$$

Reed-Muller codes represent the set of evaluations of n-variate polynomials of total degree at most d over $\mathbb{F}_q^n$. The length of the encoding $\mathrm{RM}_q(n,d)$ is q^n, and the dimension is $\binom{n+d}{n}$.

Intuitively, Punctured Reed-Muller Codes are simply Reed-Muller codes with truncation. Specifically, the evaluation points $\mathbf{u}$ are not taken from all of $\mathbb{F}_q^n$ but only a subset, denoted as $\mathcal{T} = \{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_N\}$. Typically, Punctured Reed-Muller Codes allow this set $\mathcal{T}$ to be a multiset (i.e., permitting duplicate elements), but we do not impose this requirement here. Let $\mathrm{RM}_q(n,d)|_{\mathcal{T}}$ denote the $\mathbb{F}_q$-linear code:

$$\mathrm{RM}_q(n,d)|_{\mathcal{T}} := \{(F(\mathbf{u}_1), F(\mathbf{u}_2), \cdots, F(\mathbf{u}_N)) : F \in \mathbb{F}_q[X_1, \cdots, X_n], \deg(F) \leq d\}.$$

This is called a punctured Reed-Muller code ([GJX15]). From the definition, it is evident that this is merely a subset of Reed-Muller codes, selecting only N points, which aligns with the literal meaning of "truncated" or "punctured."

With the concept of Punctured Reed-Muller Codes established, let's examine the following lemma provided in Appendix D of [ZCF23], which states that foldable linear codes are a special case of punctured Reed-Muller codes.

Lemma 1 [ZCF23, Lemma 11] (Foldable Punctured Reed-Muller Codes). Let C_d be a foldable linear code with generator matrices $(\mathbf{G}_0, \dots, \mathbf{G}_{d-1})$ and diagonal matrices $(T_0, \dots, T_{d-1})$, $(T'_0, \dots, T'_{d-1})$. Then there exists a subset $D \subset \mathbb{F}^d$ such that $C_d = \{(P(\mathbf{x}) : \mathbf{x} \in D) : P \in \mathbb{F}[X_1, \dots, X_d]\}$, i.e., each codeword in C_d is a vector obtained by evaluating a multilinear polynomial P at each point in D.

Proof: By induction. For simplicity, consider C_0 as a repetition code. In the base case, $\mathrm{Enc}_0(m) = m\|\dots\|m$ is a constant polynomial $P \equiv m$ evaluated at c distinct points. Assume that for $i < d$, there exists a set D_i such that $C_i = \{(P(\mathbf{x}) : \mathbf{x} \in D_i) : P \in \mathbb{F}[X_1, \dots, X_i]\}$. Without loss of generality, assign an integer $j \in [1, c \cdot 2^i]$ to index each element in D_i sequentially, representing x_j as the j-th element in D_i.

Let $t = \mathrm{diag}(T_i)$, $t' = \mathrm{diag}(T'_i)$, $n_i = c \cdot 2^i$, $\mathbf{v} \in \mathbb{F}^{2^{i+1}}$, and let $P \in \mathbb{F}[X_1, \dots, X_{i+1}]$ be a polynomial with coefficients from $\mathbf{v}$. Finally, let $P_l, P_r \in \mathbb{F}[X_1, \dots, X_i]$ such that $P(X_1, \dots, X_{i+1}) = P_l(X_1, \dots, X_i) + X_{i+1}P_r(X_1, \dots, X_i)$. Then,

$$
\begin{aligned}
\mathrm{Enc}_{i+1}(\mathbf{v}) &= \mathrm{Enc}_i(\mathbf{v}_l) + \mathrm{diag}(T_i) \circ \mathrm{Enc}_i(\mathbf{v}_r) \quad \| \quad \mathrm{Enc}_i(\mathbf{v}_l) + \mathrm{diag}(T_i) \circ \mathrm{Enc}_i(\mathbf{v}_r) \\
&\quad \text{(by the encoding algorithm in Protocol 1)} \\
&= (P_l(\mathbf{x}_1), \ldots, P_l(\mathbf{x}_n)) + \mathrm{diag}(T_i) \circ (P_r(\mathbf{x}_1), \ldots, P_r(\mathbf{x}_n)) \\
&\| \quad (P_l(\mathbf{x}_1), \ldots, P_l(\mathbf{x}_n)) + \mathrm{diag}(T_i') \circ (P_r(\mathbf{x}_1), \ldots, P_r(\mathbf{x}_n)) \\
&\quad \text{(by the induction hypothesis)} \\
&= (P_l(\mathbf{x}_1) + t_1 P_r(\mathbf{x}_1), \ldots, P_l(\mathbf{x}_n) + t_n P_r(\mathbf{x}_n), P_l(\mathbf{x}_1) + t_1' P_r(\mathbf{x}_1), \ldots, P_l(\mathbf{x}_n) + t_n' P_r(\mathbf{x}_n)) \\
&\quad \text{(by the definition of the Hadmard product)} \\
&= (P(\mathbf{x}_1, t_1), \ldots, P(\mathbf{x}_n, t_n), P(\mathbf{x}_1, t_1'), \ldots, P(\mathbf{x}_n, t_n')) \\
&\quad \text{(by the definition of } P\text{)}
\end{aligned}
$$

Therefore, let $D_{i+1} = \{(\mathbf{x}_1, t_1), \ldots, (\mathbf{x}_n, t_n), (\mathbf{x}_1, t_1'), \ldots, (\mathbf{x}_n, t_n')\}$, and the lemma holds for $i+1$. Thus, by induction, the proof is complete. □

5.3 Good Relative Minimum Distance

Finally, we focus on the fourth property satisfied by *Random Foldable Codes* (RFCs):

4. **Good Relative Minimum Distance**

In [ZCF23], it is proven that RFCs have tight bounds on their minimum Hamming distance (a "tight" bound means that the actual bounds are achievable). For example, an RFC over a 256-element finite field with a message length of 2^{25} and a code rate of $\frac{1}{8}$ has a relative minimum distance of 0.728 with overwhelming probability. For a code with a rate of $\frac{1}{8}$, the maximum achievable relative minimum Hamming distance is approximately $1 - \frac{1}{8} = 0.875$. Clearly, 0.728 is fairly close to 0.875. This is practically useful and implies that foldable codes generated via overwhelming probability (which enable efficient PCS provers) also have good relative minimum distances (enabling efficient PCS verifiers).

The term "overwhelming probability" arises from the distribution $(\mathbf{G}_0, \ldots, \mathbf{G}_d) \leftarrow D_d$ introduced during encoding. When uniformly sampling diagonal matrices T_i from $\mathbb{F}^\times$ and setting $T_i' = -T_i$, the relative minimum distance of C_d achieved with overwhelming probability is equal to

$$
1 - \left(\frac{\epsilon_{\mathbb{F}}^d}{c} + \frac{\epsilon_{\mathbb{F}}}{\log|\mathbb{F}|} \sum_{i=0}^{d} (\epsilon_{\mathbb{F}})^{d-i} \left(0.6 + \frac{2\log(n_i/2) + \lambda}{n_i} \right) \right) \tag{1}
$$

where c is the reciprocal of the code rate, $\epsilon_{\mathbb{F}} = \frac{\log|\mathbb{F}|}{\log|\mathbb{F}| - 1.001}$, n_i is the encoding length, d is the logarithm of the message length, and λ is the security parameter. By setting $\lambda = 128$, it ensures that (c, k_0, d)-random foldable linear codes achieve the above relative minimum distance with a probability of at least $1 - 2^{-128}$.

Next, we examine how the result in equation (1) is derived. Our goal is to analyze the relative minimum distance of foldable random codes C_d. For a linear code, the minimum distance equals the minimum Hamming weight among all non-zero codewords because

$$
d = \min_{\substack{\vec{c_1} \neq \vec{c_2} \\ \vec{c_1}, \vec{c_2} \in C_d}} \Delta(\vec{c_1}, \vec{c_2}) = \min_{\substack{\vec{c_1} \neq \vec{c_2} \\ \vec{c_1}, \vec{c_2} \in C_d}} wt(\vec{c_1} - \vec{c_2}) = \min_{\vec{c} \neq \vec{0}, \vec{c} \in C_d} wt(\vec{c})
$$

Since it is a linear code, $\vec{c_1} - \vec{c_2}$ is also a codeword in C_d, hence the last equality holds. Therefore, we want to show that for any non-zero message, i.e., $\forall \vec{m} \neq \vec{0}$, the encoded codeword $\mathrm{Enc}_d(\vec{m})$ does not have too many zero components. Suppose it has at most t_d zero components, letting $\mathrm{nzero}(\cdot)$ denote the number of zero components in a vector, we aim to show

$$
\forall \vec{m} \neq \vec{0}, \quad \mathrm{nzero}(\mathrm{Enc}_d(\vec{m})) \leq t_d \tag{2}
$$

Let n_d denote the length of the codeword $\text{Enc}_d(\vec{m})$. Then from (2), we have

$$\forall \vec{m} \neq \vec{0}, \quad wt(\text{Enc}_d(\vec{m})) \geq n_d - t_d$$

Thus, the relative minimum distance that C_d can achieve is

$$\Delta(C_d) = \frac{\min_{\vec{c}\neq\vec{0}, \vec{c}\in C_d} wt(\vec{c})}{n_d} = \frac{n_d - t_d}{n_d} = 1 - \frac{t_d}{n_d} \tag{3}$$

The result in equation (1) is derived from equation (3). The remaining task is to analyze what t_d equals, that is, how many zero components a codeword can have after encoding any non-zero message.

5.3.1 Utilizing Induction

Using the powerful tool of induction, we analyze t_d. Assume that with overwhelming probability (based on the choice of diagonal matrices $T_0, \ldots, T_{i-1}$), for any non-zero message $\vec{m} \in \mathbb{F}^{k_i} \setminus \{0^{k_i}\}$, the encoded $\text{Enc}_i(\vec{m})$ has at most t_i zero components. We analyze the case for $i+1$. For any non-zero message $\vec{m} = (\vec{m}_l, \vec{m}_r) \in \mathbb{F}^{2k_i}$,

$$\begin{aligned}
\text{Enc}_{i+1}(\vec{m}) &= (\vec{m}_l, \vec{m}_r) \begin{bmatrix} \mathbf{G}_i & \mathbf{G}_i \\ \mathbf{G}_i \cdot T_i & \mathbf{G}_i \cdot -T_i \end{bmatrix} \\
&= (\vec{m}_l \mathbf{G}_i + \vec{m}_l \mathbf{G}_i \cdot T_i, \vec{m}_l \mathbf{G}_i - \vec{m}_l \mathbf{G}_i \cdot T_i) \\
&= (\text{Enc}_i(\vec{m}) + \text{Enc}_i(\vec{m}) \circ \text{diag}(T_i), \text{Enc}_i(\vec{m}) - \text{Enc}_i(\vec{m}) \circ \text{diag}(T_i)) \\
&:= (\mathbf{M}_l \| \mathbf{M}_r)
\end{aligned}$$

This means examining the number of zero components in the vector $(\mathbf{M}_l \| \mathbf{M}_r)$. Separating $\mathbf{M}_l$ and $\mathbf{M}_r$:

$$\begin{aligned}
\mathbf{M}_l &= \text{Enc}_i(\vec{m}_l) + \text{Enc}_i(\vec{m}_r) \circ \text{diag}(T_i) \\
\mathbf{M}_r &= \text{Enc}_i(\vec{m}_l) - \text{Enc}_i(\vec{m}_r) \circ \text{diag}(T_i)
\end{aligned}$$

Let $\mathbf{t} = \text{diag}(T_i)$. For each $j \in [1, n_i]$, define $A_j = \text{Enc}_i(\vec{m}_l)[j]$, $B_j = \text{Enc}_i(\vec{m}_r)[j]$, and define a function:

$$f_j(x) = A_j + xB_j \tag{4}$$

If $f_j(\mathbf{t}[j]) = 0$ or $f_j(-\mathbf{t}[j]) = 0$, then $\mathbf{M}_l[j] = 0$ or $\mathbf{M}_r[j] = 0$, indicating a zero component in the encoded vector. Let's analyze whether $f_j(x)$ can be zero based on the values of A_j and B_j, considering the following cases:

	$A_j = 0$	$A_j \neq 0$
$B_j = 0$	$f_j(x) \equiv 0$	$f_j(x) = A_j \neq 0$
$B_j \neq 0$	$f_j(x) = xB_j$	$f_j(x) = A_j + xB_j$

First, consider the case where $A_j = B_j = 0$. Here, $f_j(x) \equiv 0$ for any x, meaning $\mathbf{M}_l[j] = 0$ and $\mathbf{M}_r[j] = 0$. Let $S \subseteq [n_i]$ denote such indices, and by the induction hypothesis, $|S| \leq t_i$. Define $m_{i+1}(S)$ as those non-zero messages that satisfy $A_j = B_j = 0$, i.e.,

$$S = \{j \in [1, n_i] : A_j = B_j = 0\}.$$

In this case, $\mathbf{M}_l[j] = \mathbf{M}_r[j] = 0$, resulting in $2|S|$ zero components in $(\mathbf{M}_l \| \mathbf{M}_r)$.

Consider the second case, where $A_j \neq 0$ and $B_j = 0$. Here, $f_j(x) = A_j \neq 0$, so no zero components are found.

Next, consider the last row of the table where $B_j \neq 0$. In this case, the index j is certainly not in S. Define a subset $\urcorner S^* \subseteq \urcorner S$ such that

$$\urcorner S^* = \{j \in [1, n_i] \backslash S, B_j \neq 0\}$$

For each $j \in \urcorner S^*$, define a random variable

$$X_j = 1\{f_j(\mathbf{t}[j]) = 0\} + 1\{f_j(-\mathbf{t}[j]) = 0\}$$

where $1(\cdot)$ is an indicator function that equals 1 if the condition inside holds, and 0 otherwise. Thus, X_j indicates how many zero components exist at position j in $\mathbf{M}_l$ and $\mathbf{M}_r$, with possible values $\{0, 1, 2\}$. Notice that X_j is an independent Bernoulli trial because $\mathbf{t}[j]$ is uniformly sampled from $\mathbb{F}^\times$. Let $z_j \in \mathbb{F}^\times$ satisfy $f_j(z_j) = 0$. Then, when $\mathbf{t}[j] = z_j$, $1\{f_j(\mathbf{t}[j]) = 0\} = 1$ and when $\mathbf{t}[j] = -z_j$, $1\{f_j(-\mathbf{t}[j]) = 0\} = 1$. Analyzing the possible values of X_j:

1. $X_j = 2$: This implies $f_j(\mathbf{t}[j]) = 0$ and $f_j(-\mathbf{t}[j]) = 0$, which leads to $\mathbf{t}[j] = z_j = -z_j$. This would mean $z_j = 0$, which is impossible since $z_j \in \mathbb{F}^\times$.
2. $X_j = 1$: This implies either $f_j(\mathbf{t}[j]) = 0$ or $f_j(-\mathbf{t}[j]) = 0$, meaning $\mathbf{t}[j] = z_j$ or $\mathbf{t}[j] = -z_j$. The probability of this occurring is $\frac{2}{|\mathbb{F}|-1}$.
3. $X_j = 0$: This occurs when $\mathbf{t}[j] \neq z_j$ and $\mathbf{t}[j] \neq -z_j$, with probability $1 - \frac{2}{|\mathbb{F}|-1}$.

For all $j \in \urcorner S^*$, summing up all X_j gives the total number of zero components in $(\mathbf{M}_l \| \mathbf{M}_r)$, denoted as $X = \sum_{j \in \urcorner S^*} X_j$.

Having analyzed all cases in the table, we obtain

$$\text{nzero}(\text{Enc}_{i+1}(\vec{m})) = 2|S| + X$$

Next, we analyze $|S|$ and X to show that for any non-zero message $\vec{m} \in \mathbb{F}^{2k_i} \backslash \{0^{2k_i}\}$, $\text{Enc}_{i+1}(\vec{m})$ has at most t_{i+1} zero components with overwhelming probability. We analyze the probability that $\text{Enc}_{i+1}(\vec{m})$ has at least $2t_i + l_i$ zero components:

$$
\begin{aligned}
\Pr[\mathrm{nzero}(\mathrm{Enc}_{i+1}(\vec{m})) \geq 2t_i + l_i] &= \Pr[2|S| + X \geq 2t_i + l_i] \\
&= \Pr[X \geq 2t_i + l_i - 2|S|] \\
&= \Pr[\sum_{j \in \urcorner S^*} X_j \geq 2t_i + l_i - 2|S|] \\
&\leq \sum_{j=2t_i+l_i-2|S|}^{|\urcorner S^*|} \binom{|\urcorner S^*|}{i} \cdot (\frac{2}{|\mathbb{F}|-1})^i \cdot (1 - \frac{2}{|\mathbb{F}|-1})^{|\urcorner S^*|-i} \\
&\quad \text{(By the binomial theorem, } \binom{|\urcorner S^*|}{i} \leq 2^{|\urcorner S^*|}\text{)} \\
&\leq |\urcorner S^*| \cdot 2^{|\urcorner S^*|} (\frac{2}{|\mathbb{F}|-1})^{2t_i+l_i-2|S|} \\
&\leq |\urcorner S| \cdot 2^{|\urcorner S|} \cdot (\frac{2}{|\mathbb{F}|-1})^{2t_i+l_i-2|S|} \quad (\urcorner S^* \subseteq \urcorner S) \\
&= |[1, n_i] \backslash S| \cdot 2^{|[1,n_i]\backslash S|} \cdot (\frac{2}{|\mathbb{F}|-1})^{2t_i+l_i-2|S|} \\
&= (n_i - |S|) \cdot 2^{n_i-|S|} \cdot (\frac{2}{|\mathbb{F}|-1})^{2t_i+l_i-2|S|} \\
&\quad \text{(Assume } |\mathbb{F}| \geq 2^{10}\text{, then } \frac{2}{|\mathbb{F}|-1} \leq \frac{2.002}{|\mathbb{F}|}\text{)} \\
&\leq n_i \cdot 2^{n_i-|S|} \left(\frac{2.002}{|\mathbb{F}|}\right)^{2t_i+l_i-2|S|}
\end{aligned}
$$

We observe that for an index set $S \subseteq [1, n_i]$, if any set S is selected, each index $i \in [1, n_i]$ has two possibilities: to include it in S or not. Therefore, there are a total of 2^{n_i} possible selections for the set S. When we enumerate all possible sets S, the union of the resulting $m_{i+1}(S)$, denoted by $\cup_{S \subseteq [1,n_i]} m_{i+1}(S)$, can cover all messages in $\mathbb{F}^{k_{i+1}} = \mathbb{F}^{2k_i}$. Lemma 2 in paper [ZCF23] tells us that the size of the set $m_{i+1}(S)$ is at most $\mathbb{F}^{t_i-|S|}$. Thus, by iterating through all 2^{n_i} possible sets S, each set S contains at most $\mathbb{F}^{t_i-|S|}$ messages $\vec{m} \in \mathbb{F}^{2k_i}$. By combining all S and considering the bounds on the size of each S, we can conclude that when l_i is sufficiently large, that is, when $|\mathbb{F}|^{l_i} \gg 2^{n_i}$, the expression $n_i \cdot 2^{n_i-|S|} \left(\frac{2.002}{|\mathbb{F}|}\right)^{2t_i+l_i-2|S|}$ becomes sufficiently small. In this case, for any non-zero vector $\vec{m} \in \mathbb{F}^{2k_i}$, we have $\mathrm{nzero}(\mathrm{Enc}_{i+1}(\vec{m})) \leq 2t_i + l_i$, that is, $\mathrm{Enc}_{i+1}(\vec{m})$ contains at most $2t_i + l_i$ zero components.

The BaseFold paper [ZCF23] presents a more specific statement in the form of a theorem.

Theorem 1 [ZCF23, Theorem 2] Fix any finite field $\mathbb{F}$ with $|\mathbb{F}| \geq 2^{10}$, and let $\lambda \in \mathbb{N}$ be the security parameter. For a vector $\mathbf{v}$ with components in $\mathbb{F}$, let $\mathrm{nzero}(\mathbf{v})$ denote the number of zero components in $\mathbf{v}$. For any $d \in \mathbb{N}$, let D_d be a (c, k_0)-foldable distribution, and for each $i \leq d$, set $k_i = k_0 2^i$, $n_i = ck_i$. Then,

$$
\Pr_{(\mathbf{G}_0, \ldots, \mathbf{G}_d) \leftarrow D_d} [\exists \mathbf{m} \in \mathbb{F}^{k_d} \backslash \{\mathbf{0}\}, \mathrm{nzero}(\mathrm{Enc}_d(\mathbf{m})) \geq t_d] \leq d \cdot 2^{-\lambda} \tag{5}
$$

where $t_0 = k_0$ and for each $i \in [d]$, $t_i = 2t_{i-1} + l_i$, with

$$
l_i := \frac{2(d-1)\log n_0 + \lambda + 2.002 t_{d-1} + 0.6 n_d}{\log |\mathbb{F}| - 1.001}.
$$

Equation (5) indicates that the number of zero components in $\mathrm{Enc}_d(\mathbf{m})$ is bounded by t_d with negligible probability if exceeded. Given the iterative formula $t_i = 2t_{i-1} + l_i$, we can compute t_d through iterative summation. Consequently, the maximum relative number of zero components in C_d is $Z_{C_d} = \frac{t_d}{n_d}$, and calculating $1 - Z_{C_d}$ yields the minimum relative Hamming distance Δ_{C_d} of C_d, resulting in equation (1):

$$1 - \left(\frac{\epsilon_{\mathbb{F}}^d}{c} + \frac{\epsilon_{\mathbb{F}}}{\log |\mathbb{F}|} \sum_{i=0}^{d} (\epsilon_{\mathbb{F}})^{d-i} \left(0.6 + \frac{2 \log(n_i/2) + \lambda}{n_i} \right) \right).$$

From the iterative formula $t_i = 2t_{i-1} + l_i$, we observe that as i increases, t_i grows by more than $2t_{i-1}$. Since the encoding length doubles each iteration, the relative maximum number of zero components increases, and therefore the minimum relative Hamming distance decreases. If Δ_{C_d} is sufficiently large, then through this iterative method, we obtain with overwhelming probability that $\Delta_{C_0} \geq \Delta_{C_1} \geq ... \geq \Delta_{C_d}$. From $i = d$ to $i = 0$, this encoding method does not decrease Δ_{C_d}. In the IOPP protocol, if the initial minimum relative Hamming distance is large, then Δ_{C_0} remains large with overwhelming probability, which plays a significant role in analyzing the soundness of the IOPP.

5.4 References

- [ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. "BaseFold: efficient field-agnostic polynomial commitment schemes from foldable codes." Annual International Cryptology Conference. Cham: Springer Nature Switzerland, 2024.
- [GJX15] Venkatesan Guruswami, Lingfei Jin, and Chaoping Xing. "Efficiently List-Decodable Punctured Reed-Muller Codes". In: IEEE Transactions on Information Theory 63 (2015), pp. 4317–4324. url: https://api.semanticscholar.org/CorpusID: 14176561.

Chapter 6

Notes on Basefold (Part V): IOPP Soundness

- Jade Xie jade@secbit.io
- Yu Guo yu.guo@secbit.io

In this article, we will outline the proof approach for IOPP soundness presented in the [ZCF23] paper, which is similar to the soundness proof for the FRI protocol in [BKS18]. It employs a binary tree method to analyze points where the Prover might cheat, a concept also appearing in the soundness proof of the DEEP-FRI protocol in [BGKS20].

6.1 IOPP Protocol

The IOPP protocol has been thoroughly introduced in the second article above. For the analysis in the later sections, we will briefly outline the IOPP protocol here. It is an extension of the FRI protocol, and the process of understanding the protocol can fully leverage the understanding of the FRI protocol, as both the commit and query phases are consistent.

Protocol 1 [ZCF23, Protocol 2] IOPP.commit

Input oracle: $\pi_d \in \mathbb{F}^{n_d}$
Output oracles: $(\pi_{d-1}, \dots, \pi_0) \in \mathbb{F}^{n_{d-1}} \times \cdots \times \mathbb{F}^{n_0}$

- For i from $d-1$ down to 0:
  1. Verifier samples and sends $\alpha_i \leftarrow \$\mathbb{F}$ from $\mathbb{F}$ to Prover
  2. For each index $j \in [1, n_i]$, Prover:
     a. Sets $f(X) := \text{interpolate}((\text{diag}(T_i)[j], \pi_{i+1}[j]), (\text{diag}(T'_i)[j], \pi_{i+1}[j+n_i]))$
     b. Sets $\pi_i[j] = f(\alpha_i)$
  3. Prover outputs oracle $\pi_i \in \mathbb{F}^{n_i}$.

Protocol 2 [ZCF23, Protocol 3] IOPP.query

Input oracles: $(\pi_{d-1}, \dots, \pi_0) \in \mathbb{F}^{n_{d-1}} \times \dots \times \mathbb{F}^{n_0}$
Output: accept or reject

- Verifier samples $\mu \leftarrow \$[1, n_{d-1}]$

- For i from $d-1$ down to 0, Verifier:
  1. Queries oracle $\pi_{i+1}[\mu], \pi_{i+1}[\mu + n_i]$
  2. Computes $p(X) := \text{interpolate}((\text{diag}(T_i)[\mu], \pi_{i+1}[\mu]), (\text{diag}(T'_i)[\mu], \pi_{i+1}[\mu + n_i]))$
  3. Checks $p(\alpha_i) = \pi_i[\mu]$

4. If $i > 0$ and $\mu > n_i - 1$, then updates $\mu \leftarrow \mu - n_{i-1}$

- If π_0 is a valid codeword with respect to the generator matrix $\mathbf{G}_0$, output `accept`; otherwise, output `reject`.

6.2 Analysis Approach

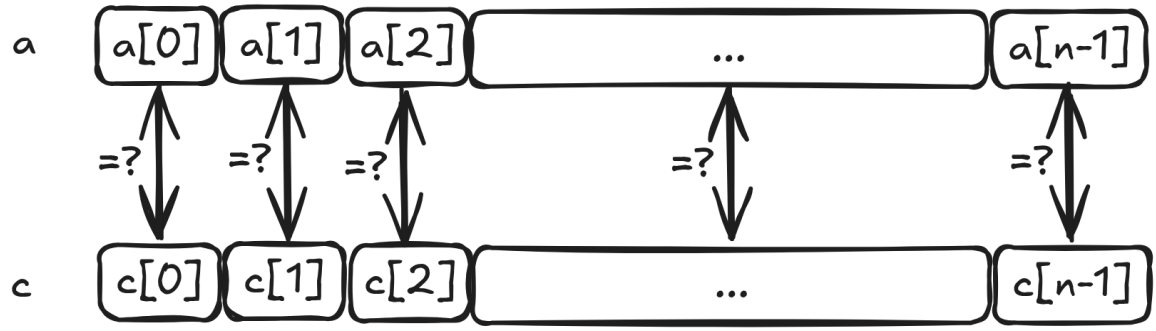
The analysis of IOPP soundness examines, for any Prover who might cheat, what is the maximum probability that the Verifier outputs `accept` in such a scenario. We aim for this probability to be sufficiently small to ensure the protocol's security. Naturally, this probability depends on certain parameters of the protocol, and in practice, we desire it to be below a predetermined security parameter λ (for example, λ could be set to 128 or 256), meaning that this probability should be less than $2^{-\lambda}$.

Let us now examine areas within the IOPP protocol where a cheating Prover might exploit to cause the Verifier to output `accept`. We note that there are two points where the Verifier introduces randomness: 1. In the IOPP.commit phase, step 1 of the protocol, the Verifier selects a random number α_i from $\mathbb{F}$ and sends it to the Prover, who uses it to fold the original π_{i+1} to obtain π_i. 2. In the IOPP.query phase, step 1 of the protocol, the Verifier samples $\mu \leftarrow \$[1, n_{d-1}]$ and then checks whether the Prover's folding was correct.

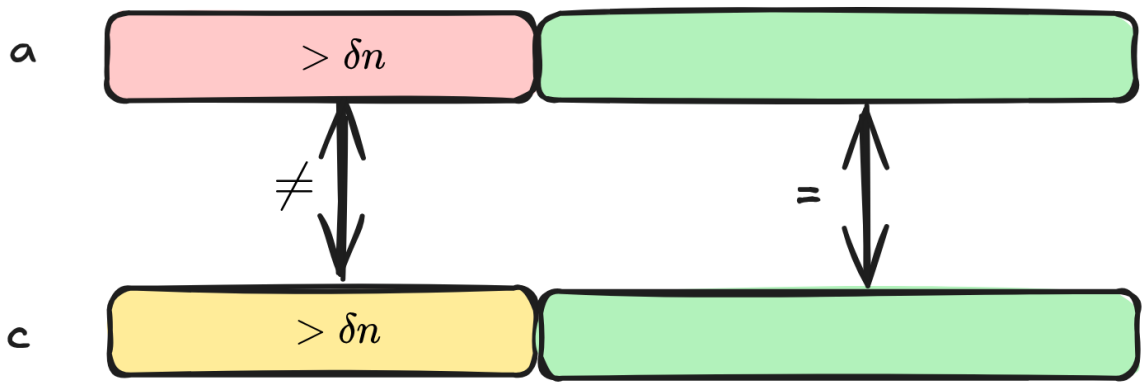
Suppose the initial cheating Prover provides a π_d that is δ away from C_d. We aim for the Verifier to ultimately check that π_0 is also at least δ away from C_0, meaning that the δ distance is preserved throughout the folding process. Another scenario is detecting that the Prover did not fold correctly. We consider two cases:

1. The Prover is extremely lucky, and the random number α_i chosen by the Verifier causes the folded π_i to be less than δ away from the corresponding C_i, leading the Verifier to output `accept`. We consider this situation very lucky for the Prover because, according to the Proximity Gaps theorem, the probability of such an event is exceedingly small (assuming this probability is ϵ), such that its occurrence is akin to the Prover winning the lottery.

2. The Prover is not as lucky as in Case 1. In this scenario, after folding with the random number, the message π_i still remains at least δ away from the corresponding C_i. Since the Verifier randomly selects $\mu \leftarrow \$[1, n_{d-1}]$ in the IOPP.query phase and only checks a subset of the Prover's foldings, this provides an opportunity for the Prover to potentially evade Verifier checks. For example, if after folding, the message a satisfies $\Delta(a, C) > \delta$ in relative Hamming distance, i.e., $\Delta(a, C) > \delta$. The Verifier will randomly check $a[i]$ against $c[i]$, and if they are unequal, the Verifier will reject.



Since $\Delta(a, C) > \delta$, more than a δ proportion of the components in a differ from the codewords in the encoding space. When the Verifier selects one of these differing positions, it will reject, hence the probability that the Verifier catches the Prover cheating exceeds δ.



If the Verifier queries l times, the probability that the Prover can pass all Verifier checks is at most $(1-\delta)^l$.

Combining the two cases, the probability that the cheating Prover succeeds is bounded above by

$$\epsilon + (1-\delta)^l \tag{1}$$

This represents an overall analysis approach. The specific expression may vary, but the following IOPP soundness theorem will elaborate further.

6.3 IOPP Soundness Theorem

Theorem 1 [ZCF23, Theorem 3] (IOPP Soundness for Foldable Linear Codes) Let C_d be a (c, k_0, d) foldable linear code with generator matrices $(G_0, \ldots, G_d)$. Let C_i $(0 \leq i < d)$ denote the code generated by G_i, and assume that for all $i \in [0, d-1]$, the relative minimum distance $\Delta C_i \geq \Delta C_{i+1}$. Let $\gamma > 0$, and set $\delta := \min(\Delta^*(\pi_d, C_d), J_\gamma(J_\gamma(\Delta_{C_d})))$, where $\Delta^*(\pi_d, C_d)$ is the relative coset minimum distance between $\mathbf{v}$ and C_d. Then, for any (adaptively chosen) Prover oracles $\pi_{d-1}, \ldots, \pi_0$, the Verifier in the IOPP.query phase repeated ℓ times outputs `accept` with probability at most $(1 - \delta + \gamma d)^\ell$.

The term $J_\gamma(J_\gamma(\Delta_{C_d}))$ mentioned in the theorem refers to the composition of two Johnson functions, defined as follows.

Definition 1 [ZCF23, Definition 4] (Johnson Bound) For any $\gamma \in (0, 1]$, define $J_\gamma : [0, 1] \to [0, 1]$ as

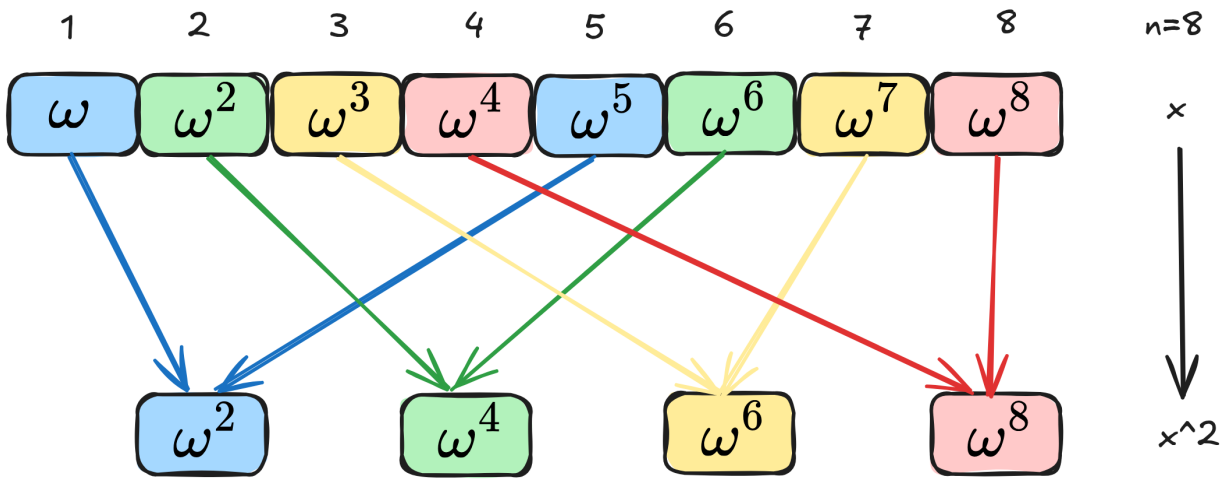
$$J_\gamma(\lambda) := 1 - \sqrt{1 - \lambda(1-\gamma)}.$$

The definition of relative coset minimum distance is as follows.

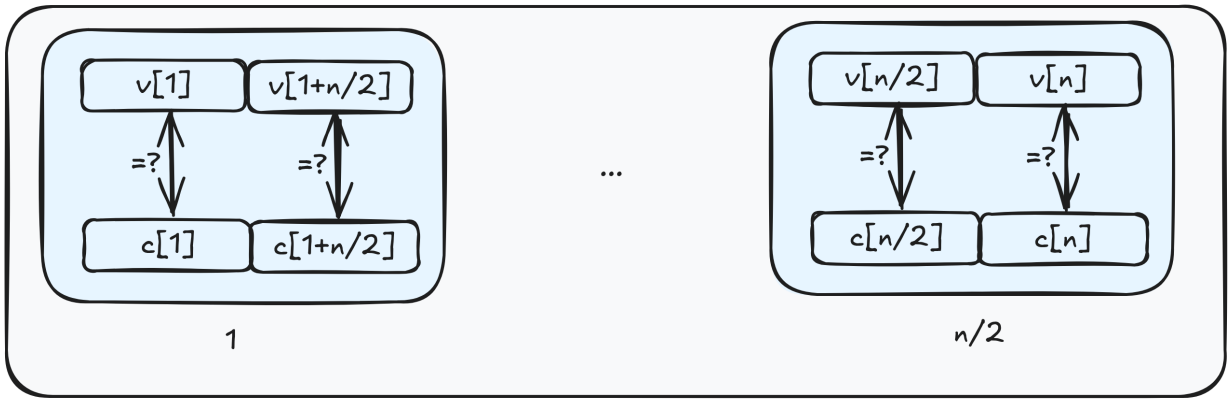
Definition 2 [ZCF23, Definition 5] (Relative Coset Minimum Distance) Let n be an even number, and let C be a $[n, k, d]$ error-correcting code. For a vector $\mathbf{v} \in \mathbb{F}^n$ and a codeword $c \in C$, the relative distance $\Delta^*(\mathbf{v}, c)$ between $\mathbf{v}$ and c is defined as

$$\Delta^*(\mathbf{v}, c) = \frac{2|\{j \in [1, n/2] : \mathbf{v}[j] \neq c[j] \vee \mathbf{v}[j + n/2] \neq c[j + n/2]\}|}{n}.$$

This definition is similar to the block-wise distance definition used in [BBHR18] to prove soundness ([BBHR18, Definition 3.2]). It is an alternative version of the relative minimum Hamming distance. Pairs $\{j, j + n/2\}$ are considered as a pair, corresponding to a coset, analogous to the FRI protocol. For example, for $n = 8$, let the generator be ω with $\omega^8 = 1$, and select the mapping $x \mapsto x^2$, then it can be seen that $\{1, 5\}$, $\{2, 6\}$, $\{3, 7\}$, $\{4, 8\}$ correspond to elements that form a coset, resulting in a total of 4 cosets.



$\Delta^*(\mathbf{v}, c)$ measures the proportion of cosets in which $\mathbf{v}$ and c are not entirely consistent.



Let $\Delta^*(\mathbf{v}, C) := \min_{c \in C} \Delta^*(\mathbf{v}, c)$. Then, it relates to the relative minimum Hamming distance as follows: $\Delta(\mathbf{v}, C) \leq \Delta^*(\mathbf{v}, C)$.

Despite introducing these different definitions and Johnson functions, the proof approach for IOPP soundness remains consistent with the earlier outlined analysis, discussing two cases. Our aim is to analyze the probability that a cheating Prover can pass all Verifier checks and ultimately output `accept`. The proof approach is as follows:

Case 1: The Prover is extremely lucky. Due to the Verifier selecting random numbers α_i, the folded messages are sufficiently close to the encoding space, allowing the Prover to pass all subsequent Verifier checks. For the Verifier, this corresponds to some "bad" events occurring, where there exists an $i \in [0, d-1]$ such that

$$\Delta(\text{fold}_{\alpha_i}(\pi_{i+1}), C_i) \leq \min(\Delta^*(\pi_{i+1}, C_{i+1}), J_\gamma(J_\gamma(\Delta_{C_d}))) - \gamma$$

Using proof by contradiction through the Correlated Agreement theorem (which can derive the corresponding Proximity Gaps theorem), it can be shown that the probability of such "bad" events is small, proven to be at most $\frac{2d}{\gamma^3 |\mathbb{F}|}$.

Case 2: Suppose the Prover is not as lucky, meaning that the "bad" events described in Case 1 do not occur. Then, in the IOPP.query phase, the Verifier selects $\mu \leftarrow \$[1, n_{d-1}]$, and in this scenario, the Prover might evade the Verifier's checks by having the Verifier select points where the Prover has not cheated. Repeating the IOPP.query phase l times, the probability that the Prover manages to pass each check is at most $(1 - \delta + \gamma d)^l$.

Combining Cases 1 and 2, for foldable linear codes, the IOPP Soundness is at least

$$\mathbf{s}^{-}(\delta) = 1 - \left(\frac{2d}{\gamma^3 |\mathbb{F}|} + (1 - \delta + \gamma d)^l \right).$$

Thus, Theorem 1 is proven.

6.3.1 Proof of Case 1

The following Corollary 1 demonstrates that for any specific i, the probability that after folding, the result is within a relative Hamming distance $\delta - \gamma$ of C_i, denoted as event $B^{(i)}$, is at most $\frac{2}{\gamma^3 |\mathbb{F}|}$. Therefore, if certain events B_i occur, their probability does not exceed the sum of the probabilities of these B_i events, i.e.,

$$\Pr\left[\bigcup_{i=0}^{d-1} B^{(i)} \right] \leq \sum_{i=0}^{d-1} \Pr[B^{(i)}] \leq \frac{2d}{\gamma^3 |\mathbb{F}|}.$$

Let us examine Corollary 1 in detail.

Corollary 1 [ZCF23, Corollary 1] For any fixed $i \in [0, d-1]$ and $\gamma, \delta > 0$, such that $\delta \leq J_\gamma(J_\gamma(\Delta_{C_d}))$, if $\Delta^*(\mathbf{v}, C_{i+1}) > \delta$, then

$$\Pr_{\alpha_i \leftarrow \$\mathbb{F}}[\Delta(\text{fold}_{\alpha_i}(\mathbf{v}), C_i) \leq \delta - \gamma] \leq \frac{2}{\gamma^3 |\mathbb{F}|}. \tag{2}$$

The function $\text{fold}_{\alpha_i}(\cdot)$ is defined as follows. Let $\mathbf{u}, \mathbf{u}' \in \mathbf{F}^{n_i}$ be the unique interpolated vectors such that

$$\pi_{i+1} = (\mathbf{u} + \text{diag}(T_i) \circ \mathbf{u}', \mathbf{u} + \text{diag}(T_i') \circ \mathbf{u}')$$

Then, $\text{fold}_{\alpha_i}(\pi_{i+1})$ is defined as

$$\text{fold}_{\alpha_i}(\pi_{i+1}) := \mathbf{u}' + \alpha_i \mathbf{u}.$$

This essentially represents the process of folding π_{i+1} with the random number α_i.

Corollary 1 generalizes [BKS18] Corollary 7.3 to general foldable linear codes.

Proof Idea of Corollary 1: To prove that the relative Hamming distance after folding with the random number α_i is smaller than the original distance is a rare event, specifically not exceeding $\frac{2}{\gamma^3 |\mathbb{F}|}$. Suppose, for contradiction, that this event occurs with a significantly higher probability. Then, by directly applying the Correlated Agreement theorem (from [BKS18] Theorem 4.4), it can be shown that for the affine space $U = \{\mathbf{u} + x\mathbf{u}' : x \in \mathbb{F}\}$, there exists a sufficiently large Correlated Agree subset T within C_i such that there exist $\mathbf{w}, \mathbf{w}' \in C_i$ agreeing with $\mathbf{u}$ and $\mathbf{u}'$ on T, respectively. Encoding $\mathbf{w}, \mathbf{w}'$ yields a codeword c_w in C_{i+1}, thereby estimating $\Delta^*(\mathbf{v}, C_{i+1}) \leq \delta$, which contradicts our assumption. Therefore, the corollary holds.

6.3.2 Proof of Case 2

To prove that repeating the IOPP.query phase l times results in the Verifier outputting `accept` with probability at most $(1 - \delta + \gamma d)^l$, we merely need to demonstrate that, in one execution of IOPP.query, the probability that the Verifier outputs `reject` is at least $\delta - \gamma d$.

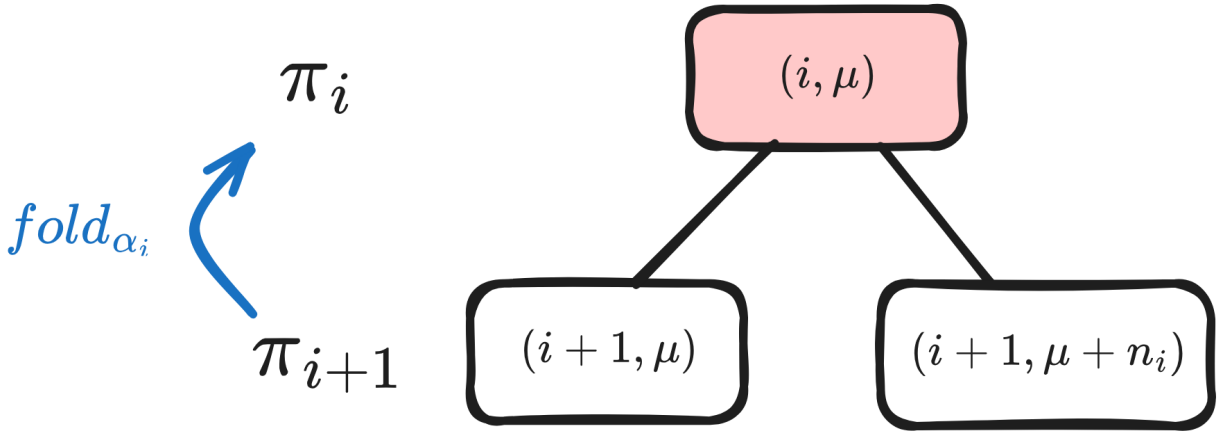
Using the binary tree concept for the proof, we first define a "bad" node (i, μ), as shown in the figure below. These are the points where the Prover fails to pass step 3 of the IOPP.query protocol, meaning that after the Verifier selects a random number μ, for any $i \in [0, d-1]$ and $\mu \in [n_i]$, the Verifier computes in step 2 of IOPP.query:

$$p(X) := \text{interpolate}((\text{diag}(T_i)[\mu], \pi_{i+1}[\mu]), (\text{diag}(T_i')[\mu], \pi_{i+1}[\mu + n_i]))$$

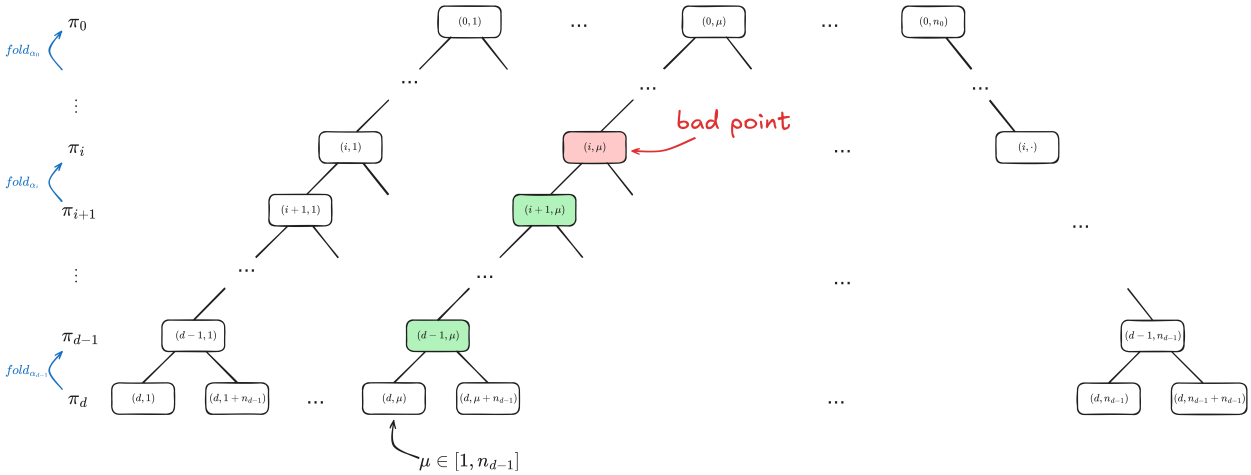
Subsequently, in step 3 of the IOPP.query protocol, the Verifier checks whether

$$p(\alpha_i) \neq \pi_i[\mu]$$

At this point, we say that the node (i, μ) is "bad".



Next, consider i from $d-1$ down to 0. For any $\mu \in [1, n_{d-1}]$, μ can generate a binary tree, forming n_0 such binary trees as depicted below.



If there exists at least one "bad" node (i, μ) within any of these trees—assuming that all nodes from layers $d-1$ down to $i+1$ and their children are "good", i.e., they pass step 3 of the IOPP.query protocol—then when a "bad" node occurs at level i, the Verifier will reject. As shown in the figure, nodes from levels $i+1$ to $d-1$ are all "good". This implies that as long as there is at least one bad node in the entire tree, the Verifier will reject. If we let β_i denote the ratio of "bad" nodes at layer i, then the probability that the Verifier rejects at layer i is β_i. Considering the entire IOPP.query phase, the Verifier's probability of rejection is thus $\sum_{i=0}^{d-1} \beta_i$, where $\beta_i := \Delta(\pi_i, \text{fold}_{\alpha_i}(\pi_{i+1}))$, representing those "bad" points where the folded π_{i+1} does not agree with π_i.

Thus, the remaining task is to estimate $\sum_{i=0}^{d-1} \beta_i$. [ZCF23, Claim 2] provides inequalities for each β_i.

Claim 1 [ZCF23, Claim 2] For any $i \in [0, d]$, define $\delta^{(i)} := \min(\Delta^*(\pi_i, C_i), J_\gamma(J_\gamma(\Delta_{C_d})))$. For all $i \in [0, d-1]$,

$$\beta_i \geq \delta^{(i+1)} - \delta^{(i)} - \gamma.$$

Under the soundness conditions, $\delta = \delta^{(d)}$. Also, since $\Delta^*(\pi_0, C_0) = \Delta(\pi_0, C_0) = 0$, we have $\delta^{(0)} = 0$. Thus, according to the claim:

$$\delta = \delta^{(d)} - \delta^{(0)} = \sum_{i=0}^{d-1} \delta^{(i+1)} - \delta^{(i)} \leq \sum_{i=0}^{d-1} \beta_i + \gamma d,$$

hence,

$$\sum_{i=0}^{d-1} \beta_i \geq \delta - \gamma d.$$

Therefore, if no bad event B occurs, executing the IOPP.query phase once, the probability that the Verifier rejects is at least $\delta - \gamma d$. This concludes the proof of Case 2.

6.4 References

- [BBHR18] Eli Ben-Sasson, Iddo Bentov, Ynon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In Proceedings of the 45th International Colloquium on Automata, Languages, and Programming (ICALP), 2018. Available online as Report 134-17 on Electronic Colloquium on Computational Complexity.
- [BGKS20] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: sampling outside the box improves soundness. In Thomas Vidick, editor, 11th Innovations in Theoretical Computer Science Conference, ITCS 2020, January 12-14, 2020, Seattle, Washington, USA, volume 151 of LIPIcs, pages 5:1–5:32. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.
- [BKS18] Eli Ben-Sasson, Swastik Kopparty, and Shubhangi Saraf. "Worst-Case to Average Case Reductions for the Distance to a Code". In: Proceedings of the 33rd Computational Complexity Conference. CCC '18. San Diego, California: Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2018. ISBN: 9783959770699.
- [ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. "BaseFold: efficient field-agnostic polynomial commitment schemes from foldable codes." Annual International Cryptology Conference. Cham: Springer Nature Switzerland, 2024.

Chapter 7

Basefold Optimization

Yu Guo yu.guo@secbit.io

Last updated: 2025-06-10

7.1 Protocol Review

For any multilinear polynomial $\tilde{f}(X_0, X_1, \dots, X_{n-1}) \in F^{\leq 1}[X_0, X_1, \dots, X_{n-1}]$ with n variables (indeterminates), if we want to prove that its evaluation at an arbitrary point $\vec{u} = (u_0, u_1, \dots, u_{n-1})$ is correct, the Basefold protocol [ZCF23] provides an elegant solution. Its core idea is to use the Sumcheck protocol to prove $\tilde{f}(u_0, u_1, \dots, u_{n-1})$, and then in the last step of the Sumcheck protocol, the Verifier needs to verify the evaluation of $\tilde{f}(r_0, r_1, \dots, r_{n-1})$. Basefold does not rely on another MLE PCS to complete this final step of proof, but instead uses a FRI protocol to assist. The original purpose of the FRI protocol is to prove the proximity of a codeword, i.e., that its distance from the correct codeword does not exceed a security parameter δ. However, in the last step of the FRI protocol, the Prover sends the message corresponding to the folded codeword. If folded sufficiently, the final message is a constant (polynomial). If the Prover is honest, this value happens to be equal to the value of $\tilde{f}(r_0, r_1, \dots, r_{n-1})$, which perfectly complements the Sumcheck situation.

Let's go through the protocol details. First, $\tilde{f}(\vec{X})$ can be rewritten as the following equation:

$$\tilde{f}(\vec{X}) = \sum_{\vec{b} \in \{0,1\}^n} \tilde{f}(\vec{b}) \cdot eq(\vec{X}, \vec{b})$$

We can see that the right side of the equation is a sum, which meets the requirements of the Sumcheck protocol. The Sumcheck protocol can prove the following inner product equation:

$$v = \sum_{\vec{b} \in \{0,1\}^n} \tilde{f}(\vec{b}) \cdot \tilde{g}(\vec{b})$$

7.1.1 Inner Product Proof Based on Sumcheck Protocol

Let's review how to use the Sumcheck protocol to prove inner products. We assume that two vectors $\vec{f}$ and $\vec{g}$ of length 2^n correspond to two n-variable multilinear polynomials, $\tilde{f}$ and $\tilde{g}$. Next, we explain how the Prover and Verifier prove the following goal:

$$s = \sum_{\vec{b} \in \{0,1\}^n} \tilde{f}(b_0, b_1, \dots, b_{n-1}) \cdot \tilde{g}(b_0, b_1, \dots, b_{n-1})$$

Round 1: The Prover constructs a degree-2 univariate polynomial $h^{(0)}(X)$ and sends its evaluations at $X = 0, 1, 2$, i.e., $(h^{(0)}(0), h^{(0)}(1), h^{(0)}(2))$.

$$h^{(0)}(X) = \sum_{\vec{b} \in \{0,1\}^{n-1}} \tilde{f}(X, b_1, b_2, \dots, b_{n-1}) \cdot \tilde{g}(X, b_1, b_2, \dots, b_{n-1})$$

The Verifier checks if $h^{(0)}(0) + h^{(0)}(1) \stackrel{?}{=} v$. If true, it responds with a random challenge $r_0 \in \mathbb{F}_q$, and the proof goal is transformed into a new goal:

$$h^{(0)}(r_0) = \sum_{\vec{b} \in \{0,1\}^{n-1}} \tilde{f}(r_0, b_1, b_2, \dots, b_{n-1}) \cdot \tilde{g}(r_0, b_1, b_2, \dots, b_{n-1})$$

Note that $\tilde{f}(r_0, X_1, \dots, X_{n-1})$ and $\tilde{g}(r_0, X_1, \dots, X_{n-1})$ are still multilinear polynomials. We denote them as $\tilde{f}^{(1)}(X_1, \dots, X_{n-1})$ and $\tilde{g}^{(1)}(X_1, \dots, X_{n-1})$:

$$\tilde{f}^{(1)}(X_1, \dots, X_{n-1}) = \tilde{f}(r_0, X_1, \dots, X_{n-1})$$
$$\tilde{g}^{(1)}(X_1, \dots, X_{n-1}) = \tilde{g}(r_0, X_1, \dots, X_{n-1})$$

So the new proof goal can be written as:

$$h^{(0)}(r_0) = v^{(1)} \stackrel{?}{=} \sum_{\vec{b} \in \{0,1\}^{n-1}} \tilde{f}^{(1)}(b_1, \dots, b_{n-1}) \cdot \tilde{g}^{(1)}(b_1, \dots, b_{n-1})$$

This way, Sumcheck can be seen as a recursive protocol, with each call halving the length of the sum. The Prover and Verifier repeat this process until the proof length of Sumcheck becomes 1.

In round n, the Prover constructs $h^{(n-1)}(X)$ and sends $(h^{(n-1)}(0), h^{(n-1)}(1), h^{(n-1)}(2))$:

$$h^{(n-1)}(X) = \tilde{f}^{(n-1)}(X) \cdot \tilde{g}^{(n-1)}(X)$$

The Verifier checks if $h^{(n-1)}(0) + h^{(n-1)}(1) \stackrel{?}{=} v^{(n-1)}$. If true, it responds with a random challenge $r_{n-1} \in \mathbb{F}_q$, and the proof goal is transformed into a new goal:

$$h^{(n-1)}(r_{n-1}) \stackrel{?}{=} \tilde{f}^{(n-1)}(r_{n-1}) \cdot \tilde{g}^{(n-1)}(r_{n-1})$$

Then the Prover doesn't need to send any more messages. The Verifier directly calculates the value of $h^{(n-1)}(r_{n-1})$, then calls the oracles of $\tilde{f}(X_0, X_1, \dots, X_{n-1})$ and $\tilde{g}(X_0, X_1, \dots, X_{n-1})$ to get the values of $\tilde{f}^{(n-1)}(r_{n-1})$ and $\tilde{g}^{(n-1)}(r_{n-1})$, and then verifies if the equation holds:

$$f^{(n-1)}(r_{n-1}) = \tilde{f}(r_0, r_1, \dots, r_{n-1})$$
$$g^{(n-1)}(r_{n-1}) = \tilde{g}(r_0, r_1, \dots, r_{n-1})$$

Typically, in the last step of Sumcheck, the oracles of $\tilde{f}$ and $\tilde{g}$ that the Verifier relies on are implemented using Polynomial Commitment. That is, before Sumcheck begins, the Prover first sends their commitments, and then in the last step of Sumcheck, the Prover sends the values of $\tilde{f}(r_0, r_1, \dots, r_{n-1})$ and $\tilde{g}(r_0, r_1, \dots, r_{n-1})$, along with the corresponding proofs of Multilinear Polynomial Evaluation.

7.1.2 Proximity Proof Based on FRI Protocol

TODO

7.1.3 Basefold Proof Process Based on Multilinear Basis

Let's go through the Prover's proof process from an implementation perspective. To help readers understand, we assume $\tilde{f}$ is a three-variable multilinear polynomial, and the protocol has a total of $n = 3$ rounds. The Prover first does one round of Sumcheck, then reuses the random numbers from Sumcheck to do one round of FRI protocol, which is equivalent to completing one round of the Basefold protocol; then continues until the proof length of Sumcheck becomes 1.

$$\begin{aligned} v &= \sum_{(b_0,b_1,b_2)\in\{0,1\}^3} \tilde{f}(b_0,b_1,b_2)\cdot eq((u_0,u_1,u_2),(b_0,b_1,b_2)) \\ &= \sum_{i=0}^{7} a_i \cdot w_i \end{aligned}$$

Here $v = \tilde{f}(u_0, u_1, u_2)$, vector $\vec{a} = (a_0, a_1, \cdots, a_7)$ is the evaluation of $\tilde{f}(\vec{X})$ on the Boolean Hypercube, and $\vec{w} = (w_0, w_1, \cdots, w_7)$ is the evaluation of $eq(\vec{u}, \vec{X})$ on the Boolean Hypercube:

$$\begin{aligned} w_0 &= (1-u_0)(1-u_1)(1-u_2) \\ w_1 &= u_0(1-u_1)(1-u_2) \\ w_2 &= (1-u_0)u_1(1-u_2) \\ w_3 &= u_0u_1(1-u_2) \\ w_4 &= (1-u_0)(1-u_1)u_2 \\ w_5 &= u_0(1-u_1)u_2 \\ w_6 &= (1-u_0)u_1u_2 \\ w_7 &= u_0u_1u_2 \end{aligned}$$

The Prover needs to prove the correctness of this inner product calculation. That is:

$$v = a_0w_0 + a_1w_1 + a_2w_2 + a_3w_3 + a_4w_4 + a_5w_5 + a_6w_6 + a_7w_7$$

The Prover maintains two arrays of length 2^n, storing $\vec{a}$ and $\vec{w}$ respectively, corresponding to the Evaluations representations of $\tilde{f}$ and eq.

$$\begin{aligned} a_i &= \tilde{f}(i_0, i_1, i_2) \\ w_i &= eq((u_0,u_1,u_2),(i_0,i_1,i_2)) \end{aligned}$$

In the first round of the Sumcheck protocol, the Prover calculates $h^{(0)}(X)$. This is a degree-2 univariate polynomial. We only need to get its evaluations at at least three different points to uniquely represent this polynomial. To optimize computation, we specifically choose the points $X = 0, 1, 2$.

$$h^{(0)}(X) = \sum_{b_1,b_2\in\{0,1\}^2} \tilde{f}(X,b_1,b_2)\cdot\tilde{g}(X,b_1,b_2)$$

And $h^{(0)}(0)$ happens to be equal to the sum of the terms at even positions in the inner product sum, while $h^{(0)}(1)$ happens to be equal to the sum of the terms at odd positions in the inner product sum, represented as follows:

$$\begin{aligned} h^{(0)}(0) &= a_0w_0 + a_2w_2 + a_4w_4 + a_6w_6 \\ h^{(0)}(1) &= a_1w_1 + a_3w_3 + a_5w_5 + a_7w_7 \end{aligned}$$

Therefore, the Prover only needs two passes of 2^{n-1} multiplications and 2^{n-1} additions (a total of 2^n multiplications and 2^n additions) to calculate $h^{(0)}(0)$ and $h^{(0)}(1)$. But how to calculate $h^{(0)}(2)$? Let's first prove the following equation, which we will use repeatedly:

$$\tilde{f}(X_0+1, X_1, X_2) = \tilde{f}(X_0, X_1, X_2) + \tilde{f}(1, X_1, X_2) - \tilde{f}(0, X_1, X_2)$$

Generalizing this, we can get the following expression:

$$\tilde{f}(\vec{Y}, X_k+1, \vec{Z}) = \tilde{f}(\vec{Y}, X_k, \vec{Z}) + \tilde{f}(\vec{Y}, 1, \vec{Z}) - \tilde{f}(\vec{Y}, 0, \vec{Z})$$

Then, according to the properties of Multilinear Polynomials, we can calculate the value of $h^{(0)}(2)$:

$$\begin{aligned} h^{(0)}(2) &= \sum_{b_1,b_2\in\{0,1\}^2} \tilde{f}(2,b_1,b_2)\cdot\tilde{w}(2,b_1,b_2) \\ &= \sum_{b_1,b_2\in\{0,1\}^2} \Big(2\cdot\tilde{f}(1,b_1,b_2) - \tilde{f}(0,b_1,b_2)\Big)\cdot\Big(2\cdot\tilde{w}(1,b_1,b_2) - \tilde{w}(0,b_1,b_2)\Big) \end{aligned}$$

This requires the Prover to perform 2^{n-1} multiplications and $5\cdot 2^{n-1}$ additions. Then the Prover sends $h^{(0)}(0), h^{(0)}(1), h^{(0)}(2)$ to the Verifier, and the Verifier checks if the following equation holds:

$$h^{(0)}(0) + h^{(0)}(1) \stackrel{?}{=} v$$

If it holds, the Verifier replies with a random number $r_0 \in \mathbb{F}_q$, and the proof goal is reduced to a new proof goal:

$$\begin{aligned} h^{(0)}(r_0) &= \sum_{b_1,b_2\in\{0,1\}^2} \tilde{f}(r_0,b_1,b_2)\cdot\tilde{w}(r_0,b_1,b_2) \\ &= \sum_{b_1,b_2\in\{0,1\}^2} \tilde{f}^{(1)}(b_1,b_2)\cdot\tilde{w}^{(1)}(b_1,b_2) \end{aligned}$$

Here, the Evaluations representations of $f^{(1)}(X_1, X_2)$ and $w^{(1)}(X_1, X_2)$ on the two-dimensional Boolean Hypercube can actually be calculated directly from the Evaluations of $\tilde{f}$ and $\tilde{w}$:

$$\begin{aligned} a_0^{(1)} &= \tilde{f}^{(1)}(0,0) = (1-r_0)\cdot\tilde{f}(0,0,0) + r_0\cdot\tilde{f}(1,0,0) \\ a_1^{(1)} &= \tilde{f}^{(1)}(0,1) = (1-r_0)\cdot\tilde{f}(0,0,1) + r_0\cdot\tilde{f}(1,0,1) \\ a_2^{(1)} &= \tilde{f}^{(1)}(1,0) = (1-r_0)\cdot\tilde{f}(0,1,0) + r_0\cdot\tilde{f}(1,1,0) \\ a_3^{(1)} &= \tilde{f}^{(1)}(1,1) = (1-r_0)\cdot\tilde{f}(0,1,1) + r_0\cdot\tilde{f}(1,1,1) \end{aligned}$$

So this requires the Prover to perform 2^{n-1} multiplications and 2^n additions to get the Evaluations representation of $f^{(1)}(X_1, X_2)$, recorded in $(a_0^{(1)}, a_1^{(1)}, a_2^{(1)}, a_3^{(1)})$:

$$\begin{aligned} a_0^{(1)} &= (1-r_0)\cdot a_0 + r_0\cdot a_1 \\ a_1^{(1)} &= (1-r_0)\cdot a_2 + r_0\cdot a_3 \\ a_2^{(1)} &= (1-r_0)\cdot a_4 + r_0\cdot a_5 \\ a_3^{(1)} &= (1-r_0)\cdot a_6 + r_0\cdot a_7 \end{aligned}$$

In addition, the Prover also needs to calculate the Evaluations representation of $\tilde{w}^{(1)}(X_1, X_2)$ in the same way (2^{n-1} multiplications and 2^n additions):

$$
\begin{aligned}
w_0^{(1)} &= \tilde{w}^{(1)}(0,0) = (1-r_0)\cdot\tilde{w}(0,0,0) + r_0\cdot\tilde{w}(1,0,0)\\
w_1^{(1)} &= \tilde{w}^{(1)}(0,1) = (1-r_0)\cdot\tilde{w}(0,0,1) + r_0\cdot\tilde{w}(1,0,1)\\
w_2^{(1)} &= \tilde{w}^{(1)}(1,0) = (1-r_0)\cdot\tilde{w}(0,1,0) + r_0\cdot\tilde{w}(1,1,0)\\
w_3^{(1)} &= \tilde{w}^{(1)}(1,1) = (1-r_0)\cdot\tilde{w}(0,1,1) + r_0\cdot\tilde{w}(1,1,1)
\end{aligned}
$$

These two parts of folding calculations for Multilinear Polynomials together require a total of 2^n multiplications and $2\cdot 2^n$ additions. The calculation results are stored in two arrays $\vec{a}^{(1)}$ and $\vec{w}^{(1)}$ of length 2^{n-1}.

Next, the Prover and Verifier complete the first round of the FRI protocol (Commit-phase), which is to calculate the RS Code of the folded univariate polynomial. Below is the univariate polynomial $\hat{f}(X)$ corresponding to $\tilde{f}(X_0, X_1, X_2)$. Please note that its coefficient form corresponds to the Evaluations representation of $\tilde{f}(X)$, which is different from the form in the Basefold paper [ZCF23]:

$$
\hat{f}(X) = a_0 + a_1X + a_2X^2 + a_3X^3 + a_4X^4 + a_5X^5 + a_6X^6 + a_7X^7
$$

The Prover has already calculated the Evaluations representation of the folded $\tilde{f}^{(1)}(X_1, X_2)$, i.e., $\vec{a}^{(1)}$, in the first round of Sumcheck. It also corresponds to a univariate polynomial $\hat{f}^{(1)}(X)$, whose coefficient form is:

$$
\hat{f}^{(1)}(X) = a_0^{(1)} + a_1^{(1)}X + a_2^{(1)}X^2 + a_3^{(1)}X^3
$$

Next, the Prover needs to calculate the RS Code of $\tilde{f}^{(1)}(X)$. If calculated directly, this would require the Prover to perform $(n2^{n-1})$ multiplications. Fortunately, since RS Code is a "linear code", the Prover can directly fold on the RS Code of $\hat{f}(X)$ using $(1-r_0, r_0)$ to obtain the RS Code of $\hat{f}^{(1)}(X)$.

Let's first review the familiar polynomial decomposition of $\hat{f}(X)$:

$$
\begin{aligned}
\hat{f}(X) &= f_e(X^2) + X\cdot f_o(X^2)\\
\hat{f}(-X) &= f_e(X^2) - X\cdot f_o(X^2)
\end{aligned}
$$

Where $f_e(X)$ is the polynomial composed of even terms, and $f_o(X)$ is the polynomial composed of odd terms:

$$
\begin{aligned}
f_e(X) &= a_0 + a_2X + a_4X^2 + a_6X^3\\
f_o(X) &= a_1 + a_3X + a_5X^2 + a_7X^3
\end{aligned}
$$

Also because

$$
\begin{aligned}
\hat{f}^{(1)}(X^2) &= (1-r_0)\cdot f_e(X^2) + r_0\cdot f_o(X^2)\\
&= (1-r_0)\cdot\frac{\hat{f}(X)+\hat{f}(-X)}{2} + r_0\cdot\frac{\hat{f}(X)-\hat{f}(-X)}{2}
\end{aligned}
$$

Therefore, the Prover only needs to perform the same folding operation on the RS Code of $\hat{f}(X)$ to obtain the RS Code of $\hat{f}^{(1)}(X)$:

$$
\mathsf{encode}(\hat{f}^{(1)})_i = (1-r_0)\cdot\mathsf{encode}(\hat{f})_i + r_0\cdot\mathsf{encode}(\hat{f})_{(i+N/2)},\quad i\in[0,N)
$$

Here, N is the length after encoding a message of length 2^n, where $\rho = 2^n/N$ is the code rate.

At this point, the Prover sends the Merkle Root of $\mathsf{encode}(\hat{f}^{(1)})$ as a commitment, then the Prover and Verifier proceed to the second round of Sumcheck, repeating the above process until the sum length of the

Sumcheck protocol becomes 1. At this time, the accompanying FRI protocol has also folded the polynomial to a constant polynomial $\hat{f}^{(3)}(X)$. If the Prover is honest, then the constant term of $\hat{f}^{(3)}(X)$ happens to be $\tilde{f}(r_0, r_1, r_2)$:

$$\hat{f}^{(3)}(X) = \tilde{f}(r_0, r_1, r_2)$$

This value can be used by the Verifier for verification:

$$v^{(3)} \stackrel{?}{=} \hat{f}^{(3)}(0) \cdot \tilde{eq}((u_0, u_1, u_2), (r_0, r_1, r_2))$$

At this point, the Verifier needs to calculate $\tilde{eq}((u_0, u_1, u_2), (r_0, r_1, r_2))$ to complete the final verification step, which requires $2n$ multiplications:

$$\begin{aligned}\tilde{eq}((u_0, u_1, u_2), (r_0, r_1, r_2)) &= \prod_{i=0}^{2} \Big((1 - r_i) \cdot (1 - u_i) + r_i \cdot u_i\Big) \\ &= \prod_{i=0}^{2} \Big(1 + 2r_i \cdot u_i - r_i - u_i\Big)\end{aligned}$$

For the Query-phase part of FRI, we omit it here.

7.1.4 Sumcheck Performance Analysis

In the Sumcheck protocol part, the Prover's total computation is:

- Calculate $\vec{w}$: 2^n multiplications
- Calculate $h^i(0)$: 2^n multiplications, $2 \cdot 2^n$ additions
- Calculate $h^i(1)$: 2^n multiplications, $2 \cdot 2^n$ additions
- Calculate $h^i(2)$: 2^n multiplications and $5 \cdot 2^n$ additions
- Folding calculation of $\vec{a}$: 2^n multiplications and $2 \cdot 2^n$ additions
- Folding calculation of $\vec{w}$: 2^n multiplications and $2 \cdot 2^n$ additions

Verifier's computation:

- Verify $h^{(i)}(0) + h^{(i)}(1) \stackrel{?}{=} h^{(i-1)}(r_{i-1})$: n additions
- Interpolate to calculate $h^{(i)}(r_i)$: $4n$ divisions, $3n$ multiplications and $9n$ additions
- Calculate $\tilde{eq}(r_0, r_1, \cdots, r_{n-1})$: n multiplications, $4n$ additions
- Final verification: 1 multiplication

7.2 Sumcheck Optimization

In the above protocol, the Prover needs to send three degree-2 polynomials $h^{(0)}(X), h^{(1)}(X), h^{(2)}(X)$. The most straightforward implementation is for the Prover to send the evaluations of these polynomials at $X = 0, 1, 2$. The Verifier checks the first two values, then uses the third value to calculate Lagrange Interpolation to get $h^{(0)}(r_0)$.

We can slightly modify the above protocol to allow the Prover to only send linear polynomials of degree 1. Then the Prover only needs to send the evaluations at two points, and the Verifier only needs to do linear interpolation to calculate the next sum value.

Habock gave an optimized version of the Basefold protocol in [Hab24], which can make $h(X)$ a linear polynomial of degree 1. This optimization technique first appeared in [Gruen24]. Let's briefly describe this optimization technique.

According to the definition of $\tilde{eq}(\vec{X}, \vec{Y})$, it can be decomposed as:

$$\tilde{eq}(\vec{X}_0 \parallel \vec{X}_1, \vec{Y}_0 \parallel \vec{Y}_1) = eq(\vec{X}_0, \vec{Y}_0) \cdot eq(\vec{X}_1, \vec{Y}_1)$$

Let's first observe the definition of $h^{(0)}(X)$:

$$h^{(0)}(X) = \sum_{b_1, b_2 \in \{0,1\}^2} \tilde{f}(X, b_1, b_2) \cdot \tilde{eq}((u_0, u_1, u_2), (X, b_1, b_2))$$

The right side of its equation can be rewritten as:

$$\begin{aligned} h^{(0)}(X) &= \sum_{b_1, b_2 \in \{0,1\}^2} \tilde{f}(X, b_1, b_2) \cdot \tilde{eq}((u_0, u_1, u_2), (X, b_1, b_2)) \\ &= \sum_{b_1, b_2 \in \{0,1\}^2} \tilde{f}(X, b_1, b_2) \cdot eq(u_0, X) \cdot eq((u_1, u_2), (b_1, b_2)) \\ &= eq(u_0, X) \cdot \sum_{(b_1, b_2) \in \{0,1\}^2} \Big(\tilde{f}(X, b_1, b_2) \cdot eq((u_1, u_2), (b_1, b_2)) \Big) \end{aligned}$$

We introduce the notation $g^{(0)}(X)$ to represent the linear polynomial on the right side of the equation except for $eq(u_0, X)$, then $h^{(0)}(X)$ can be represented as the product of two linear polynomials:

$$h^{(0)}(X) = eq(u_0, X) \cdot g^{(0)}(X)$$

We modify the protocol as follows: let the Prover not directly send $h_0(X)$, but send the linear polynomial $g_0(X)$, i.e., $g^{(0)}(0), g^{(0)}(1)$,

$$g^{(0)}(X) = \sum_{(b_1, b_2) \in \{0,1\}^2} \Big(\tilde{f}(X, b_1, b_2) \cdot eq((u_1, u_2), (b_1, b_2)) \Big)$$

After receiving the polynomial $g^{(0)}(0), g^{(0)}(1)$, the Verifier can calculate the values of $h^{(0)}(0), h^{(0)}(1)$ on its own, because:

$$\begin{aligned} h^{(0)}(0) &= (1 - u_0) \cdot g^{(0)}(0) \\ h^{(0)}(1) &= u_0 \cdot g^{(0)}(1) \end{aligned}$$

Then the Verifier can verify $h^{(0)}(0) + h^{(0)}(1) \stackrel{?}{=} v$. If the verification passes, the Verifier can calculate $h^{(0)}(r_0)$:

$$h^{(0)}(r_0) = \Big((1 - u_0)(1 - r_0) + u_0 r_0 \Big) \cdot g^{(0)}(r_0)$$

Here $g^{(0)}(r_0)$ can also be calculated from $g^{(0)}(0), g^{(0)}(1)$:

$$g^{(0)}(r_0) = g^{(0)}(0) + \Big(g^{(0)}(1) - g^{(0)}(0) \Big) \cdot r_0$$

Finally, calculate $h_0(r_0)$. In summary, the Verifier needs one multiplication to verify v, and three multiplications to calculate $h^{(0)}(r_0)$.

7.2.1 Performance Analysis

The Prover's total computation is:

- Calculate $g^i(0)$: 2^n multiplications, $2 \cdot 2^n$ additions
- Calculate $g^i(1)$: 2^n multiplications
- Folding calculation of $\vec{a}$: 2^n multiplications and $2 \cdot 2^n$ additions
- Folding calculation of $\vec{w}$: 2^n multiplications and $2 \cdot 2^n$ additions

Verifier's computation:

- Calculate $h^{(i)}(0)$: n multiplications, n additions
- Calculate $h^{(i)}(1)$: n multiplications, n additions
- Verify $h^{(i)}(0) + h^{(i)}(1)$: n additions
- Calculate $h^{(i)}(r_i)$: $3n$ multiplications, $6n$ additions
- Calculate $\tilde{eq}(r_0, r_1, \cdots, r_{n-1})$: n multiplications, $4n$ additions
- Final verification: 1 multiplication

7.3 Further Optimization of the Verifier

Let's observe the definition of $g^{(0)}(X)$ again:

$$g^{(0)}(X) = \sum_{(b_1,b_2)\in\{0,1\}^2} \Big(\tilde{f}(X, b_1, b_2) \cdot eq((u_1, u_2), (b_1, b_2))\Big)$$

This sum happens to be equal to $\tilde{f}(X, u_1, u_2)$, so we can review the above optimization protocol from another perspective:

In the first step, the Prover sends $g^{(0)}(0), g^{(0)}(1)$, which is actually sending:

$$g^{(0)}(0) = \tilde{f}(0, u_1, u_2)$$
$$g^{(0)}(1) = \tilde{f}(1, u_1, u_2)$$

The Verifier checks the correctness of $h^{(0)}(X)$, which is equivalent to verifying the correctness of $g^{(0)}(X)$:

$$h^{(0)}(u_0) = eq(u_0, u_0) \cdot g^{(0)}(u_0) = g^{(0)}(u_0) = g^{(0)}(0) + \Big(g^{(0)}(1) - g^{(0)}(0)\Big) \cdot u_0 \stackrel{?}{=} v$$

Then the Verifier calculates $g^{(0)}(r_0)$ as the sum value for the next round of Sumcheck.

$$g^{(0)}(r_0) = g^{(0)}(0) + \Big(g^{(0)}(1) - g^{(0)}(0)\Big) \cdot r_0$$

Therefore, it seems we no longer need to introduce $h^{(i)}(X)$, but can directly use $g^{(i)}(X)$. This way, in each round of the Sumcheck protocol, the Prover needs to send $g^{(i)}(0), g^{(i)}(1)$, and the Verifier only needs to perform two multiplications, one to calculate $g^{(i)}(u_i)$, and one to calculate $g^{(i)}(r_i)$. And in the last step of Sumcheck, the Verifier only needs to verify:

$$g^{(n-1)}(r_{n-1}) \stackrel{?}{=} \tilde{f}(r_0, r_1, \cdots, r_{n-1})$$

It's not hard to see that in the i-th round, if the Prover is honest, $g^{(i)}(X)$ should be exactly equal to $\tilde{f}^{(i)}(r_0, r_1, \cdots, r_{i-1}, X, u_{i+1}, \cdots, u_{n-1})$. Let's rewrite the transformed Sumcheck protocol:

Round 1: The Prover sends $\tilde{f}(0, u_1, u_2), \tilde{f}(1, u_1, u_2)$, the Verifier verifies:

$$(1-u_0)\cdot\tilde{f}(0,u_1,u_2)+u_0\cdot\tilde{f}(1,u_1,u_2)\stackrel{?}{=}\tilde{f}(u_0,u_1,u_2)=v$$

The Verifier responds with $r_0\in F$, and calculates $g^{(0)}(r_0)=\tilde{f}(r_0,u_1,u_2)$ as the new sum value $v^{(1)}$:

$$\tilde{f}(r_0,u_1,u_2)=(1-r_0)\cdot\tilde{f}(0,u_1,u_2)+r_0\cdot\tilde{f}(1,u_1,u_2)$$

Round 2: The Prover sends $\tilde{f}(r_0,0,u_2),\tilde{f}(r_0,1,u_2)$, the Verifier verifies:

$$(1-u_1)\cdot\tilde{f}(r_0,0,u_2)+u_1\cdot\tilde{f}(r_0,1,u_2)\stackrel{?}{=}\tilde{f}(r_0,u_1,u_2)=v^{(1)}$$

The Verifier responds with $r_1\in F$, and calculates $g^{(1)}(r_1)=\tilde{f}(r_0,r_1,u_2)$ as the new sum value:

$$\tilde{f}(r_0,r_1,u_2)=(1-r_1)\cdot\tilde{f}(r_0,0,u_2)+r_1\cdot\tilde{f}(r_0,1,u_2)$$

Round 3: The Prover sends $\tilde{f}(r_0,r_1,0),\tilde{f}(r_0,r_1,1)$, the Verifier verifies:

$$(1-u_2)\cdot\tilde{f}(r_0,r_1,0)+u_2\cdot\tilde{f}(r_0,r_1,1)\stackrel{?}{=}\tilde{f}(r_0,r_1,u_2)=v^{(2)}$$

The Verifier responds with $r_2\in F$, and calculates $g^{(2)}(r_2)=\tilde{f}(r_0,r_1,r_2)$ as the new sum value:

$$\tilde{f}(r_0,r_1,r_2)=(1-r_2)\cdot\tilde{f}(r_0,r_1,0)+r_2\cdot\tilde{f}(r_0,r_1,1)$$

Verification step: The Verifier verifies the following equation through the $\tilde{f}(X_0,X_1,X_2)$ Oracle:

$$g^{(2)}(r_2)\stackrel{?}{=}\tilde{f}(r_0,r_1,r_2)$$

This way, the Verifier only needs to perform two multiplications in each round. The first multiplication is to verify the correctness of the sum value $v^{(i)}$, and the second multiplication is to calculate $g^{(i)}(r_i)$, totaling $2n$ multiplications.

7.4 Prover's Computation Optimization

In the three rounds, the Prover needs to send $g^{(0)}(0)$ and $g^{(0)}(1)$, $g^{(1)}(0)$ and $g^{(1)}(1)$, $g^{(2)}(0)$ and $g^{(2)}(1)$ in turn.

$$\begin{aligned}
g^{(0)}(0)&=\tilde{f}(0,u_1,u_2)\\
g^{(0)}(1)&=\tilde{f}(1,u_1,u_2)\\
g^{(1)}(0)&=\tilde{f}(r_0,0,u_2)\\
g^{(1)}(1)&=\tilde{f}(r_0,1,u_2)\\
g^{(2)}(0)&=\tilde{f}(r_0,r_1,0)\\
g^{(2)}(1)&=\tilde{f}(r_0,r_1,1)
\end{aligned}$$

In this section, we'll look at how the Prover can efficiently calculate $g^{(i)}(0),g^{(i)}(1)$.

As mentioned earlier, the Prover maintains an array $\vec{a}$ of length 2^n:

$$
\begin{aligned}
a_0 &= \tilde{f}(0,0,0) \\
a_1 &= \tilde{f}(1,0,0) \\
a_2 &= \tilde{f}(0,1,0) \\
a_3 &= \tilde{f}(1,1,0) \\
a_4 &= \tilde{f}(0,0,1) \\
a_5 &= \tilde{f}(1,0,1) \\
a_6 &= \tilde{f}(0,1,1) \\
a_7 &= \tilde{f}(1,1,1)
\end{aligned}
$$

It's folded in each round of Sumcheck. For example, after the first round of folding (with r_0 as the folding factor), the Prover gets:

$$
\begin{aligned}
a_0^{(1)} &= \tilde{f}^{(1)}(0,0) = (1-r_0)\cdot\tilde{f}(0,0,0) + r_0\cdot\tilde{f}(1,0,0) \\
a_1^{(1)} &= \tilde{f}^{(1)}(0,1) = (1-r_0)\cdot\tilde{f}(0,0,1) + r_0\cdot\tilde{f}(1,0,1) \\
a_2^{(1)} &= \tilde{f}^{(1)}(1,0) = (1-r_0)\cdot\tilde{f}(0,1,0) + r_0\cdot\tilde{f}(1,1,0) \\
a_3^{(1)} &= \tilde{f}^{(1)}(1,1) = (1-r_0)\cdot\tilde{f}(0,1,1) + r_0\cdot\tilde{f}(1,1,1)
\end{aligned}
$$

To efficiently calculate $g^{(i)}(0), g^{(i)}(1)$, we let the Prover do some pre-computation before the Sumcheck protocol starts, obtaining a vector $\vec{d}$ of length $2^n - 1$.

This vector is the result of folding the vector $\vec{a}$ (with u_2, u_1, u_0 as folding factors). We first fold the vector $\vec{a}$ using u_2 to get d_0, d_1, d_2, d_3:

$$
\begin{aligned}
d_0 &= (1-u_2)\cdot a_0 + u_2\cdot a_4 = \tilde{f}(0,0,u_2) \\
d_1 &= (1-u_2)\cdot a_1 + u_2\cdot a_5 = \tilde{f}(1,0,u_2) \\
d_2 &= (1-u_2)\cdot a_2 + u_2\cdot a_6 = \tilde{f}(0,1,u_2) \\
d_3 &= (1-u_2)\cdot a_3 + u_2\cdot a_7 = \tilde{f}(1,1,u_2)
\end{aligned}
$$

Then the Prover performs folding again on d_0, d_1, d_2, d_3 using u_1 as the folding factor to get d_4, d_5:

$$
\begin{aligned}
d_4 &= (1-u_1)\cdot d_0 + u_1\cdot d_2 = \tilde{f}(0,u_1,u_2) \\
d_5 &= (1-u_1)\cdot d_1 + u_1\cdot d_3 = \tilde{f}(1,u_1,u_2)
\end{aligned}
$$

Finally, the Prover performs folding on d_4, d_5 using u_0 as the folding factor to get d_6:

$$
d_6 = (1-u_0)\cdot d_4 + u_0\cdot d_5 = \tilde{f}(u_0,u_1,u_2)
$$

Observe that the end value d_6 of the vector $(d_0, d_1, d_2, d_3, d_4, d_5, d_6)$ happens to be $\tilde{f}(u_0, u_1, u_2)$, and the second-to-last and third-to-last elements happen to be $g^{(0)}(0), g^{(0)}(1)$:

$$
\begin{aligned}
g^{(0)}(0) &= d_4 \\
g^{(0)}(1) &= d_5
\end{aligned}
$$

If the Prover removes d_6 and folds the remaining vector $(d_0, d_1, d_2, d_3, d_4, d_5)$ along with $\vec{a}$ (using r_0 as the folding factor), we can get:

$$d'_0 = (1 - r_0) \cdot d_0 + r_0 \cdot d_1 = \tilde{f}(r_0, 0, u_2)$$
$$d'_1 = (1 - r_0) \cdot d_2 + r_0 \cdot d_3 = \tilde{f}(r_0, 1, u_2)$$
$$d'_2 = (1 - r_0) \cdot d_4 + r_0 \cdot d_5 = \tilde{f}(r_0, u_1, u_2)$$

We surprisingly find that d'_2 happens to be the value of $g^{(0)}(r_0)$, and d'_0, d'_1 happen to be the values of $g^{(1)}(0), g^{(1)}(1)$.

Next, in the second round of Sumcheck, the Prover folds (d'_0, d'_1) along with $\vec{a}'$ using r_1 to get the value of d''_0:

$$d''_0 = (1 - r_1) \cdot d'_0 + r_1 \cdot d'_1 = \tilde{f}(r_0, r_1, u_2)$$

This happens to be the value of $g^{(1)}(r_1)$.

Then in the third round of Sumcheck, the Prover needs to send the values of $g^{(2)}(0), g^{(2)}(1)$, but the vector $\vec{d}$ has already been folded and disappeared. However, at this time, the Prover has the folded vector $\vec{a}$:

$$a_0^{(2)} = \tilde{f}^{(2)}(0) = \tilde{f}(r_0, r_1, 0)$$
$$a_1^{(2)} = \tilde{f}^{(2)}(1) = \tilde{f}(r_0, r_1, 1)$$

These two values happen to be equal to the values of $g^{(2)}(0), g^{(2)}(1)$.

7.4.1 Performance Analysis

To summarize, we only need to let the Prover pre-compute the vector $\vec{d}$ before Sumcheck starts. It stores all the intermediate results of $\tilde{f}(X_0, X_1, X_2)$ after Partial Evaluation by substituting $X_2 = u_2, X_1 = u_1, X_0 = u_0$ in turn. And this vector is folded along with the vector $\vec{a}$ using the same random challenge numbers, so the values of $g^{(i)}(0), g^{(i)}(1), g^{(i)}(r_i)$ can be efficiently calculated. The pre-computation requires $2^n - 1$ multiplications, and the folding requires a total of $2^n - 2$ multiplications.

Prover's computation:

- Pre-compute $\vec{d}$: 2^n multiplications and $2 \cdot 2^n$ additions
- Fold $\vec{a}$: 2^n multiplications and $2 \cdot 2^n$ additions
- Fold $\vec{d}$: 2^n multiplications and $2 \cdot 2^n$ additions

Verifier's computation:

- Verify $g^{(i)}(0), g^{(i)}(1)$: n multiplications and $2n$ additions
- Calculate $g^{(i)}(r_i)$: n multiplications and $2n$ additions

7.5 Further Optimization of the Verifier

In the above protocol, the Prover needs to send $g^{(i)}(0), g^{(i)}(1)$, then the Verifier verifies:

$$g^{(i)}(0) + (g^{(i)}(1) - g^{(i)}(0)) \cdot u_i \stackrel{?}{=} g^{(i-1)}(r_{i-1}) = g^{(i)}(u_i)$$

So actually, the Prover only needs to send one value $g^{(i)}(0)$ in each round of Sumcheck, then the Verifier can calculate $g^{(i)}(1)$ backwards based on the "verification equation":

$$g^{(i)}(1) = \frac{g^{(i-1)}(r_{i-1}) - g^{(i)}(0)}{u_i} + g^{(i)}(0)$$

This way, the Verifier uses one division calculation to exchange for a reduction in communication (by half). But note that the computational cost of finite field division (finding inverse) is significantly higher than multiplication, with a complexity of at least $o(\log q)$. But anyway, we can further reduce the communication, i.e., the Proof size, through this method.

In fact, it's not hard to see that this relatively costly division can be optimized away. This idea comes from Deepfold []. We let the Prover send $g^{(i)}(u_i + 1)$ instead of $g_i(0)$ or $g_i(1)$ in each round of Sumcheck:

$$g^{(i)}(u_i + 1) = g^{(i)}(u_i) + g^{(i)}(1) - g^{(i)}(0)$$

Again, $g^{(i)}(u_i)$ is the tail element after each folding of the vector $\vec{d}$, so its computation cost is already included in the folding calculation of $\vec{d}$. Therefore, the Prover's cost is just two additional additions in each round.

Then, because the Verifier already has $g^{(i)}(u_i) = g^{(i-1)}(r_{i-1})$ at this time, the Verifier can get $g^{(i)}(r_i)$ without division:

$$g^{(i)}(r_i) = g^{(i)}(u_i) + \Big(g^{(i)}(u_i + 1) - g^{(i)}(u_i)\Big) \cdot (r_i - u_i)$$

The computational cost is only three additions and one multiplication.

7.5.1 Performance Analysis

Prover's computation:

- Pre-compute $\vec{d}$: 2^n multiplications and $2 \cdot 2^n$ additions
- Fold $\vec{a}$: 2^n multiplications and $2 \cdot 2^n$ additions
- Fold $\vec{d}$: 2^n multiplications and $2 \cdot 2^n$ additions
- Calculate $g^{(i)}(u_i + 1)$: $2n$ additions

Verifier's computation:

- Calculate $g^{(i)}(r_i)$: n multiplications and $3n$ additions

7.6 Understanding Sumcheck Optimization from Another Perspective

After multiple steps of optimization above, we have reached a relatively ideal state. Let's see if we can understand this concise protocol more easily.

We can view the above optimized protocol as a Recursive Split-and-fold style sum proof. Because $\tilde{f}(u_0, u_1, u_2)$ itself is a sum of 8 terms:

$$\begin{aligned}
\tilde{f}(u_0, u_1, u_2) &= a_0(1-u_0)(1-u_1)(1-u_2) + a_1u_0(1-u_1)(1-u_2) \\
&\quad + a_2(1-u_0)u_1(1-u_2) + a_3u_0u_1(1-u_2) \\
&\quad + a_4(1-u_0)u_1u_2 + a_5u_0(1-u_1)u_2 \\
&\quad + a_6u_0u_1(1-u_2) + a_7u_0u_1u_2 \\
&= (1-u_0) \cdot \Big(a_0(1-u_1)(1-u_2) + a_2u_1(1-u_2) + a_4(1-u_1)u_2 + a_6u_1u_2\Big) \\
&\quad + u_0 \cdot \Big(a_1(1-u_1)(1-u_2) + a_3u_1(1-u_2) + a_5(1-u_1)u_2 + a_7u_1u_2\Big)
\end{aligned}$$

In the first round, the Prover sends the sum of 4 terms in the left bracket and the sum of 4 terms in the right bracket, recorded as $g_0(0)$ and $g_0(1)$, and sends them to the Verifier:

$$g_0(0) = a_0(1-u_1)(1-u_2) + a_2u_1(1-u_2) + a_4(1-u_1)u_2 + a_6u_1u_2$$
$$g_0(1) = a_1(1-u_1)(1-u_2) + a_3u_1(1-u_2) + a_5(1-u_1)u_2 + a_7u_1u_2$$

Or

$$g_0(0) = \tilde{f}(0, u_1, u_2)$$
$$g_0(1) = \tilde{f}(1, u_1, u_2)$$

Then the Verifier first verifies whether the result of the inner product of $(g_0(0), g_0(1))$ and $(1-u_0, u_0)$ is equal to the sum v to be proved,

$$(1-u_0) \cdot g_0(0) + u_0 \cdot g_0(1) \stackrel{?}{=} \tilde{f}(u_0, u_1, u_2)$$

This way, the Prover and Verifier have reduced the proof of a sum of length 8 to two proofs of sums of length 4. And because these two new sums actually have the same structure, that is, they are both the result of the inner product of the odd and even term coefficients of the polynomial with the same vector $(1-u_1, u_1) \otimes (1-u_2, u_2)$,

$$g^{(0)}(0) = \langle (a_0, a_2, a_4, a_6), (1-u_1, u_1) \otimes (1-u_2, u_2) \rangle$$
$$g^{(0)}(1) = \langle (a_1, a_3, a_5, a_7), (1-u_1, u_1) \otimes (1-u_2, u_2) \rangle$$

So the Verifier can give a random number r_0, using it to merge (Batching) these two new sum proofs together. Coincidentally, the merged sum value of length 4 happens to be $g^{(0)}(r_0)$, and it equals $\tilde{f}(r_0, u_1, u_2)$. Of course, this is not a coincidence, because we deliberately use $(1-r_0, r_0)$ this Multilinear Basis to fold $g^{(0)}(0), g^{(0)}(1)$, with the purpose of keeping it equal to $g^{(0)}(r_0)$:

$$(1-r_0) \cdot g_0(0) + r_0 \cdot g_0(1) = \tilde{f}(r_0, u_1, u_2)$$

This way, the Prover and Verifier next prove that the sum value of the merged vector of length 4 equals $\tilde{f}(r_0, u_1, u_2)$, that is

$$(1-r_0) \cdot g^{(0)}(0) + r_0 \cdot g^{(0)}(1) \stackrel{?}{=} a'_0w'_0 + a'_1w'_1 + a'_2w'_2 + a'_3w'_3$$

And so on, this protocol is consistent with the idea of the Sumcheck protocol, which is to split a sum equation into sums of different parts, and then use random numbers to merge these different sum segments together to prove. Until the last round, the sum proof is Reduced to the polynomial evaluation proof, and the Prover and Verifier use other tools to supplement the last part of the proof.

The original Sumcheck in Basefold protocol did not utilize the internal structure of the sum, but adopted a more general proof of inner product sum. Therefore, the concise protocol introduced in this article fully utilizes the internal structure of the sum terms.

7.7 References

- [GLH+24] Yanpei Guo, Xuanming Liu, Kexi Huang, Wenjie Qu, Tianyang Tao, and Jiaheng Zhang. "DeepFold: Efficient Multilinear Polynomial Commitment from Reed-Solomon Code and Its Application to Zero-knowledge Proofs." *Cryptology ePrint Archive* (2024).
- [ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. "WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification." *Cryptology ePrint Archive* (2024).

- [ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. "BaseFold: efficient field-agnostic polynomial commitment schemes from foldable codes." Annual International Cryptology Conference. Cham: Springer Nature Switzerland, 2024.
- [Hab24] Ulrich Haböck. "Basefold in the List Decoding Regime." *Cryptology ePrint Archive*(2024).

Chapter 8

Note on Basefold's Soundness Proof under List Decoding

- Jade Xie jade@secbit.io
- Yu Guo yu.guo@secbit.io

This article mainly outlines the security proof for the Basefold [ZCF23] multilinear PCS under list decoding, as presented in Ulrich Haböck's paper [H24]. In [ZCF23], the soundness proof was given under unique decoding for foldable linear codes, while in [H24], the proof is for Reed-Solomon codes under list decoding, raising the bound to the Johnson bound, i.e., $1 - \sqrt{\rho}$. To prove security, the paper presents two correlated agreement theorems stronger than the one given in [BCIKS20]:

1. [H24, Theorem 3] Correlated agreement for subcodes.
2. [H24, Theorem 4] Weighted correlated agreement for subcodes.

When considering the Basefold protocol applied to Reed-Solomon codes, the protocol combines FRI and sumcheck. To prove its security, [H24] proposes subcodes that incorporate sumcheck-like constraints on top of Reed-Solomon codes. By combining this with the corresponding correlated agreement theorems, the security of the protocol can be proven.

8.1 Basefold Protocol

For a multilinear polynomial $P(X_1, X_2, \dots, X_n) \in F[X_1, \dots, X_n]$, we want to prove that for any query $\vec{\omega} = (\omega_1, \dots, \omega_n)$ from F^n, we have $v = P(\omega_1, \dots, \omega_n)$. To implement PCS for the multilinear polynomial $P(X_1, X_2, \dots, X_n)$, the Basefold protocol combines the Sumcheck and FRI protocols. The following introduction is based on the description in [H24].

8.1.1 Combining with Sumcheck Protocol

To prove $v = P(\omega_1, \dots, \omega_n)$, we first convert the query value $P(\omega_1, \dots, \omega_n)$ into a Sumcheck sum form:

$$P(\omega_1, \dots, \omega_n) = \sum_{\vec{x}=(x_1,\dots,x_n)\in H_n} L(\vec{x}, \vec{\omega}) \cdot P(\vec{x})$$

where $H_n = \{0, 1\}^n$, and $L(\vec{x}, \vec{\omega})$ is actually the $eq(\cdot, \cdot)$ function:

$$L(\vec{x}, \vec{\omega}) = \prod_{i=1}^{n} ((1 - x_i)(1 - \omega_i) + x_i\omega_i)$$

Therefore, the proof of $v = P(\omega_1, \dots, \omega_n)$ is transformed into proving the sum over H_n:

$$\sum_{\vec{x}=(x_1,\dots,x_n)\in H_n} L(\vec{x}, \vec{\omega}) \cdot P(\vec{x}) = v$$

The Sumcheck protocol can then be used to prove this sum is correct.

For $i = 1, \dots, n-1$, the Prover needs to construct a univariate polynomial based on the challenge random numbers $\lambda_1, \dots, \lambda_i$:

$$q_i(X) = \sum_{\vec{x}=(x_{i+2},\dots,x_n)\in H_{n-(i+1)}} L(\lambda_1, \dots, \lambda_i, X, \vec{x}, \vec{\omega}) \cdot P(\lambda_1, \dots, \lambda_i, X, \vec{x})$$

This corresponds to the polynomial $P(\lambda_1, \dots, \lambda_i, X, \omega_{i+2}, \dots, \omega_n)$.

We can see that in $q_i(X)$, $L(\lambda_1, \dots, \lambda_i, X, \vec{x}, \vec{\omega})$ is linear in X, and $P(\lambda_1, \dots, \lambda_i, X, \vec{x})$ is also linear in X. Their product becomes quadratic in X. To correspond with the correlated agreement theorem for linear subcodes later, we extract the linear term in X. The Prover needs to send the linear polynomial:

$$\Lambda_i(X) = \sum_{\vec{x}=(x_{i+2},\dots,x_n)\in H_{n-(i+1)}} L(\vec{x}, (\omega_{i+2}, \dots, \omega_n)) \cdot P(\lambda_1, \dots, \lambda_i, X, \vec{x})$$

Since

$$\begin{aligned}
L((\lambda_1, \dots, \lambda_i, X, \vec{x}), \vec{\omega}) &= L((\lambda_1, \dots, \lambda_i, X, \vec{x}), (\omega_1, \dots, \omega_i, \omega_{i+1}, \omega_{i+2}, \dots, \omega_n)) \\
&= \left(\prod_{j=1}^{i} [(1-\lambda_j)(1-\omega_j) + \lambda_j \omega_j] \right) \\
&\quad \cdot ((1-\lambda_{i+1})(1-\omega_{i+1}) + X \cdot \omega_{i+1}) \\
&\quad \cdot \left(\prod_{j=i+2}^{n} [(1-\lambda_j)(1-\omega_j) + \lambda_j \omega_j] \right) \\
&= L(\lambda_1, \dots, \lambda_i, \omega_1, \dots, \omega_i) \cdot L(X, \omega_{i+1}) \cdot L(\vec{x}, (\omega_{i+2}, \dots, \omega_n))
\end{aligned}$$

Therefore,

$$q_i(X) = L(\lambda_1, \dots, \lambda_i, \omega_1, \dots, \omega_i) \cdot L(X, \omega_{i+1}) \cdot \Lambda_i(X).$$

The Prover only needs to provide $\Lambda_i(X)$, and the Verifier can calculate $q_i(X)$ using the above equation.

In the Sumcheck protocol, the Prover first sends a univariate polynomial $\Lambda_0(X) = \sum_{\vec{x}=(x_2,\dots,x_n)\in H_{n-1}} L(\vec{x}, (\omega_2, \dots, \omega_n)) \cdot P(X, \vec{x})$ and $s_0 = v$. Then in round $1 \le i \le n-1$:

1. The Verifier can calculate $q_{i-1}(X)$ based on $\Lambda_{i-1}(X)$ and check if $s_{i-1} = q_{i-1}(0) + q_{i-1}(1)$. Then choose a random number $\lambda_i \leftarrow \$F$ and send it to the Prover.
2. The Prover calculates $P(\lambda_1, \dots, \lambda_i, X_{i+1}, \dots, X_n)$ based on λ_i, computes $\Lambda_i(X)$ and sends it to the Verifier. Both the Prover and Verifier set $s_i = q_{i-1}(\lambda_i)$.

In the last step of Sumcheck, we need to obtain the value of $P(X_1, \dots, X_n)$ at a random point $(\lambda_1, \lambda_2, \dots, \lambda_n)$, i.e.,

$$P(\lambda_1, \dots, \lambda_n),$$

This value can be obtained by folding a univariate polynomial $f_0(X)$ of degree not exceeding $2^n - 1$ corresponding to the multilinear polynomial $P(X_1, X_2, \dots, X_n)$ using the same random numbers $\lambda_1, \dots, \lambda_n$ in the FRI protocol. After folding $f_0(X)$ n times, we will get a constant c, and we want its value to be $P(\lambda_1, \dots, \lambda_n) = c$.

8.1.2 Combining with FRI Protocol

For the multilinear polynomial $P(X_1, X_2, \dots, X_n)$, there is a univariate polynomial $f_0(X)$ (called *univariate representation* in [H24]) corresponding to it:

$$f_0(X) = \sum_{i=0}^{2^n-1} P(i_1, \dots, i_n) \cdot X^i$$

where $i_1, \dots, i_n$ is the binary representation of i, with i_1 being the least significant bit and i_n being the most significant bit.

For example, when $n = 3$, suppose the multilinear polynomial is:

$$P(X_1, X_2, X_3) = a_0 + a_1X_1 + a_2X_2 + a_3X_1X_2 + a_4X_3 + a_5X_1X_3 + a_6X_2X_3 + a_7X_1X_2X_3$$

Then the univariate polynomial $f_0(X)$ corresponding to $P(X_1, X_2, X_3)$ is:

$$\begin{aligned}
f_0(X) &= \sum_{i=0}^{7} P(i_1, i_2, i_3) \cdot X^i \\
&= P(0,0,0) + P(1,0,0)X + P(0,1,0)X^2 + P(1,1,0)X^3 \\
&\quad + P(0,0,1)X^4 + P(1,0,1)X^5 + P(0,1,1)X^6 + P(1,1,1)X^7 \\
&= (P(0,0,0) + P(0,1,0)X^2 + P(0,0,1)X^4) \\
&\quad + X \cdot (P(1,0,0) + P(1,1,0)X^2 + P(1,0,1)X^4 + P(1,1,1)X^6) \\
&= f_{0,0}(X^2) + X \cdot f_{0,1}(X^2)
\end{aligned}$$

Here, $f_{0,0}(X^2)$ corresponds to the even terms of $f_0(X)$, while $f_{0,1}(X^2)$ corresponds to the odd terms. We can see that the coefficients in the even terms $P(0,0,0) + P(0,1,0)X^2 + P(0,0,1)X^4$ correspond to $P(0, \cdot, \cdot)$ in the multilinear polynomial, while the coefficients in the odd terms correspond to $P(1, \cdot, \cdot)$. In other words, $f_{0,0}(X)$ is the *univariate representation* of $P(0, X_2, X_3)$, and $f_{0,1}(X)$ is the univariate representation of $P(1, X_2, X_3)$, because:

$$\begin{aligned}
f_{0,0}(X) &= \sum_{i=0}^{3} P(0, i_1, i_2) \cdot X^i \\
f_{0,1}(X) &= \sum_{i=0}^{3} P(1, i_1, i_2) \cdot X^i
\end{aligned}$$

Using λ_1 to fold $f_{0,0}(X)$ and $f_{0,1}(X)$, we get:

$$f_1(X) = (1 - \lambda_1) \cdot f_{0,0}(X) + \lambda_1 \cdot f_{0,1}(X)$$

And $f_1(X)$ is precisely the *univariate representation* of $P(\lambda_1, X_2, X_3)$. Note that the folding method here is not the common one in the FRI protocol:

$$f_1(X) = f_{0,0}(X) + \lambda_1 \cdot f_{0,1}(X)$$

This is because in this case, the resulting $f_1(X)$ does not correspond to $P(\lambda_1, X_2, \dots, X_n)$. This is tied to the correspondence relationship between univariate polynomials and multilinear polynomials. Under this folding method, their correspondence relationship should change to (the WHIR paper [ACFY24] adopts this correspondence method):

$$f_0(X) = P(X^{2^0}, X^{2^1}, \dots, X^{2^{n-1}})$$

We won't elaborate here on how $f_1(X)$ can correspond to $P(\lambda_1, X_2, \dots, X_n)$ under this correspondence relationship.

Returning to the correspondence relationship between univariate polynomials and multilinear polynomials given in [H24], let's now derive that $f_1(X)$ obtained by folding $f_0(X)$ with $1 - \lambda_1$ and λ_1 indeed corresponds to $P(\lambda_1, X_2, X_3)$. For general $P(X_1, \dots, X_n)$, we have:

$$P(\vec{X}) = \sum_{\vec{x} \in H_n} P(\vec{x}) \cdot L(\vec{x}, \vec{X})$$

Therefore,

$$\begin{aligned}
P(\lambda_1, X_2, \dots, X_n) &= \sum_{\vec{b} \in H_n} P(\vec{b}) \cdot L(\vec{b}, (\lambda_1, X_2, \dots, X_n)) \\
&= \sum_{\vec{b} \in H_n} \left(P(\vec{b}) \cdot ((1-b_1)(1-\lambda_1) + b_1\lambda_1) \prod_{i=2}^{n} [(1-b_i)(1-X_i) + b_i X_i] \right) \\
&= \sum_{\vec{b} \in H_{n-1}} \left(P(0, \vec{b}) \cdot ((1-0)(1-\lambda_1) + 0 \cdot \lambda_1) \prod_{i=2}^{n} [(1-b_i)(1-X_i) + b_i X_i] \right) \\
&\quad + \sum_{\vec{b} \in H_{n-1}} \left(P(1, \vec{b}) \cdot ((1-1)(1-\lambda_1) + 1 \cdot \lambda_1) \prod_{i=2}^{n} [(1-b_i)(1-X_i) + b_i X_i] \right) \\
&= \sum_{\vec{b} \in H_{n-1}} \left(P(0, \vec{b}) \cdot (1-\lambda_1) \prod_{i=2}^{n} [(1-b_i)(1-X_i) + b_i X_i] \right) \\
&\quad + \sum_{\vec{b} \in H_{n-1}} \left(P(1, \vec{b}) \cdot \lambda_1 \prod_{i=2}^{n} [(1-b_i)(1-X_i) + b_i X_i] \right) \\
&= (1-\lambda_1) \sum_{\vec{b} \in H_{n-1}} \left(P(0, \vec{b}) \cdot \prod_{i=2}^{n} [(1-b_i)(1-X_i) + b_i X_i] \right) \\
&\quad + \lambda_1 \sum_{\vec{b} \in H_{n-1}} \left(P(1, \vec{b}) \cdot \prod_{i=2}^{n} [(1-b_i)(1-X_i) + b_i X_i] \right) \\
&= (1-\lambda_1) P(0, X_2, \dots, X_n) + \lambda_1 P(1, X_2, \dots, X_n)
\end{aligned}$$

Thus, we have:

$$
\begin{aligned}
f_1(X) &= (1-\lambda_1)\cdot f_{0,0}(X) + \lambda_1 \cdot f_{0,1}(X) \\
&= (1-\lambda_1)\cdot \sum_{i=0}^{3} P(0,i_1,i_2)\cdot X^i + \lambda_1 \cdot \sum_{i=0}^{3} P(1,i_1,i_2)\cdot X^i \\
&= \sum_{i=0}^{3} ((1-\lambda_1)P(0,i_1,i_2) + \lambda_1 P(1,i_1,i_2))\cdot X^i \\
&= \sum_{i=0}^{3} P(\lambda_1,i_1,i_2)\cdot X^i
\end{aligned}
$$

This also shows that $f_1(X)$ is the univariate representation of $P(\lambda_1, X_2, X_3)$. Then, by folding $f_1(X)$ in this way using random numbers λ_2, λ_3, we finally get a constant polynomial whose value corresponds exactly to $P(\lambda_1, \lambda_2, \lambda_3)$. To summarize, the Basefold protocol performs the Sumcheck protocol on the multilinear polynomial using random numbers on one hand, and the FRI protocol on the corresponding univariate polynomial using the same random numbers on the other hand, thus achieving PCS for multilinear polynomials. This corresponds to [H24, Protocol 1], which is the Basefold protocol for Reed-Solomon codes. The overall protocol idea is as such, so we won't repeat the specific protocol process here. See [H24, Protocol 1] for details.

> **typo** In [H24, Protocol 1], during the Query phase, the paper states that the folding relationship to be checked is:
>
> $$f_{i+1}(x_{i+1}) = \frac{f_0(x_i) + f_0(-x_i)}{2} + \lambda_i \cdot \frac{f_0(x_i) + f_0(-x_i)}{2\cdot x_i}$$
>
> However, based on the folding relationship given earlier, I believe it should be changed to:
>
> $$f_{i+1}(x_{i+1}) = (1-\lambda_i)\cdot \frac{f_0(x_i) + f_0(-x_i)}{2} + \lambda_i \cdot \frac{f_0(x_i) - f_0(-x_i)}{2\cdot x_i}$$

The soundness proof in [H24] is for a more general protocol, namely the batch version of the Basefold protocol.

> **Protocol 2** [H24, Protocol 2] (Batch Reed-Solomon code Basefold). The prover shares the Reed-Solomon codewords $g_0, \ldots, g_M \in \mathcal{C}_0 = \mathrm{RS}_{2^n}[F, D_0] = \{q(x)|_{x\in D_0} : q(x) \in F[X]^{<2^n}\}$ of the multilinears $G_0, \ldots, G_M$, together with their evaluation claims $v_0, \ldots, v_M$ at $\vec{\omega} \in F^n$ with the verifier. Then they engage in the following extension of Protocol 1: 1. In a preceding round $i = 0$, the verifier sends a random $\lambda_0 \leftarrow \$F$, and the prover answers with the oracle for
>
> $$f_0 = \sum_{k=0}^{M} \lambda_0^k \cdot g_k \tag{1}$$
>
> Then both prover and verifier engage in Protocol 1 on f_0 and the claim $v_0 = \sum_{k=0}^{M} \lambda_0^k \cdot v_k$. In addition to the checks in Protocol 1, the verifier also checks that equation (1) holds at every sample x from D_0 .

The batch version of the Basefold protocol essentially uses a random number λ_0 to linearly combine $g_0, \ldots, g_M$ through its powers, transforming them into a function f_0, and then applies Protocol 1 to it.

8.2 Soundness Overview

This section mainly analyzes the proof approach for the soundness error of Protocol 2. First, let's explain the meaning of soundness error. For any potentially malicious Prover P^*, if there exists a g_k among the provided $g_0, \ldots, g_M$ that is more than θ distant from the Reed-Solomon encoding space $\mathcal{C}_0$ ([H24] studies the proof

under list decoding, so it considers the parameter $\theta \in \left(\frac{1-\rho}{2}, 1-\sqrt{\rho}\right)$), or if the multilinear representation P_k corresponding to g_k does not satisfy the evaluation claim $P_k(\vec{\omega}) = v_k$, under this condition, the probability that P^* passes the Verifier's checks does not exceed ε. This probability ε is called the soundness error. In other words, the soundness error analyzes the probability that a malicious Prover P^* can luckily pass the Verifier's checks. P^* might luckily pass the checks at places where randomness is introduced. Analyzing the protocol, we find three such places: 1. Commit phase 1. Using the random number λ_0 to batch $g_0, \ldots, g_M$, let this probability be ε_{C_1}. 2. Using random numbers $\lambda_1, \ldots, \lambda_n$ for the sumcheck protocol and FRI-like folding process, let this probability be ε_{C_2}. 2. Query phase 1. The Verifier randomly selects $x_0 \leftarrow D_0$ to check if the folding is correct, let this probability be $\varepsilon_{\text{query}}$

Therefore, the soundness error of the entire protocol is:

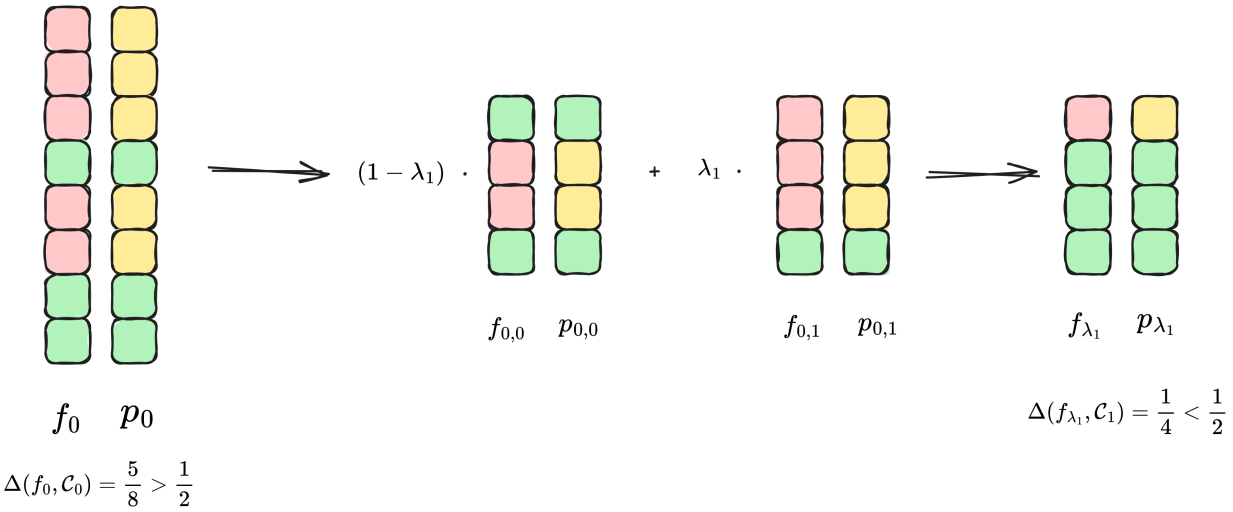
$$\varepsilon < \varepsilon_{C_1} + \varepsilon_{C_2} + \varepsilon_{\text{query}}.$$

8.2.1 Commit Phase

Now let's consider the case of folding f_0 into f_{λ_1} using $\lambda_1 \leftarrow \$F$, i.e.,

$$f_{\lambda_1} = (1-\lambda_1) \cdot f_{0,0} + \lambda_1 \cdot f_{0,1}$$

Assume the given parameter $\theta = \frac{1}{2}$. Since λ_1 is a random number selected from F, the following situation might occur:



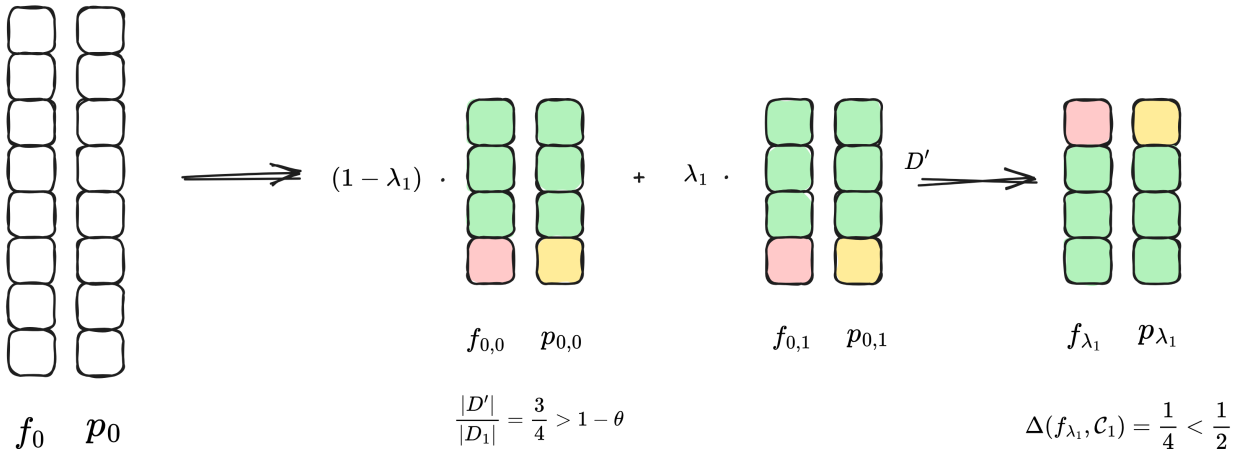
In the figure, $p_0, p_{0,0}, p_{0,1}, p_{\lambda_1}$ are the closest codewords in the corresponding Reed-Solomon encoding space to $f_0, f_{0,0}, f_{0,1}, f_{\lambda_1}$ respectively. The same green color indicates that they have the same value at that point, while different colors indicate different values at that point. We can see that for the f_0 provided by the malicious Prover, its distance from the Reed-Solomon space $\mathcal{C}_0$ is greater than $\theta = \frac{1}{2}$, but after folding with λ_1, the resulting f_{λ_1} might end up less than θ distant from the Reed-Solomon space $\mathcal{C}_1$. This way, f_1 would pass the Verifier's folding verification in the subsequent protocol, and P^* would successfully deceive the Verifier.

So what's the probability of this situation occurring? It's given by the Correlated Agreement theorem from [BCIKS20]. This theorem states that if

$$\Pr_{\lambda_1 \in F}[\Delta((1-\lambda_1)f_{0,0} + \lambda_1 f_{0,1}, \mathcal{C}_1) \leq \theta] > \epsilon$$

where ϵ is an expression related to $\theta, \rho, |F|, |D_1|$, which can also be written as $\epsilon(\theta, \rho, |F|, |D_1|)$, and its form differs under unique decoding and list decoding (this part will be explained in detail in the next section). In other words, if we take all possible λ_1 in F to get f_{λ_1}, and the proportion of those not exceeding θ in distance from $\mathcal{C}_1$ is greater than ϵ, then there must exist a subset $D' \subset D_1$ and codewords $p_{0,0}, p_{0,1}$ in $\mathcal{C}_1$ such that:

1. $|D'|/|D_1| \geq 1 - \theta$,
2. $f_{0,0}|_{D'} = p_{0,0}|_{D'}$ and $f_{0,1}|_{D'} = p_{0,1}|_{D'}$.



Now we can see that not only are $f_{0,0}$ and $f_{0,1}$ close to the encoding space $\mathcal{C}_1$, but they also share the same set D' where they match the corresponding codewords. This is a good conclusion that can help us derive the distance of the original f_0 to $\mathcal{C}_0$.

Through the mapping $\pi : x \mapsto x^2$, points in D_0 can be mapped to D_1. Now we can use π^{-1} to map the points in $D' \subseteq D_1$ back to D_0. For example, let $\omega^8 = 1$ and

$$D_0 = \{\omega^0, \omega^1, \omega^2, \omega^3, \omega^4, \omega^5, \omega^6, \omega^7\}$$

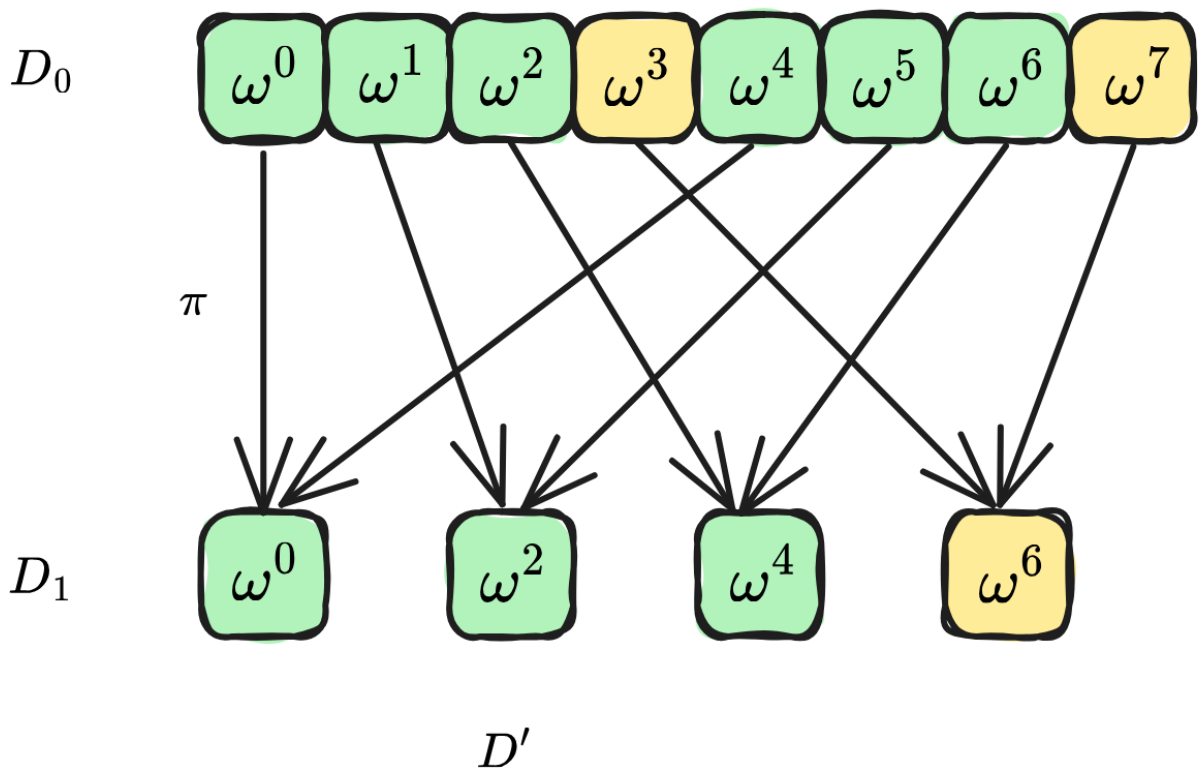
Then through the mapping $\pi : x \mapsto x^2$, we can get

$$D_1 = \{\omega^0, \omega^2, \omega^4, \omega^6\}$$

Suppose $D' = \{\omega^0, \omega^2, \omega^4\}$, then we can get

$$\pi^{-1}(D') = \{\omega^0, \omega^1, \omega^2, \omega^4, \omega^5, \omega^6\}$$

As shown in the following figure:

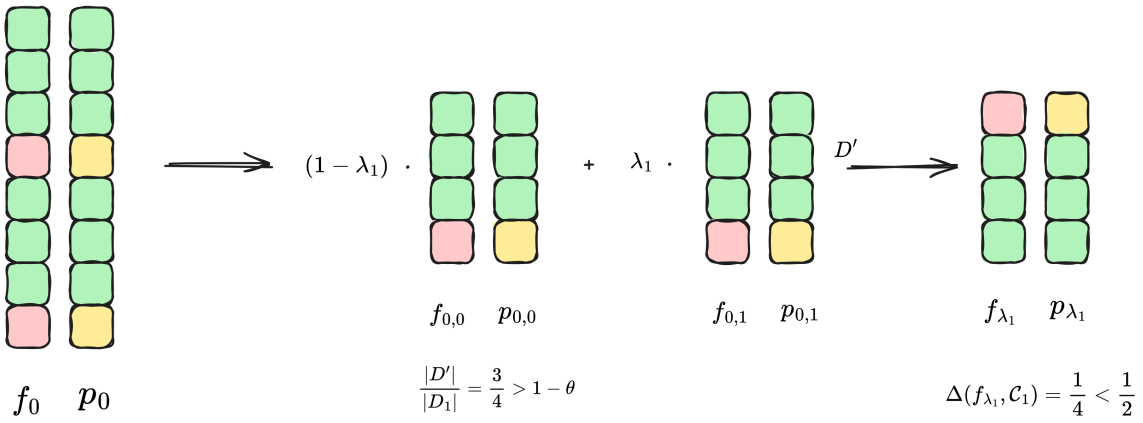


Now, according to the correlated agreement theorem, we have $f_{0,0}|_{D'} = p_{0,0}|_{D'}$ and $f_{0,1}|_{D'} = p_{0,1}|_{D'}$. Therefore, we can obtain the polynomial before folding based on $p_{0,0}$ and $p_{0,1}$:

$$p_0(X) = p_{0,0}(X^2) + X \cdot p_{0,1}(X^2)$$

We can conclude that $f_0(X)$ and $p_0(X)$ have consistent values on $\pi^{-1}(D')$, thus obtaining the distance of $f_0(X)$ to the encoding space $\mathcal{C}_0$:

$$\Delta(f_0, \mathcal{C}_0) \leq \frac{|\pi^{-1}(D')|}{|D_0|} \leq \theta$$

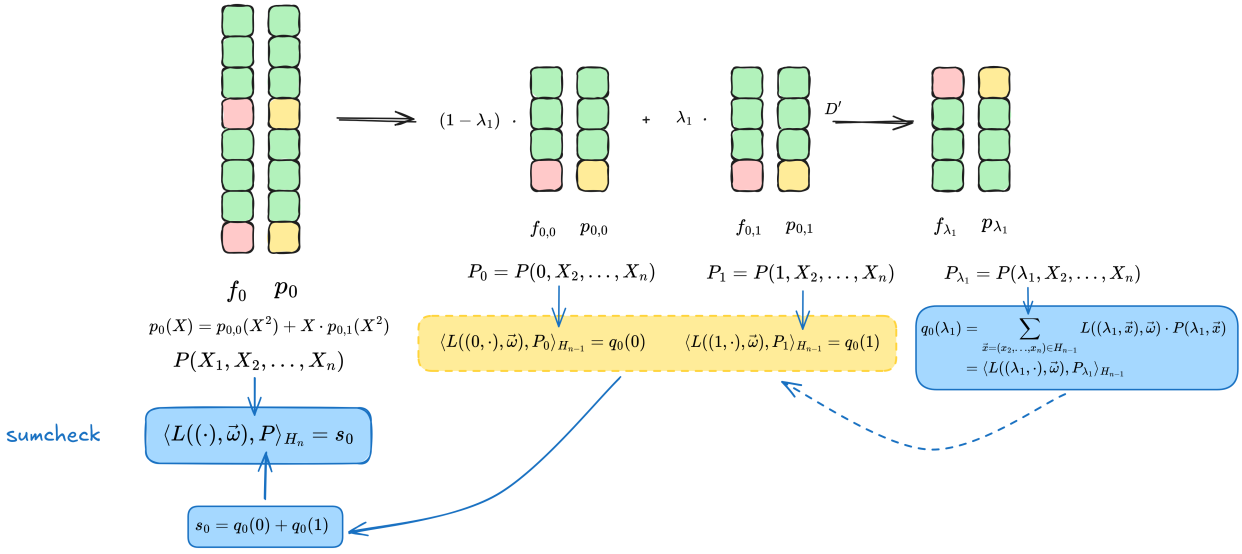


$$p_0(X) = p_{0,0}(X^2) + X \cdot p_{0,1}(X^2)$$

$$\Delta(f_0, \mathcal{C}_0) < \theta$$

This also indicates that the Prover did not cheat, and the function f_0 is not more than θ distant from the corresponding encoding space. Returning to the initial question, we wanted to analyze the probability of a cheating Prover successfully deceiving the Verifier. Now the correlated agreement theorem tells us that except for a probability ϵ, we can ensure the Prover did not cheat, which also means that if the Prover cheats, the probability of successfully deceiving the Verifier will not exceed this probability ϵ.

Have we finished analyzing the soundness error of the Commit phase? Reviewing the above analysis, we used the correlated agreement theorem to obtain the probability that the folded polynomial could deceive the Verifier due to the introduction of the folding random number λ_1. However, one thing to remember is that the Basefold protocol not only checks if the FRI-like folding is correct but also simultaneously checks the sumcheck constraint. Therefore, the above analysis is not sufficient. Following the idea of the correlated agreement theorem, we add the sumcheck constraint on top of it. If there exists a polynomial p_{λ_1} corresponding to $P_{\lambda_1} = P(\lambda_1, X_2, \ldots, X_n)$ that satisfies the sumcheck constraint, we want to obtain $P_0 = P(0, X_2, \ldots, X_n)$ and $P_1 = P(1, X_2, \ldots, X_n)$ corresponding to $p_{0,0}(X)$ and $p_{0,1}(X)$ that also satisfy the sumcheck constraint. This way, we can infer whether the sumcheck constraint is satisfied before folding.



Now consider the sumcheck constraint. We know:

$$\langle L((\lambda_1, \cdot), \vec{\omega}), P_{\lambda_1} \rangle_{H_{n-1}} = q_0(\lambda_1)$$

We want to obtain:

$$\langle L((0, \cdot), \vec{\omega}), P_0 \rangle_{H_{n-1}} = q_0(0) \tag{2}$$

$$\langle L((1, \cdot), \vec{\omega}), P_1 \rangle_{H_{n-1}} = q_0(1) \tag{3}$$

If equations (2) and (3) hold, since $s_0 = q_0(0) + q_0(1)$, we can conclude that the multilinear polynomial $P(X)$ corresponding to $p_0(X)$ obtained from $p_{0,0}(X)$ and $q_{0,1}(X)$ satisfies the sumcheck constraint.

Following the approach in Section 3.2 of [H24], we derive that equations (2) and (3) hold. Based on the relationship between $q_i(X)$ and $\Lambda_i(X)$, we get:

$$q_0(\lambda_1) = L(\lambda_1, \omega_1) \cdot \Lambda_0(\lambda_1)$$

And:

$$\langle L((\lambda_1, \cdot), \vec{\omega}), P_{\lambda_1} \rangle_{H_{n-1}} = L(\lambda_1, \omega_1) \cdot \langle L(\cdot, \omega_2, \dots, \omega_n), P_{\lambda_1} \rangle_{H_{n-1}}$$

Therefore:

$$L(\lambda_1, \omega_1) \cdot \langle L(\cdot, \omega_2, \dots, \omega_n), P_{\lambda_1} \rangle_{H_{n-1}} = L(\lambda_1, \omega_1) \cdot \Lambda_0(\lambda_1)$$

Since $L(X, \omega_1) = (1 - X)(1 - \omega_1) + X \cdot \omega_1$ is a linear polynomial, $L(X)$ has only one zero point in F. When λ_1 takes this point, we have $L(\lambda_1, \omega_1) = 0$, and the above equation naturally holds. The probability of this happening is $1/|F|$. If $L(\lambda_1, \omega_1) \neq 0$, then:

$$\langle L(\cdot, \omega_2, \dots, \omega_n), P_{\lambda_1} \rangle_{H_{n-1}} = \Lambda_0(\lambda_1)$$

[H24] gives:

$$\langle L(\cdot, \omega_2, \dots, \omega_n), P_{\lambda_1} - \Lambda_0(\lambda_1) \rangle_{H_{n-1}} = 0 \tag{4}$$

Here's a detailed derivation: Since $P_{\lambda_1} = P(\lambda_1, X_2, \dots, X_n)$, we have:

$$\begin{aligned} \langle L(\cdot, \omega_2, \dots, \omega_n), P_{\lambda_1} \rangle_{H_{n-1}} &= \sum_{\vec{x} \in H_{n-1}} L(\vec{x}, (\omega_2, \dots, \omega_n)) \cdot P(\lambda_1, \vec{x}) \\ &= P(\lambda_1, \omega_2, \dots, \omega_n) \end{aligned}$$

So the above equation becomes:

$$P(\lambda_1, \omega_2, \dots, \omega_n) = \Lambda_0(\lambda_1)$$

Let a function be $P'(X_2, \dots, X_n) = P(\lambda_1, X_2, \dots, X_n) - \Lambda_0(\lambda_1)$, its evaluation at point $(\omega_2, \dots, \omega_n)$ is $P'(\omega_2, \dots, \omega_n) = 0$, and $P'(\omega_2, \dots, \omega_n)$ can be expressed as:

$$\begin{aligned} P'(\omega_2, \dots, \omega_n) &= \langle L(\cdot, \omega_2, \dots, \omega_n), P'(\cdot) \rangle_{H_{n-1}} \\ &= \langle L(\cdot, \omega_2, \dots, \omega_n), P(\lambda_1, \cdot) - \Lambda_0(\lambda_1) \rangle_{H_{n-1}} \\ &= \langle L(\cdot, \omega_2, \dots, \omega_n), P_{\lambda_1} - \Lambda_0(\lambda_1) \rangle_{H_{n-1}} \\ &= 0 \end{aligned}$$

Therefore:

$$\langle L(\cdot, \omega_2, \dots, \omega_n), P_{\lambda_1} - \Lambda_0(\lambda_1) \rangle_{H_{n-1}} = 0$$

By linearity, we know:

$$\Lambda_0(\lambda_1) = (1 - \lambda_1) \cdot \Lambda_0(0) + \lambda_1 \cdot \Lambda_0(1) \tag{5}$$

At the same time:

$$f_{\lambda_1} = (1 - \lambda_1) \cdot f_{0,0} + \lambda_1 \cdot f_{0,1} \tag{6}$$

Subtracting (5) from (6), we get:

$$f_{\lambda_1} - \Lambda_0(\lambda_1) = (1 - \lambda_1) \cdot (f_{0,0} - \Lambda_0(0)) + \lambda_1 \cdot (f_{0,1} - \Lambda_0(1))$$

Let the new polynomial be:

$$f'_{\lambda_1} = (1 - \lambda_1) \cdot (f_{0,0} - \Lambda_0(0)) + \lambda_1 \cdot (f_{0,1} - \Lambda_0(1))$$

Following the idea of the correlated agreement theorem, according to the condition, if:

$$\Pr_{\lambda_1 \in F}[\Delta((1 - \lambda_1) f_{0,0} + \lambda_1 f_{0,1}, \mathcal{C}_1) \leq \theta] > \epsilon$$

That is, if f_{λ_1} is not more than θ distant from p_{λ_1} with a probability greater than a bound ϵ, then subtracting a number $\Lambda_0(\lambda_1)$ from both of them does not affect the distance between them. Therefore, f'_{λ_1} is not more than θ distant from $p'_{\lambda_1} = p_{\lambda_1} - \Lambda_0(\lambda_1)$.

Which encoding space does $p'_{\lambda_1} = p_{\lambda_1} - \Lambda_0(\lambda_1)$ belong to? We know that $p_{\lambda_1} \in \mathcal{P}_{n-1} = F[X]^{<2^{n-1}}$, and $\Lambda_0(\lambda_1)$ is essentially a number, so p'_{λ_1} is still in the $\mathcal{P}_{n-1}$ space. At the same time, we have already derived:

$$\langle L(\cdot, \omega_2, \dots, \omega_n), P_{\lambda_1} - \Lambda_0(\lambda_1) \rangle_{H_{n-1}} = 0$$

This indicates that the multilinear polynomial corresponding to p'_{λ_1} also satisfies such an inner product constraint. Therefore, we can say that p'_{λ_1} is in a subspace of $\mathcal{P}_{n-1}$, namely:

$$\mathcal{P}'_{n-1} = \{u(X) \in \mathcal{P}_{n-1} : \langle L(\cdot, \omega_2, \dots, \omega_n), U \rangle_{H_{n-1}} = 0\}$$

In the above equation, U is the multilinear polynomial corresponding to $u(X)$. Such a polynomial subspace can form a linear subcode $\mathcal{C}'_1$ of the encoding space $\mathcal{C}_1$. We can see that by extracting the linear term $\Lambda_i(X)$ from $q_i(X)$ and adding a sumcheck-like constraint, we find that the encoding space to be considered is a linear subspace of the original encoding space.

Now let's summarize the conclusions we've reached so far. Let $f'_{0,0} := f_{0,0} - \Lambda_0(0)$, $f'_{0,1} := f_{0,1} - \Lambda_0(1)$, we have:

$$f'_{\lambda_1} = (1-\lambda_1) \cdot f'_{0,0} + \lambda_1 \cdot f'_{0,1}$$

At the same time, f'_{λ_1} is not more than θ distant from $p'_{\lambda_1} = p_{\lambda_1} - \Lambda_0(\lambda_1)$ with a probability greater than ϵ, and $p'_{\lambda_1} \in \mathcal{P}'_{n-1}$, i.e.,

$$\Pr_{\lambda_1 \in F}[\Delta((1-\lambda_1)f'_{0,0} + \lambda_1 f'_{0,1}, \mathcal{C}'_1) \leq \theta] > \epsilon$$

[H24, Theorem 3] gives the correlated agreement theorem for linear subcodes, whose strict description will be introduced in the next section. The conclusion of this theorem states that there exist polynomials $p'_{0,0}$ and $p'_{0,1}$ from $\mathcal{P}'_{n-1}$, and $D' \subseteq D_1$, satisfying: 1. $|D'|/|D_1| \geq 1-\theta$, 2. $f'_{0,0}|_{D'} = p'_{0,0}|_{D'}$ and $f'_{0,1}|_{D'} = p'_{0,1}|_{D'}$.

Here, $\mathcal{P}'_{n-1}$ includes the sumcheck constraint. According to the definitions of $f'_{0,0}$ and $f'_{0,1}$, we can get:

$$f_{0,0} = f'_{0,0} + \Lambda_0(0)$$
$$f_{0,1} = f'_{0,1} + \Lambda_0(1)$$

Since $\Lambda_0(0)$ and $\Lambda_0(1)$ essentially represent numbers, according to conclusion 2, we have:

$$f_{0,0}(X)|_{D'} = p'_{0,0}(X)|_{D'} + \Lambda_0(0) = (p'_{0,0}(X) + \Lambda_0(0))|_{D'}$$
$$f_{0,1}(X)|_{D'} = p'_{0,1}(X)|_{D'} + \Lambda_0(1) = (p'_{0,1}(X) + \Lambda_0(1))|_{D'}$$

Let:

$$p_{0,0}(X) = p'_{0,0}(X) + \Lambda_0(0)$$
$$p_{0,1}(X) = p'_{0,1}(X) + \Lambda_0(1)$$

Therefore, $f_{0,0}(X)$ and $f_{0,1}(X)$ are consistent with $p_{0,0}(X)$ and $p_{0,1}(X)$ on D' respectively. From $p_{0,0}(X)$ and $p_{0,1}(X)$, we can obtain their corresponding multivariate polynomials $P_0, P_1 \in F[X_2, \ldots, X_n]$. Since $p'_{0,0}(X), p'_{0,1}(X) \in \mathcal{P}'_{n-1}$, their corresponding multilinear polynomials $P'_{0,0}, P'_{0,1}$ satisfy:

$$\langle L(\cdot, \omega_2, \ldots, \omega_n), P'_{0,0} \rangle_{H_{n-1}} = 0$$
$$\langle L(\cdot, \omega_2, \ldots, \omega_n), P'_{0,1} \rangle_{H_{n-1}} = 0$$

Therefore:

$$\begin{aligned}
&\langle L(\cdot, \omega_2, \ldots, \omega_n), P'_{0,0} + \Lambda_0(0) - \Lambda_0(0) \rangle_{H_{n-1}} = 0 \\
&\langle L(\cdot, \omega_2, \ldots, \omega_n), P'_{0,1} + \Lambda_0(1) - \Lambda_0(1) \rangle_{H_{n-1}} = 0 \\
\Rightarrow \\
&\langle L(\cdot, \omega_2, \ldots, \omega_n), P_0 - \Lambda_0(0) \rangle_{H_{n-1}} = 0 \\
&\langle L(\cdot, \omega_2, \ldots, \omega_n), P_1 - \Lambda_0(1) \rangle_{H_{n-1}} = 0 \\
\Rightarrow \\
&\langle L(\cdot, \omega_2, \ldots, \omega_n), P_0 \rangle_{H_{n-1}} = \Lambda_0(0) \\
&\langle L(\cdot, \omega_2, \ldots, \omega_n), P_1 \rangle_{H_{n-1}} = \Lambda_0(1)
\end{aligned}$$

Multiplying both sides by $L(0, \omega_1), L(1, \omega_1)$ respectively, and using $q_0(X) = L(X, \omega_1) \cdot \Lambda_0(X)$, we get:

$$\langle L((0,\cdot),\vec{\omega}), P_0\rangle_{H_{n-1}} = q_0(0)$$
$$\langle L((1,\cdot),\vec{\omega}), P_1\rangle_{H_{n-1}} = q_0(1)$$

This also shows that equations (2) and (3) hold, which implies that $P(X)$ corresponding to $p_0(X)$ satisfies the sumcheck constraint.

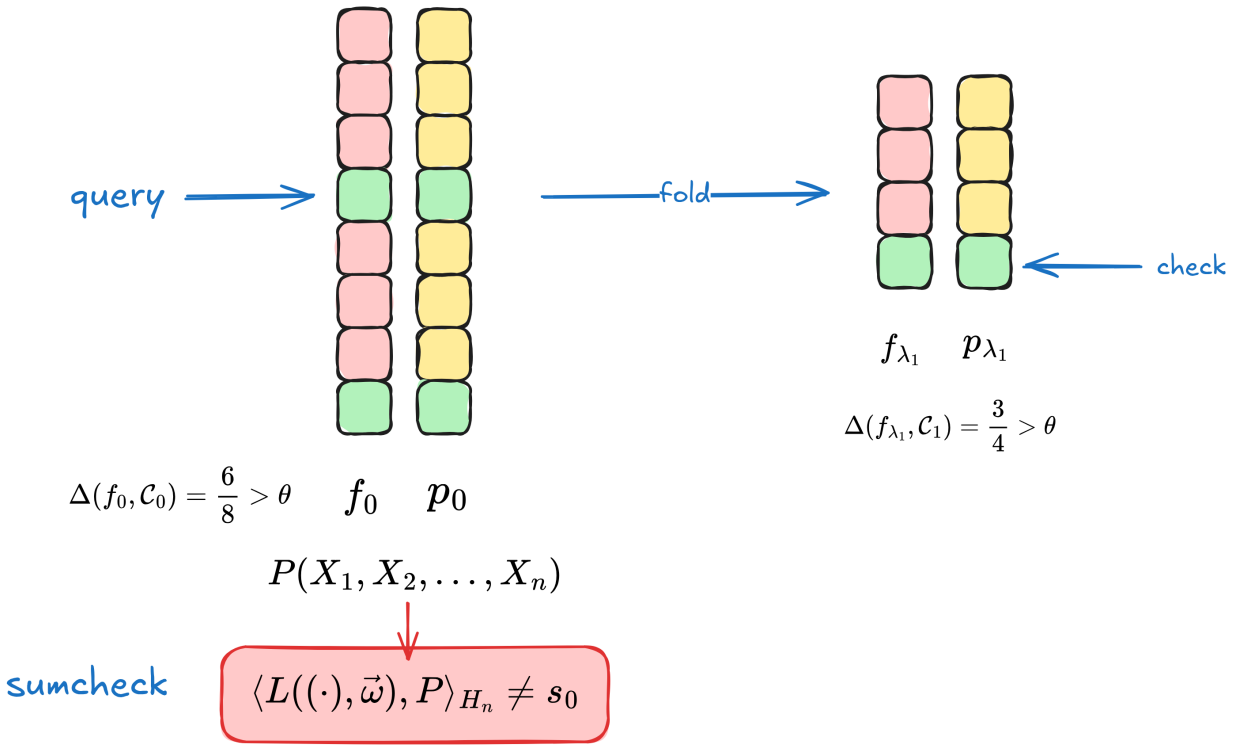
In summary, the soundness error in the commit phase can be analyzed following the above approach. The specific probability is given by the correlated agreement theorem. [H24, Theorem 1] gives the soundness error in the commit phase as:

1. Batching phase: $\varepsilon_{C_1} = \varepsilon(\mathcal{C}_0, M, 1, \theta)$.
2. Sumcheck and FRI-like folding phase: $\varepsilon_{C_2} = \sum_{i=1}^{n} \left(\frac{1}{|F|} + \varepsilon(\mathcal{C}_i, 1, B_i, \theta)\right)$, where $\frac{1}{|F|}$ is the additional probability introduced when simplifying the sumcheck constraint to make $L(X, \omega_i) = 0$.

The above $\varepsilon(\mathcal{C}_i, M_i, B_i, \theta)$ is given by the weighted correlated agreement theorem [H24, Theorem 4].

8.2.2 Query Phase

For a malicious Prover P^*, now excluding the case where it can luckily pass the Verifier's check in the Commit phase, after one folding, f_{λ_1} will be θ far from $\mathcal{C}_1$, or the sumcheck constraint will be incorrect.



For $\Delta(f_0, \mathcal{C}_0) > \theta$, since the Verifier will randomly select an x_0 from D_0 to check if the folding is correct, if it queries those points where f_{λ_1} and p_{λ_1} are consistent on D_1, it will pass the check. This proportion does not exceed $1 - \theta$. If the query is repeated s times, then the probability of P^* luckily passing the check does not exceed $(1 - \theta)^s$.

For the case where the sumcheck constraint is incorrect, the verifier will use the sumcheck protocol to check if the constraint is correct. Here, P^* cannot successfully cheat and will definitely be caught.

In summary, the soundness error in the query phase is $\varepsilon_{\text{query}} = (1 - \theta)^s$.

Therefore, we obtain the soundness error of the entire protocol given by [H24, Theorem 1]:

$$\begin{aligned}\varepsilon &< \varepsilon_{C_1} + \varepsilon_{C_2} + \varepsilon_{\text{query}} \\ &= \varepsilon(\mathcal{C}_0, M, 1, \theta) + \sum_{i=1}^{n} \left(\frac{1}{|F|} + \varepsilon(\mathcal{C}_i, 1, B_i, \theta) \right) + (1-\theta)^s\end{aligned}$$

> **typo** > I believe the condition $\theta = (1 + \frac{1}{2m}) \cdot \sqrt{\rho}$ given in [H24, Theorem 1] is incorrect. Based on the condition given in [H24, Theorem 4] later, it should be changed to $\theta = 1 - (1 + \frac{1}{2m}) \cdot \sqrt{\rho}$.

8.3 Correlated Agreement Theorems

This section introduces the correlated agreement theorem given by [BCIKS20], as well as the correlated agreement theorem for subcodes given by [H24] based on this.

First is the correlated agreement theorem given by [BCIKS20], which includes two theorems: one for the unique decoding bound and one for reaching the Johnson bound under list decoding. Some symbols have been changed. Let F denote a finite field, $\mathcal{C} = RS_k[F, D]$ denote a Reed-Solomon code over F, with evaluation domain D and rate $\rho = k/|D|$.

Theorem 3 [BCIKS20, Theorem 6.1] Suppose $\theta \leq \frac{1-\rho}{2}$. Let $f_0, f_1, ..., f_M \in F^D$ be functions from D to F. If

$$\frac{|\{z \in F : \Delta(f_0 + z \cdot f_1 + ... + z^M \cdot f_M, \mathcal{C}) \leq \theta\}|}{|F|} > \varepsilon$$

where

$$\varepsilon = M \cdot \frac{|D|}{|F|}$$

then for any $z \in F$, we have

$$\Delta(f_0 + z \cdot f_1 + ... + z^M \cdot f_M, \mathcal{C}) \leq \theta,$$

Moreover, there exist $p_0, ..., p_M \in \mathcal{C}$ such that for all $z \in F$,

$$\Delta(u_0 + zu_1 + \cdots + z_l u_l, v_0 + zv_1 + \cdots + z_l v_l) \leq \theta$$

In fact,

$$|\{x \in D : (u_0(x), ..., u_l(x)) \neq (v_0(x), ..., v_l(x))\}| \leq \theta |D|.$$

Theorem 4 [BCIKS20, Theorem 6.2] Let $f_0, f_1, ..., f_M \in F^D$ be functions from D to F. Let $m \geq 3$, define $\theta_0(\rho, m) := 1 - \sqrt{\rho} \cdot (1 + \frac{1}{2m})$, and let $\theta \leq \theta_0(\rho, m)$. If

$$\frac{|\{z \in F : \Delta(f_0 + z \cdot f_1 + ... + z^M \cdot f_M, \mathcal{C}) \leq \theta\}|}{|F|} > \varepsilon$$

where

$$\varepsilon = M \cdot \frac{(m + \frac{1}{2})^7}{3 \cdot \rho^{3/2}} \cdot \frac{|D|^2}{|F|}$$

then $f_0, f_1, \ldots, f_M$ are simultaneously θ-close to $\mathcal{C}_0$, i.e., there exist $p_0, \ldots, p_M \in \mathcal{C}$ such that

$$|\{x \in D : \forall 0 \leq i \leq M, f_i(x) = p_i(x)\}| \geq (1-\theta)|D|.$$

Theorem 3 and Theorem 4 give the correlated agreement theorems under unique decoding and list decoding, respectively. Although their formulations are somewhat different from those given in the previous section, they express the same meaning. Here, the specific expressions for ε are given.

In [H24], by analyzing the Guruswami-Sudan list decoder in the proof of the correlated agreement theorem in [BCIKS20], a correlated agreement theorem for subcodes under list decoding is obtained.

Theorem 5 [H24, Theorem 3] (Correlated Agreement for Subcodes) Let F be a finite field of arbitrary characteristic, $\mathcal{C} = RS_k[F, D]$ be a Reed-Solomon code over F with evaluation domain D and rate $\rho = k/|D|$. Let $\mathcal{C}'$ be a linear subcode of $\mathcal{C}$, generated by a subspace $\mathcal{P}'$ of polynomials from $F[X]^{<k}$. Given a proximity parameter $\theta = 1 - \sqrt{\rho} \cdot \left(1 + \frac{1}{2m}\right)$, where $m \geq 3$, and $f_0, f_1, \ldots, f_M \in F^D$ satisfying

$$\frac{|\{z \in F : \Delta(f_0 + z \cdot f_1 + \ldots + z^M \cdot f_M, \mathcal{C}') < \theta\}|}{|F|} > \varepsilon,$$

where

$$\varepsilon = M \cdot \frac{(m + \frac{1}{2})^7}{3 \cdot \rho^{3/2}} \cdot \frac{|D|^2}{|F|},$$

then there exist polynomials $p_0, p_1, \ldots, p_M \in \mathcal{P}'$, and a set $D' \subseteq D$, satisfying 1. $|D'|/|D| \geq 1 - \theta$ 2. $f_0, f_1, \ldots, f_M$ are consistent with $p_0, p_1, \ldots, p_M$ respectively on D'.

Comparing Theorem 5 and Theorem 4, in terms of the expression of ε, their forms can be said to be consistent. The difference is that Theorem 5 is considered in a linear subcode of the Reed-Solomon encoding space. It's natural to conjecture that for unique decoding, there is also a similar result to Theorem 4 for subcodes.

Conjecture 6 Let F be a finite field of arbitrary characteristic, $\mathcal{C} = RS_k[F, D]$ be a Reed-Solomon code over F with evaluation domain D and rate $\rho = k/|D|$. Let $\mathcal{C}'$ be a linear subcode of $\mathcal{C}$, generated by a subspace $\mathcal{P}'$ of polynomials from $F[X]^{<k}$. Let $\theta \leq \frac{1-\rho}{2}$, and $f_0, f_1, \ldots, f_M \in F^D$ satisfying

$$\frac{|\{z \in F : \Delta(f_0 + z \cdot f_1 + \ldots + z^M \cdot f_M, \mathcal{C}') < \theta\}|}{|F|} > \varepsilon,$$

where

$$\varepsilon = M \cdot \frac{|D|}{|F|}$$

then there exist polynomials $p_0, p_1, \ldots, p_M \in \mathcal{P}'$, and a set $D' \subseteq D$, satisfying 1. $|D'|/|D| \geq 1 - \theta$ 2. $f_0, f_1, \ldots, f_M$ are consistent with $p_0, p_1, \ldots, p_M$ respectively on D'.

Similar to the soundness proof of the batch FRI protocol in [BCIKS20], which used a weighted version of the correlated agreement theorem, [H24] also gives a weighted correlated agreement theorem for the batch Basefold protocol. According to the description in [H24], let's first explain the meaning of "weighted". Given a sub-probability measure μ on D and $f \in F^D$, we write

$$\text{agree}_{\mu}(f, \mathcal{C}') \geq 1 - \theta$$

to mean that there exists a polynomial $p(X)$ in $\mathcal{P}'$ such that $\mu(\{x \in D : f(x) = p(x)\}) \geq 1 - \theta$. This means using the measure μ to calculate the set of x in D that satisfy $f(x) = p(x)$. For completeness, here's the weighted correlated agreement theorem for list decoding given in [H24].

Theorem 7 [H24, Theorem 4] (Weighted Correlated Agreement for Subcodes) Let C' be a linear subcode of $RS_k[F, D]$, and choose $\theta = 1 - \sqrt{\rho} \cdot \left(1 + \frac{1}{2m}\right)$, for some integer $m \geq 3$, where $\rho = k/|D|$. Assume a density function $\delta : D \to [0, 1] \cap \mathbb{Q}$ with common denominator $B \geq 1$, i.e. for all x in D,

$$\delta(x) = \frac{m_x}{B},$$

for an integer value $m_x \in [0, B]$, and let μ be the sub-probability measure with density δ, defined by $\mu(\{x\}) = \delta(x)/|D|$. If for $f_0, f_1, \ldots, f_M \in F^D$,

$$\frac{\{z \in F : \text{agree}_\mu(f_0 + z \cdot f_1 + \ldots + z^M \cdot f_M, \mathcal{C}') \geq 1 - \theta\}}{|F|} > \varepsilon(\mathcal{C}, M, B, \theta)$$

where

$$\varepsilon(\mathcal{C}, M, B, \theta) = \frac{M}{|F|} \cdot \frac{(m + \frac{1}{2})}{\sqrt{\rho}} \cdot \max\left(\frac{(m + \frac{1}{2})^6}{3 \cdot \rho} \cdot |D|^2, 2 \cdot (B \cdot |D| + 1)\right),$$

then there exist polynomials $p_0(X), p_1(X), \ldots, p_M(X)$ belonging to the subcode $\mathcal{C}'$, and a set A with $\mu(A) \geq 1 - \theta$ on which $f_0, f_1, \ldots, f_M$ coincide with $p_0(X), p_1(X), \ldots, p_M(X)$, respectively.

The advantage of the weighted correlated agreement theorem is that in the process of proving the protocol's soundness, μ can be defined by oneself, increasing flexibility. The details of the soundness proof for the Basefold protocol will be introduced in the next article.

8.4 References

- [H24] Ulrich Haböck. "Basefold in the List Decoding Regime." *Cryptology ePrint Archive*(2024).
- [ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. "BaseFold: efficient field-agnostic polynomial commitment schemes from foldable codes." Annual International Cryptology Conference. Cham: Springer Nature Switzerland, 2024.
- [BCIKS20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity Gaps for Reed–Solomon Codes. In *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science*, pages 900–909, 2020.
- [ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. "WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification."*Cryptology ePrint Archive*(2024).

Chapter 9

Note on Soundness Proof of Basefold under List Decoding

- Jade Xie jade@secbit.io
- Yu Guo yu.guo@secbit.io

In the previous article "Overview of Basefold's Soundness Proof under List Decoding", we outlined the approach to the soundness proof in the [H24] paper. This article will delve deeper into the proof details following this approach, focusing mainly on the proof of [H24, Lemma 1], which demonstrates the soundness error of the Basefold protocol in the commit phase.

Lemma 1 [H24, Lemma 1] (Soundness commit phase). Take a proximity parameter $\theta = 1 - \left(1 + \frac{1}{2 \cdot m}\right) \cdot \sqrt{\rho}$, with $m \geq 3$. Suppose that a (possibly computationally unbounded) algorithm P^* succeeds the commitment phase with $r \geq 0$ rounds with probability larger than

$$\varepsilon_C = \varepsilon_0 + \varepsilon_1 + ... + \varepsilon_r,$$

where $\varepsilon_0 = \varepsilon(\mathcal{C}_i, M, \theta)$ is the soundness error from Theorem 3, and

$$\varepsilon_i := \varepsilon(\mathcal{C}_i, 1, B_i, \theta) + \frac{1}{|F|},$$

with $\varepsilon(\mathcal{C}_i, 1, B_i, \theta)$ being the soundness error from Theorem 4, where $B_i = \frac{|D|}{|D_i|} = 2^i$. Then $(g_0, ..., g_M)$ belongs to $\mathcal{R}$.

[H24, Theorem 3] mentioned in the lemma is the correlated agreement theorem for subcodes under list decoding, while [H24, Theorem 4] is the weighted version of [H24, Theorem 3].

The relation $\mathcal{R}$ implies that P^* has not cheated, indicating that the committed polynomials $(g_0, ..., g_M)$ are both within distance θ from the corresponding encoding space and consistent with the committed values $v_0, ..., v_M$ at the query point $\vec{\omega} = (\omega_1, ..., \omega_n)$, i.e.,

$$\mathcal{R} = \left\{ \begin{array}{c} \exists p_0, ..., p_M \in \mathcal{F}[X]^{<2^n} \text{ s.t.} \\ (g_0, ..., g_M) : d\left((g_0, ..., g_M), (p_0, ..., p_M)\right) < \theta \\ \wedge \bigwedge_{k=0}^{M} P_k\left(\omega_1, ..., \omega_n\right) = v_k \end{array} \right\}.$$

Lemma 1 states that if P^*'s success probability in the commit phase exceeds ε_C, we can trust that P^* has not cheated, and the claimed relation $\mathcal{R}$ holds.

Here, we need to mathematically define what it means for P^* to succeed in the $0 \leq r \leq n$ round of the commit phase. This is the concept of α-good given in the [H24] paper. From the protocol itself, P^*'s success means that the verifier receives $f_0, \Lambda_0, f_1, \Lambda_1, f_2, \Lambda_2, \dots, \Lambda_{r-1}, f_r$ from P^*, then performs checks: one is the sumcheck, and the other is randomly selecting x in D_0 to verify that the FRI folding is correct. First, the parameter $\alpha = 1 - \theta \in (0, 1)$, i.e.,

$$\alpha = \left(1 + \frac{1}{2 \cdot m}\right) \cdot \sqrt{\rho}$$

Let $\mathcal{F}_i$ represent the polynomial space corresponding to the Reed-Solomon code $\mathcal{C}_i = \mathrm{RS}_{2^{n-i}}[F, D_i]$, where D_i is the result of applying the mapping π to D i times, i.e., $D_i = \pi^i(D), i = 0, \dots, n$. Therefore, the polynomial subspace corresponding to $\mathcal{C}'_i \subseteq \mathcal{C}_i$ is defined as

$$\mathcal{F}'_i = \{p(X) \in \mathcal{F}_i : P(\omega_{i+1}, \dots, \omega_n) = 0\}.$$

1. The sumcheck is correct. This means there exists $p_r(X) \in \mathcal{F}_r$, with corresponding multivariate polynomial P_r satisfying

$$L((\omega_1, \dots, \omega_r), (\lambda_1, \dots, \lambda_r)) \cdot P_r(\omega_1, \dots, \omega_n) = q_{r-1}(\lambda_r)$$

   Based on the relationship between $q_i(X)$ and $\Lambda_i(X)$, we can deduce that P_r needs to satisfy

$$\begin{aligned} L((\omega_1, \dots, \omega_r), (\lambda_1, \dots, \lambda_r)) \cdot P_r(\omega_{r+1}, \dots, \omega_n) &= q_{r-1}(\lambda_r) \\ &= L((\omega_1, \dots, \omega_r), (\lambda_1, \dots, \lambda_r)) \cdot \Lambda_{r-1}(\lambda_r) \end{aligned} \tag{1}$$

2. The folding is correct. It needs to satisfy

$$\left| \left\{ x \in D_0 : \begin{array}{c} (f_0, \dots, f_r) \text{ satisfy all folding checks along } x \\ \wedge f_r(\pi^r(x)) = p_r(\pi^r(x)) \end{array} \right\} \right| \geq \alpha \cdot |D_0| \tag{2}$$

   Here, only when the proportion of x in D_0 satisfying the folding check is greater than α, after mapping through π^r, will the verifier pass in the end.

When conditions 1 and 2 are met, we say that such $(f_0, \Lambda_0, f_1, \Lambda_1, f_2, \Lambda_2, \dots, \Lambda_{r-1}, f_r)$ is α-good for $(\lambda_0, \dots, \lambda_r)$.

9.1 Proof of Lemma 1

The proof of Lemma 1 uses mathematical induction. First, it proves that the conclusion holds when $r = 0$, using [H24, Theorem 3]. Then, assuming Lemma 1 holds for $0 \leq r < n$, it proves that the conclusion also holds for $r + 1$. This process uses the weighted [H24, Theorem 4], following a similar approach to the one introduced in the previous article. For example, in the $r + 1$ round, starting with the conditions satisfied by f_{r+1} obtained after folding with the random number λ_{r+1}, which is close to the corresponding encoding space and satisfies the sumcheck constraint, we first deduce that the corresponding f'_{r+1} satisfies some conditions. This allows us to use the correlated agreement theorem for subcodes. Applying the theorem's conclusion, we can then derive the properties satisfied by $f_{r,0}$ and $f_{r,1}$ before folding, and from this, deduce the properties satisfied by f_r. At this point, applying the induction hypothesis, we can obtain that the conditions of the lemma are satisfied in the r-th round, thus proving that the conclusion holds in the r-th round, which in turn proves that the lemma holds in the $(r + 1)$-th round.

Proof: First, prove that the lemma holds when $r = 0$. The given condition is that P^*'s success probability in the commit phase is greater than $\varepsilon(\mathcal{C}_0, M, \theta)$, and we want to prove that $(g_1, \dots, g_M) \in \mathcal{R}$. According to the condition and the definition of α-good, we can deduce that with a probability greater than $\varepsilon(\mathcal{C}_0, M, \theta)$, the f_0 provided by P^* is α-good for λ_0. Then, considering the polynomials $g'_k = g_k - v_k$ before folding,

the probability that they are within distance θ from the corresponding subcode $\mathcal{C}'_0 \subseteq \mathcal{C}_0$ (which means the consistent part is greater than α) is

$$\Pr\left[\lambda_0 : \exists p'_0 \in \mathcal{F}'_0 \text{ s.t. agree}\left(\sum_{k=0}^{M} g'_k \cdot \lambda_0^k, p'_0(X)\right) \geq \alpha\right] > \varepsilon(\mathcal{C}_0, M, \theta)$$

The purpose of considering polynomials $g'_k = g_k - v_k$ instead of g_k is to allow our analysis to enter the scope of the linear subcode $\mathcal{C}'_0$, so we can use [H24, Theorem 3] to obtain polynomials

$$p'_0(X), \ldots, p'_M(X) \in \mathcal{F}'_0$$

and a set $D'_0 \subseteq D$, satisfying

1. $|D'_0|/|D| \geq \alpha$
2. $p'_k(X)|_{D'_0} = g'_k(X)|_{D'_0}$

Now that we have found polynomials $p'_0(X), \ldots, p'_M(X)$, for polynomials

$$p'_0(X) + v_0, \ldots, p'_M(X) + v_M \in \mathcal{F}_0$$

they satisfy

$$(p'_k(X) + v_k)|_{D'_0} = (g'_k(X) + v_k)|_{D'_0} = g_k(X)|_{D'_0} \quad 0 \leq k \leq M$$

The multilinear polynomial $P_k \in F[X_1, \ldots, X_n]$ corresponding to $p'_0(X) + v_0$ also satisfies $P_k(\vec{\omega}) = v_k$, therefore $(g_1, \ldots, g_M) \in \mathcal{R}$.

Now assume the lemma holds for $0 \leq r < n$, and we want to prove that it still holds for $r + 1$. According to the conditions of the lemma, in the $(r + 1)$-th round, P^*'s success probability in the commit phase exceeds $(\varepsilon_0 + \varepsilon_1 + \ldots + \varepsilon_r) + \varepsilon_{r+1}$. Let $\mathfrak{T}$ be the set composed of $\mathrm{tr}_r = (\lambda_0, f_0, \Lambda_0, \ldots, \lambda_r, f_r, \Lambda_r)$. Therefore, under the condition

$$\Pr[\mathfrak{T}] > \varepsilon_0 + \ldots + \varepsilon_r$$

P^*'s success probability is greater than ε_{r+1}, i.e.,

$$\Pr\left[\lambda_{r+1} : \begin{array}{c} \exists f_{r+1} \text{ s.t. } (\lambda_0, f_0, \Lambda_0, \ldots, \lambda_r, f_r, \Lambda_r, f_{r+1}) \\ \text{is } \alpha\text{-good for } (\lambda_0, \ldots, \lambda_{r+1}) \end{array}\right] > \varepsilon_{r+1}$$

From the definition of α-good, we can deduce that for λ_{r+1} satisfying α-good, there exists a polynomial $p_{r+1} \in \mathcal{F}_{r+1}$ satisfying the sumcheck constraint, such that

$$\mathrm{agree}_{\nu_r}((1 - \lambda_{r+1}) \cdot f_{r,0} + \lambda_{r+1} \cdot f_{r,1}, p_{r+1}) \geq \alpha \tag{3}$$

Here, ν_r is a sub-probability measure with density function defined as, for $y \in D_{r+1}$

$$\delta_r(y) := \frac{|\{x \in \pi^{-(r+1)}(y) : (f_0, \ldots, f_r) \text{ satisfies all folding checks along } x\}|}{|\pi^{-(r+1)}(y)|}$$

Here's an explanation of what equation (3) essentially represents: it's equivalent to equation (2) in the definition of α-good. According to the definition of the agree function, equation (3) is equivalent to

$$\frac{\nu_r(\{y \in D_{r+1} : ((1-\lambda_{r+1}) \cdot f_{r,0} + \lambda_{r+1} \cdot f_{r,1})(y) = p_{r+1}(y)\})}{|D_{r+1}|} \geq \alpha$$

First, let's form a set S_{r+1} consisting of y in D_{r+1} that satisfy the folding relation, then calculate this set using the ν_r function.

$$\begin{aligned}
\nu_r(S_{r+1}) &= \sum_{y \in S_{r+1}} \delta_r(y) \\
&= \sum_{y \in S_{r+1}} \frac{|\{x \in \pi^{-(r+1)}(y) : (f_0, \ldots, f_r) \text{ satisfies all folding checks along } x\}|}{|\pi^{-(r+1)}(y)|} \\
&= \sum_{y \in S_{r+1}} \frac{|\{x \in \pi^{-(r+1)}(y) : (f_0, \ldots, f_r) \text{ satisfies all folding checks along } x\}|}{2^{r+1}} \\
&:= \sum_{y \in S_{r+1}} \frac{|S_{y,0}|}{2^{r+1}} \\
&= \frac{\sum_{y \in S_{r+1}} |S_{y,0}|}{2^{r+1}}
\end{aligned}$$

Therefore

$$\begin{aligned}
\text{agree}_{\nu_r}((1-\lambda_{r+1}) \cdot f_{r,0} + \lambda_{r+1} \cdot f_{r,1}, p_{r+1}) &= \frac{\nu_r(S_{r+1})}{|D_{r+1}|} \\
&= \frac{\sum_{y \in S_{r+1}} |S_{y,0}|}{2^{r+1} \cdot |D_{r+1}|} \\
&= \frac{\sum_{y \in S_{r+1}} |S_{y,0}|}{|D_0|}
\end{aligned}$$

The numerator $\sum_{y \in S_{r+1}} |S_{y,0}|$ in the above equation represents the number of points in D_0 that satisfy the $(r+1)$-th folding correctly, and also pass the folding checks for $(f_0, \ldots, f_r)$. Equation (3) becomes

$$\sum_{y \in S_{r+1}} |S_{y,0}| \geq \alpha \cdot |D_0|$$

This is completely consistent with equation (2) in the definition of α-good. Next, following the soundness proof approach introduced in the previous article, since the multilinear polynomial P_{r+1} corresponding to $p_{r+1}(X)$ satisfies the sumcheck constraint, it satisfies

$$\begin{aligned}
L((\omega_1, \ldots, \omega_{r+1}), (\lambda_1, \ldots, \lambda_{r+1})) \cdot P_{r+1}(\omega_{r+2}, \ldots, \omega_n) &= q_r(\lambda_{r+1}) \\
&= L((\omega_1, \ldots, \omega_{r+1}), (\lambda_1, \ldots, \lambda_{r+1})) \cdot \Lambda_r(\lambda_{r+1})
\end{aligned}$$

This leads to

$$\begin{aligned}
L((\omega_1, \ldots, \omega_r), (\lambda_1, \ldots, \lambda_r)) \cdot L(\omega_{r+1}, \lambda_{r+1}) \cdot P_{r+1}(\omega_{r+2}, \ldots, \omega_n) & \\
= L((\omega_1, \ldots, \omega_r), (\lambda_1, \ldots, \lambda_r)) \cdot L(\omega_{r+1}, \lambda_{r+1}) \cdot \Lambda_r(\lambda_{r+1}) &
\end{aligned}$$

For the choice of λ_{r+1}, there is a $1/|F|$ probability that $L(\omega_{r+1}, \lambda_{r+1}) = 0$, making the above equation hold. Therefore, with a probability exceeding

$$\varepsilon_{r+1} - \frac{1}{|F|} = \varepsilon(\mathcal{C}_{i+1}, 1, B_{r+1}, \theta)$$

polynomials $p'_{r+1} = p_{r+1} - \Lambda_r(\lambda_{r+1}) \in \mathcal{F}'_{r+1}$, and $f'_{r,0} = f_{r,0} - \Lambda_r(0)$, $f'_{r,1} = f_{r,1} - \Lambda_r(1)$ satisfy

$$\text{agree}_{\nu_r}((1-\lambda_{r+1}) \cdot f'_{r,0} + \lambda_{r+1} \cdot f'_{r,1}, p'_{r+1}) \geq \alpha$$

The above satisfied condition can be written as

$$\Pr\left[\lambda_{r+1} : \begin{array}{c} \exists p'_{r+1} \in \mathcal{F}'_{r+1} \text{ s.t.} \\ \text{agree}_{\nu_r}((1-\lambda_{r+1}) \cdot f'_{r,0} + \lambda_{r+1} \cdot f'_{r,1}, p'_{r+1}) \geq \alpha \end{array}\right] > \varepsilon(\mathcal{C}_{i+1}, 1, B_{r+1}, \theta)$$

This also satisfies the conditions of the weighted correlated agreement theorem [H24, Theorem 4], so we can obtain polynomials $p'_{r,0}(X), p'_{r,1}(X) \in \mathcal{F}'_{r+1}$, and a set $A_{r+1} \subseteq D_{r+1}$ satisfying: 1. $\nu_r(A_{r+1}) \geq 1 - \theta$ 2. $p'_{r,0}(X)|_{A_{r+1}} = f'_{r,0}(X)|_{A_{r+1}}$, $p'_{r,1}(X)|_{A_{r+1}} = f'_{r,1}(X)|_{A_{r+1}}$

Now that we have found polynomials $p'_{r,0}(X), p'_{r,1}(X)$, there exist polynomials

$$p_{r,0}(X) = p'_{r,0}(X) + \Lambda_r(0), \quad p_{r,1}(X) = p'_{r,1}(X) + \Lambda_r(1) \in \mathcal{F}_{r+1}$$

and

$$f_{r,0}(X) = f'_{r,0}(X) + \Lambda_r(0), \quad f_{r,1}(X) = f'_{r,1}(X) + \Lambda_r(1)$$

According to conclusion 2 given by correlated agreement, we can get

$$p_{r,0}(X)|_{A_{r+1}} = f_{r,0}(X)|_{A_{r+1}}, \quad p_{r,1}(X)|_{A_{r+1}} = f_{r,1}(X)|_{A_{r+1}}$$

For the multilinear polynomials $P_{r,0}$ and $P_{r,1}$ corresponding to $p_{r,0}(X), p_{r,1}(X)$, according to the definition of $\mathcal{F}'_r$, we can get

$$\begin{aligned} P_{r,0}(\omega_{r+2}, \dots, \omega_n) &= \Lambda_r(0) \\ P_{r,1}(\omega_{r+2}, \dots, \omega_n) &= \Lambda_r(1) \end{aligned}$$

Obtain $A_r = \pi^{-1}(A_{r+1}) \subseteq D_r$ by inverse mapping the points in set A_{r+1} through π. At these points, f_r must be consistent with

$$p_r(X) = p_{r,0}(X^2) + X \cdot p_{r,1}(X^2) \in \mathcal{F}_r$$

For the multilinear polynomial P_r corresponding to $p_r(X)$, it satisfies

$$\begin{aligned} P_r(\omega_{r+1}, \omega_{r+2}, \dots, \omega_n) &= (1-\omega_{r+1}) \cdot P_{r,0}(\omega_{r+2}, \dots, \omega_n) + \omega_{r+1} \cdot P_{r,1}(\omega_{r+2}, \dots, \omega_n) \\ &= (1-\omega_{r+1}) \cdot \Lambda_r(0) + \omega_{r+1} \cdot \Lambda_r(1) \\ &= L(\omega_{r+1}, 0) \cdot \Lambda_r(0) + L(\omega_{r+1}, 1) \cdot \Lambda_r(1) \end{aligned}$$

From this, we can conclude that the sumcheck in the r-th round is satisfied:

$$
\begin{aligned}
&L(\omega_1, \dots, \omega_r, \lambda_1, \dots, \lambda_r) \cdot P_r(\omega_{r+1}, \omega_{r+2}, \dots, \omega_n) \\
&\qquad = L(\omega_1, \dots, \omega_r, \lambda_1, \dots, \lambda_r) \cdot L(\omega_{r+1}, 0) \cdot \Lambda_r(0) \\
&\qquad\quad + L(\omega_1, \dots, \omega_r, \lambda_1, \dots, \lambda_r) \cdot L(\omega_{r+1}, 1) \cdot \Lambda_r(1) \\
&\qquad = q_r(0) + q_r(1) \\
&\qquad = q_{r-1}(\lambda_r)
\end{aligned}
$$

Now that we have obtained that the sumcheck in the r-th round is satisfied, we need to consider whether the folding relation is satisfied. Consider $x \in \pi^{-1}(A_r)$, we have

$$
\begin{aligned}
&\frac{|\{x \in \pi^{-r}(A_r) : \text{all folding checks hold for } f_0, \dots, f_r\}|}{|D_0|} \\
&= \frac{1}{|D_0|} \cdot \sum_{y \in A_{r+1}} \delta(y) \cdot |\pi^{-(r+1)}(y)| \\
&= \frac{2^{r+1}}{|D_0|} \cdot \sum_{y \in A_{r+1}} \delta(y) \\
&= \frac{1}{|D_{r+1}|} \cdot \sum_{y \in A_{r+1}} \delta(y) \\
&= \nu_r(A_{r+1})
\end{aligned}
$$

We have already obtained $\nu_r(A_{r+1}) \geq \alpha$ through the correlated agreement theorem, so the proportion of x in D_0 that can satisfy the folding check exceeds α. Combining the sumcheck constraint and folding relation in the r-th round, we get that $(f_0, \Lambda_0, \dots, f_r, \Lambda_r)$ is α-good for $(\lambda_0, \dots, \lambda_r)$. Since the probability of generating such a trace set is

$$
\Pr[\mathfrak{T}] > \varepsilon_0 + \dots + \varepsilon_r
$$

it satisfies the conditions of the lemma. By the induction hypothesis, the lemma holds in the r-th round, so we can conclude that $(g_0, \dots, g_M) \in \mathcal{R}$. This proves that the lemma also holds in the $(r+1)$-th round. Thus, the lemma is proved. $\square$

9.2 References

- [H24] Ulrich Haböck. "Basefold in the List Decoding Regime." *Cryptology ePrint Archive*(2024).